

A Comprehensive Overview of Large Language Models

Humza Naveed^a, Asad Ullah Khan^{b,*}, Shi Qiu^{c,*}, Muhammad Saqib^{d,e,*}, Saeed Anwar^{f,g}, Muhammad Usman^{f,g}, Naveed Akhtar^{h,j}, Nick Barnesⁱ, Ajmal Mian^j

^aThe University of Sydney, Sydney, Australia

^bUniversity of Engineering and Technology (UET), Lahore, Pakistan

^cThe Chinese University of Hong Kong (CUHK), HKSAR, China

^dUniversity of Technology Sydney (UTS), Sydney, Australia

^eCommonwealth Scientific and Industrial Research Organisation (CSIRO), Sydney, Australia

^fKing Fahd University of Petroleum and Minerals (KFUPM), Dhahran, Saudi Arabia

^gSDAIA-KFUPM Joint Research Center for Artificial Intelligence (JRC AI), Dhahran, Saudi Arabia

^hThe University of Melbourne (UoM), Melbourne, Australia

ⁱAustralian National University (ANU), Canberra, Australia

^jThe University of Western Australia (UWA), Perth, Australia

Abstract

Large Language Models (LLMs) have recently demonstrated remarkable capabilities in natural language processing tasks and beyond. This success of LLMs has led to a large influx of research contributions in this direction. These works encompass diverse topics such as architectural innovations, better training strategies, context length improvements, fine-tuning, multi-modal LLMs, robotics, datasets, benchmarking, efficiency, and more. With the rapid development of techniques and regular breakthroughs in LLM research, it has become considerably challenging to perceive the bigger picture of the advances in this direction. Considering the rapidly emerging plethora of literature on LLMs, it is imperative that the research community is able to benefit from a concise yet comprehensive overview of the recent developments in this field. This article provides an overview of the literature on a broad range of LLM-related concepts. Our self-contained comprehensive overview of LLMs discusses relevant background concepts along with covering the advanced topics at the frontier of research in LLMs. This review article is intended to provide not only a systematic survey but also a quick, comprehensive reference for the researchers and practitioners to draw insights from extensive, informative summaries of the existing works to advance the LLM research.

Keywords:

Large Language Models, LLMs, chatGPT, Augmented LLMs, Multimodal LLMs, LLM training, LLM Benchmarking

1. Introduction

Language plays a fundamental role in facilitating communication and self-expression for humans and their interaction with machines. The need for generalized models stems from the growing demand for machines to handle complex language tasks, including translation, summarization, information retrieval, conversational interactions, etc. Recently, significant breakthroughs have been witnessed in language models, primarily attributed to transformers [1], increased computational capabilities, and the availability of large-scale training data. These developments have brought about a revolutionary transformation by enabling the creation of LLMs that can approximate human-level performance on various tasks [2, 3]. Large

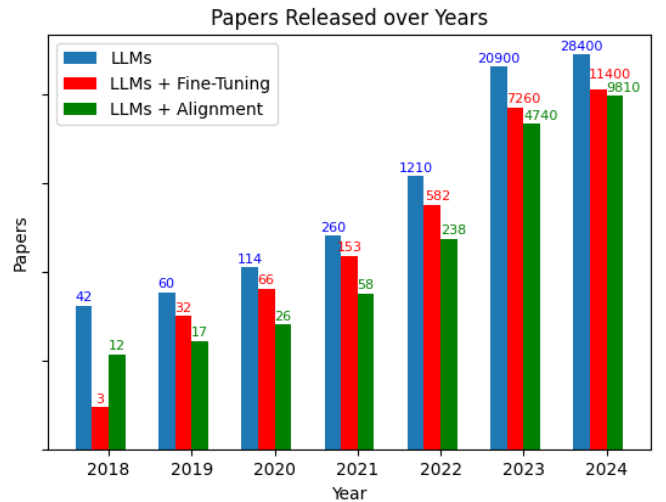


Figure 1: The trend of papers released over the years containing keywords "Large Language Model", "Large Language Model + Fine-Tuning", and "Large Language Model + Alignment".

*Equal contribution

Email addresses: humza_naveed@yahoo.com (Humza Naveed), aukhane@gmail.com (Asad Ullah Khan), shiqiu@cse.cuhk.edu.hk (Shi Qiu), muhammad.saqib@data61.csiro.au (Muhammad Saqib), saeed.anwar@kfupm.edu.sa (Saeed Anwar), muhammad.usman@kfupm.edu.sa (Muhammad Usman), naveed.akhtar1@unimelb.edu.au (Naveed Akhtar), nick.barnes@anu.edu.au (Nick Barnes), ajmal.mian@uwa.edu.au (Ajmal Mian)

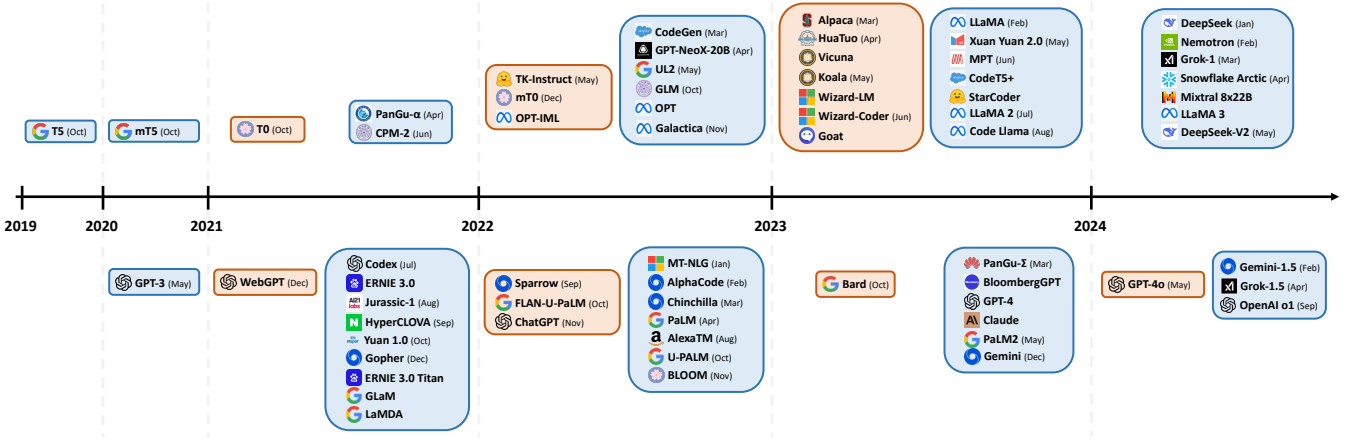


Figure 2: Chronological display of LLM releases: blue cards represent ‘pre-trained’ models, while orange cards correspond to ‘instruction-tuned’ models. Models on the upper half signify open-source availability, whereas those on the bottom are closed-source. The chart illustrates the increasing trend towards instruction-tuned and open-source models, highlighting the evolving landscape and trends in natural language processing research.

Language Models (LLMs) have emerged as cutting-edge artificial intelligence systems that can process and generate text with coherent communication [4] and generalize to multiple tasks [5, 6].

The historical progress in natural language processing (NLP) evolved from statistical to neural language modeling and then from pre-trained language models (PLMs) to LLMs. While conventional language modeling (LM) trains task-specific models in supervised settings, PLMs are trained in a self-supervised setting on a large corpus of text [7, 8, 9] with the aim of learning a generic representation that is shareable among various NLP tasks. After fine-tuning for downstream tasks, PLMs surpass the performance gains of traditional language modeling (LM). The larger PLMs bring more performance gains, which has led to the transitioning of PLMs to LLMs by significantly increasing model parameters (tens to hundreds of billions) [10] and training dataset (many GBs and TBs) [10, 11]. Following this development, numerous LLMs have been proposed in the literature [10, 11, 12, 6, 13, 14, 15]. An increasing trend in the number of released LLMs and names of a few significant LLMs proposed over the years are shown in Fig 1 and Fig 2, respectively.

The early work on LLMs, such as T5 [10] and mT5 [11] employed transfer learning until GPT-3 [6] showed LLMs are zero-shot transferable to downstream tasks without fine-tuning. LLMs accurately respond to task queries when prompted with task descriptions and examples. However, pre-trained LLMs fail to follow user intent and perform worse in zero-shot settings than in few-shot. Fine-tuning them with task instructions data [16, 17, 18, 19] and aligning with human preferences [20, 21] enhances generalization to unseen tasks, improving zero-shot performance significantly and reducing misaligned behavior.

In addition to better generalization and domain adaptation, LLMs appear to have emergent abilities, such as reasoning, planning, decision-making, in-context learning, answering in zero-shot settings, etc. These abilities are known to be acquired by them due to their gigantic scale even when the pre-trained LLMs are not trained specifically to possess these attributes [22, 23, 24]. Such abilities have led LLMs to be widely adopted in diverse settings, including multi-modal, robotics,

tool manipulation, question answering, autonomous agents, etc. Various improvements have also been suggested in these areas either by task-specific training [25, 26, 27, 28, 29, 30, 31] or better prompting [32].

The LLMs abilities to solve diverse tasks with human-level performance come at the cost of slow training and inference, extensive hardware requirements, and higher running costs. Such requirements have limited their adoption and opened up opportunities to devise better architectures [15, 33, 34, 35] and training strategies [36, 37, 21, 38, 39, 40, 41]. Parameter efficient tuning [38, 41, 40], pruning [42, 43], quantization [44, 45], knowledge distillation, and context length interpolation [46, 47, 48, 49] among others are some of the methods widely studied for efficient LLM utilization.

Due to the success of LLMs on a wide variety of tasks, the research literature has recently experienced a large influx of LLM-related contributions. Researchers have organized the LLMs literature in surveys [50, 51, 52, 53], and topic-specific surveys in [54, 55, 56, 57, 58]. In contrast to these surveys, our contribution focuses on providing a comprehensive yet concise overview of the general direction of LLM research. This article summarizes architectural and training details of pre-trained LLMs and delves deeper into the details of concepts like fine-tuning, multi-modal LLMs, augmented LLMs, datasets, evaluation, applications, challenges, and others to provide a self-contained comprehensive overview. Our key contributions are summarized as follows.

- We present a survey on the developments in LLM research, providing a concise, comprehensive overview of the direction.
- We present extensive summaries of pre-trained models that include fine-grained details of architecture and training details.
- We summarize major findings of the popular contributions and provide a detailed discussion on the key design and development aspects of LLMs to help practitioners effectively leverage this technology.
- In this self-contained article, we cover a range of concepts to present the general direction of LLMs comprehensively, including background, pre-training, fine-tuning,

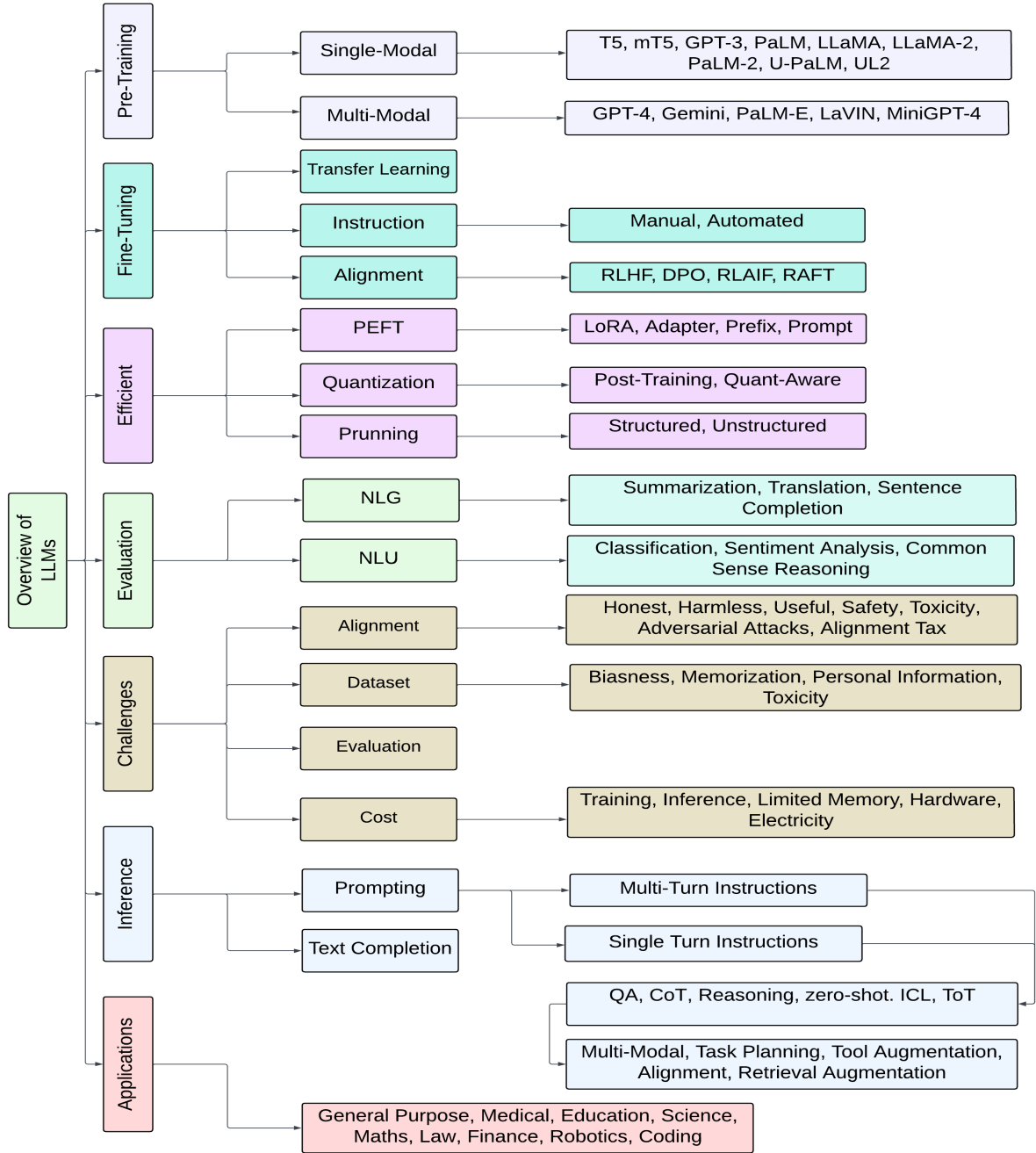


Figure 3: A broader overview of LLMs, dividing LLMs into seven branches: 1. Pre-Training 2. Fine-Tuning 3. Efficient 4. Inference 5. Evaluation 6. Applications 7. Challenges

multi-modal LLMs, augmented LLMs, LLMs-powered agents, datasets, evaluation, etc.

We loosely follow the existing terminology to ensure a standardized outlook of this research direction. For instance, following [50], our survey discusses pre-trained LLMs with 10B parameters or more. We refer the readers interested in smaller pre-trained models to [51, 52, 53].

The organization of this paper is as follows. Section 2 discusses the background of LLMs. Section 3 focuses on LLMs overview, architectures, training pipelines and strategies, fine-tuning, and

utilization in different domains. Section 4 highlights the configuration and parameters that play a crucial role in the functioning of these models. Summary and discussions are presented in section 3.8. The LLM training and evaluation, datasets, and benchmarks are discussed in section 5, followed by challenges and future directions, and conclusion in sections 7 and 8, respectively.

2. Background

We provide the relevant background to understand the fundamentals related to LLMs in this section. We briefly discuss necessary components in LLMs and refer the readers interested in details to the original works.

2.1. Tokenization

Tokenization [59] is an essential pre-processing step in LLM training that parses the text into non-decomposing units called tokens. Tokens can be characters, subwords [60], symbols [61], or words, depending on the tokenization process. Some of the commonly used tokenization schemes in LLMs include wordpiece [62], byte pair encoding (BPE) [61], and unigramLM [60]. Readers are encouraged to refer to [63] for a detailed survey.

2.2. Encoding Positions

The transformer processes input sequences in parallel and independently of each other. Moreover, the attention module in the transformer does not capture positional information. As a result, positional encodings were introduced in transformer [64], where a positional embedding vector is added to the token embedding. Variants of positional embedding include absolute, relative, or learned positional encodings. Within relative encoding, Alibi and RoPE are two widely used positional embeddings in LLMs.

Alibi [65]: It subtracts a scalar bias from the attention score that increases with the distance between token positions. This favors using recent tokens for attention.

RoPE [66]: It rotates query and key representations at an angle proportional to the token absolute position in the input sequence, resulting in a relative positional encoding scheme which decays with the distance between the tokens.

2.3. Attention in LLMs

Attention assigns weights to input tokens based on importance so that the model gives more emphasis to relevant tokens. Attention in transformers [64] calculates query, key, and value mappings for input sequences, where the attention score is obtained by multiplying the query and key, and later used to weight values. We discuss different attention strategies used in LLMs below.

Self-Attention [64]: Calculates attention using queries, keys, and values from the same block (encoder or decoder).

Cross Attention: It is used in encoder-decoder architectures, where encoder outputs are the queries, and key-value pairs come from the decoder.

Sparse Attention [67]: Self-attention has $O(n^2)$ time complexity which becomes infeasible for large sequences. To speed up the computation, sparse attention [67] iteratively calculates attention in sliding windows for speed gains.

Flash Attention [68]: Memory access is the major bottleneck in calculating attention using GPUs. To speed up, flash attention employs input tiling to minimize the memory reads and writes between the GPU high bandwidth memory (HBM) and the on-chip SRAM.

2.4. Activation Functions

The activation functions serve a crucial role in the curve-fitting abilities of neural networks [69]. We discuss activation functions used in LLMs in this section.

ReLU [70]: The Rectified linear unit (ReLU) is defined as:

$$\text{ReLU}(x) = \max(0, x) \quad (1)$$

GeLU [71]: The Gaussian Error Linear Unit (GeLU) is the combination of ReLU, dropout [72] and zoneout [73].

GLU variants [74]: The Gated Linear Unit [75] is a neural network layer that is an element-wise product ($\otimes$) of a linear transformation and a sigmoid transformed (σ) linear projection of the input given as:

$$\text{GLU}(x, W, V, b, c) = (xW + b) \otimes \sigma(xV + c), \quad (2)$$

where X is the input of layer and l , W, b, V and c are learned parameters. Other GLU variants [74] used in LLMs are:

$$\text{ReLU}(x, W, V, b, c) = \max(0, xW + b) \otimes,$$

$$\text{GEGLU}(x, W, V, b, c) = \text{GELU}(xW + b) \otimes (xV + c),$$

$$\text{SwiGLU}(x, W, V, b, c, \beta) = \text{Swish}\beta(xW + b) \otimes (xV + c).$$

2.5. Layer Normalization

Layer normalization leads to faster convergence and is an integrated component of transformers [64]. In addition to Layer-Norm [76] and RMSNorm [77], LLMs use pre-layer normalization [78], applying it before multi-head attention (MHA). Pre-norm is shown to provide training stability in LLMs. Another normalization variant, DeepNorm [79] fixes the issue with larger gradients in pre-norm.

2.6. Distributed LLM Training

This section describes distributed LLM training approaches briefly. More details are available in [13, 37, 80, 81].

Data Parallelism: Data parallelism replicates the model on multiple devices where data in a batch gets divided across devices. At the end of each training iteration weights are synchronized across all devices.

Tensor Parallelism: Tensor parallelism shards a tensor computation across devices. It is also known as horizontal parallelism or intra-layer model parallelism.

Pipeline Parallelism: Pipeline parallelism shards model layers across different devices. This is also known as vertical parallelism.

Model Parallelism: A combination of tensor and pipeline parallelism is known as model parallelism.

3D Parallelism: A combination of data, tensor, and model parallelism is known as 3D parallelism.

Optimizer Parallelism: Optimizer parallelism also known as zero redundancy optimizer [37] implements optimizer state partitioning, gradient partitioning, and parameter partitioning across devices to reduce memory consumption while keeping the communication costs as low as possible.

2.7. Libraries

Some commonly used libraries for LLMs training are: **Transformers [82]**: The library provides access to various pre-trained transformer models with APIs to train, fine-tune, infer, and develop custom models.

DeepSpeed [36]: A library for scalable distributed training and inference of deep learning models.

Megatron-LM [80]: It provides GPU-optimized techniques for large-scale training of LLMs.

JAX [83]: A Python library for high-performance numerical computing and scaleable machine learning. It can differentiate native Python and NumPy functions and execute them on GPUs.

Colossal-AI [84]: A collection of components to write distributed deep learning models.

BMTrain [81]: A library to write efficient stand-alone LLMs training code.

FastMoE [85]: Provides API to build mixture-of-experts (MoE) model in PyTorch.

MindSpore [86]: A deep learning training and inference framework extendable to mobile, edge, and cloud computing.

PyTorch [87]: A framework developed by Facebook AI Research lab (FAIR) to build deep learning models. The main features of PyTorch include a dynamic computation graph and a pythonic coding style.

Tensorflow [88]: A deep learning framework written by Google. The key features of TensorFlow are graph-based computation, eager execution, scalability, etc.

MXNet [89]: Apache MXNet is a deep learning framework with support to write programs in multiple languages, including, Python, C++, Scala, R, etc. It also provides support for dynamic and static computation graphs.

2.8. Data PreProcessing

This section briefly summarizes data preprocessing techniques used in LLMs training.

Quality Filtering: For better results, training data quality is essential. Some approaches to filtering data are: 1) classifier-based and 2) heuristics-based. Classifier-based approaches train a classifier on high-quality data and predict the quality of text for filtering, whereas heuristics-based employ some rules for filtering like language, metrics, statistics, and keywords.

Data Deduplication: Duplicated data can affect model performance and increase data memorization; therefore, to train LLMs, data deduplication is one of the preprocessing steps. This can be performed at multiple levels, like sentences, documents, and datasets.

Privacy Reduction: Most of the training data for LLMs is collected through web sources. This data contains private information; therefore, many LLMs employ heuristics-based methods to filter information such as names, addresses, and phone numbers to avoid learning personal information.

2.9. Architectures

Here we discuss the variants of the transformer architectures used in LLMs. The difference arises due to the application of

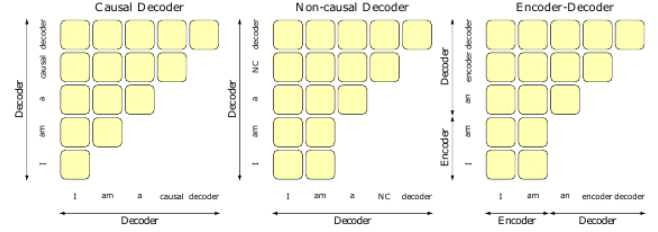


Figure 4: An example of attention patterns in language models, image is taken from [93].

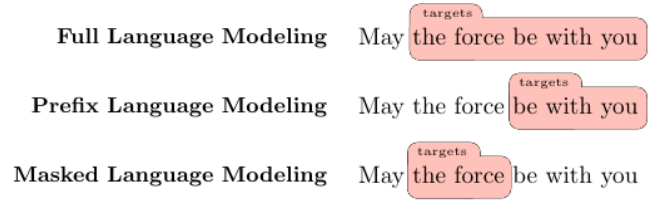


Figure 5: An example of language model training objectives, image from [93].

the attention and the connection of transformer blocks. An illustration of attention patterns of these architectures is shown in Figure 4.

Encoder Decoder: This architecture processes inputs through the encoder and passes the intermediate representation to the decoder to generate the output. Here, the encoder sees the complete sequence utilizing self-attention whereas the decoder processes the sequence one after the other with implementing cross-attention.

Causal Decoder: A type of architecture that does not have an encoder and processes and generates output using a decoder, where the predicted token depends only on the previous time steps.

Prefix Decoder: It is also known as a non-causal decoder, where the attention calculation is not strictly dependent on the past information and the attention is bidirectional. An example of a non-causal attention mask is shown in Figure 4.

Mixture-of-Experts: It is a variant of transformer architecture with parallel independent experts and a router to route tokens to experts. These experts are feed-forward layers after the attention block [90]. Mixture-of-Experts (MoE) is an efficient sparse architecture that offers comparable performance to dense models and allows increasing the model size without increasing the computational cost by activating only a few experts at a time [91, 92].

2.10. Pre-Training Objectives

This section describes LLMs pre-training objectives. For more details see the paper [93].

Full Language Modeling: An autoregressive language modeling objective where the model is asked to predict future tokens given the previous tokens, an example is shown in Figure 5.

Prefix Language Modeling: A non-causal training objective, where a prefix is chosen randomly and only remaining target tokens are used to calculate the loss. An example is shown in Figure 5.

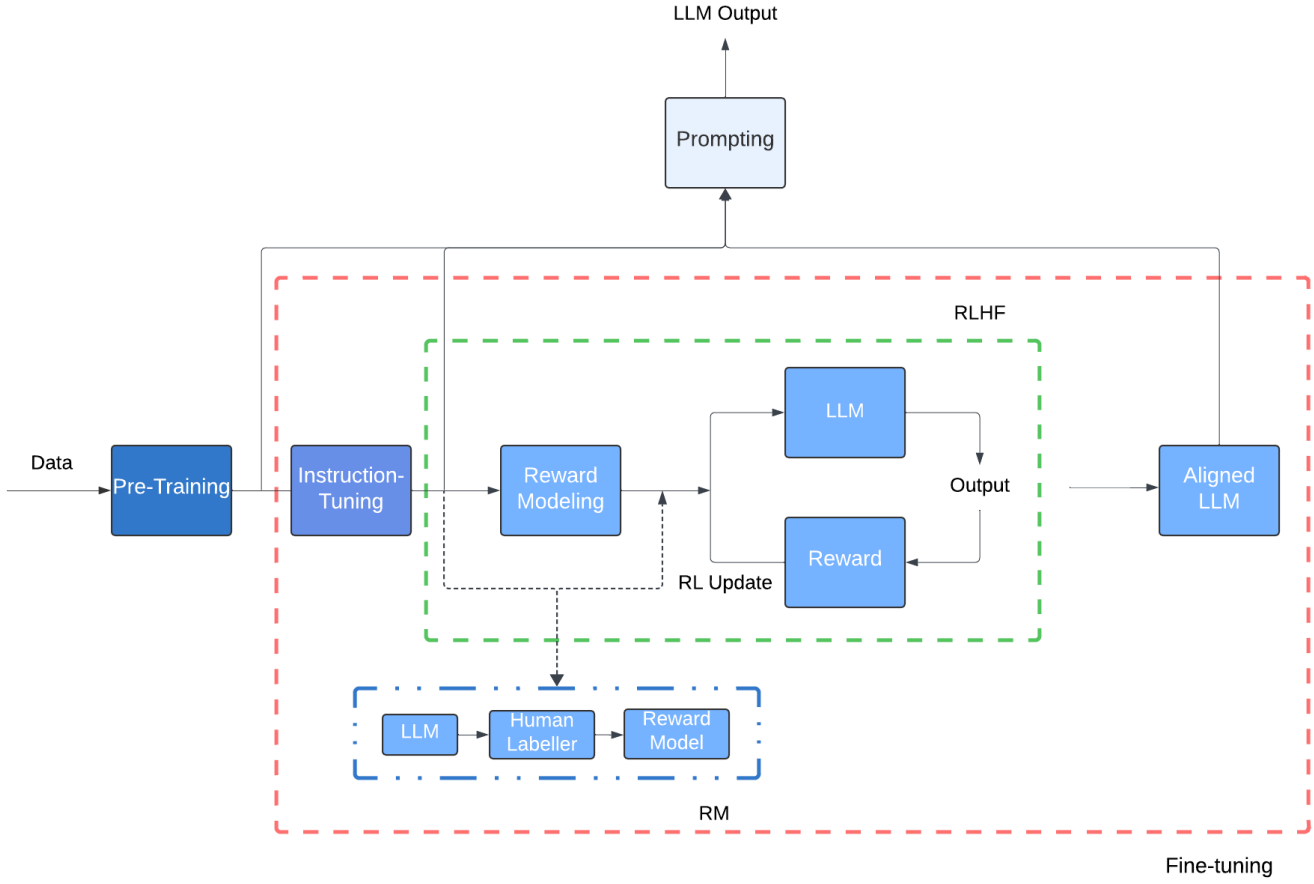


Figure 6: A basic flow diagram depicting various stages of LLMs from pre-training to prompting/utilization. Prompting LLMs to generate responses is possible at different training stages like pre-training, instruction-tuning, or alignment tuning. “RL” stands for reinforcement learning, “RM” represents reward-modeling, and “RLHF” represents reinforcement learning with human feedback.

Masked Language Modeling: In this training objective, tokens or spans (a sequence of tokens) are masked randomly and the model is asked to predict masked tokens given the past and future context. An example is shown in Figure 5.

Unified Language Modeling: Unified language modeling [94] is a combination of causal, non-causal, and masked language training objectives. Here in masked language modeling, the attention is not bidirectional but unidirectional, attending either left-to-right or right-to-left context.

2.11. LLMs Scaling Laws

Scaling laws study the optimal combination of model parameters, dataset size, and computational resources that predict the improvement in the model performance. It has been shown that the loss scales according to the power-law with model size, dataset size, and compute resources [95]. This study suggests larger models are more important than big data for better performance. Another variant of scaling law [96] suggests the model size and the number of training tokens should be scaled equally.

2.12. LLMs Adaptation Stages

This section discusses the fundamentals of LLMs adaptation stages, from pre-training to fine-tuning for downstream tasks and utilization. An example of different training stages and inference in LLMs is shown in Figure 6. In this paper, we refer to alignment-tuning as aligning with human preferences, while occasionally the literature uses the term alignment for different purposes.

2.12.1. Pre-Training

In the very first stage, the model is trained in a self-supervised manner on a large corpus to predict the next tokens given the input. The design choices of LLMs vary from encoder-decoder to decoder-only architectures with different building blocks and loss functions in sections 2.5, 2.4, 2.10.

2.12.2. Fine-Tuning

There are different styles to fine-tune an LLM. This section briefly discusses fine-tuning approaches.

Transfer Learning: The pre-trained LLMs perform well for various tasks [6, 15]. However, to improve the performance for

a downstream task, pre-trained models are fine-tuned with the task-specific data [10, 11], known as transfer learning.

Instruction-tuning: To enable a model to respond to user queries effectively, the pre-trained model is fine-tuned on instruction formatted data i.e., instruction and an input-output pair. Instructions generally comprise multi-task data in plain natural language, guiding the model to respond according to the prompt and the input. This type of fine-tuning improves zero-shot generalization and downstream task performance. Details on formatting instruction data and its various styles are available in [16, 50, 97].

Alignment-tuning: LLMs are prone to generating false, biased, and harmful text. To make them helpful, honest, and harmless, models are aligned using human feedback. Alignment involves asking LLMs to generate unexpected responses and then updating their parameters to avoid such responses [20, 21, 98].

It ensures LLMs operate according to human intentions and values. A model is defined to be an “aligned” model if the model fulfills three criteria of helpful, honest, and harmless or “HHH” [99].

Researchers employ reinforcement learning with human feedback (RLHF) [100] for model alignment. In RLHF, a fine-tuned model on demonstrations is further trained with reward modeling (RM) and reinforcement learning (RL), shown in Figure 6. Below we briefly discuss RM and RL pipelines in RLHF.

Reward modeling: trains a model to rank generated responses according to human preferences using a classification objective. To train the classifier humans annotate LLMs generated responses based on the HHH criteria.

Reinforcement learning: in combination with the reward model is used for alignment in the next stage. The previously trained reward model ranks LLM-generated responses into preferred vs. non-preferred, which is used to align the model with proximal policy optimization (PPO). This process repeats iteratively until convergence.

2.12.3. Prompting/Utilization

Prompting is a method to query trained LLMs for generating responses, as illustrated in Figure 6. LLMs can be prompted in various prompt setups, where they can be adapted to the instructions without fine-tuning and in other cases with fine-tuning on data containing different prompt styles [16, 101, 102]. A good guide on prompt engineering is available at [32]. Below, we will discuss various widely used prompt setups.

Zero-Shot Prompting: LLMs are zero-shot learners and capable of answering queries never seen before. This style of prompting requires LLMs to answer user questions without seeing any examples in the prompt.

In-context Learning: Also known as few-shot learning, here, multiple input-output demonstration pairs are shown to the model to generate the desired response. This adaptation style is also called few-shot learning. A discussion on formatting in-context learning (ICL) templates is available in [54, 50, 18, 16].

Reasoning in LLMs: LLMs are zero-shot reasoners and can be provoked to generate answers to logical problems, task planning, critical thinking, etc. with reasoning. Generating reasons is possible only by using different prompting styles,

whereas to improve LLMs further on reasoning tasks many methods [16, 97] train them on reasoning datasets. We discuss various prompting techniques for reasoning below.

Chain-of-Thought (CoT): A special case of prompting where demonstrations contain reasoning information aggregated with inputs and outputs so that the model generates outcomes with step-by-step reasoning. More details on CoT prompts are available in [55, 103, 101].

Self-Consistency: Improves CoT performance by generating multiple responses and selecting the most frequent answer [104].

Tree-of-Thought (ToT): Explores multiple reasoning paths with possibilities to look ahead and backtrack for problem-solving [105].

Single-Turn Instructions: In this prompting setup, LLMs are queried only once with all the relevant information in the prompt. LLMs generate responses by understanding the context either in a zero-shot or few-shot setting.

Multi-Turn Instructions: Solving a complex task requires multiple interactions with LLMs, where feedback and responses from the other tools are given as input to the LLM for the next rounds. This style of using LLMs in the loop is common in autonomous agents.

3. Large Language Models

This section reviews LLMs, briefly describing their architectures, training objectives, pipelines, datasets, and fine-tuning details.

3.1. Pre-Trained LLMs

Here, we provide summaries of various well-known pre-trained LLMs with significant discoveries, changing the course of research and development in NLP. These LLMs have considerably improved the performance in NLU and NLG domains, and are widely fine-tuned for downstream tasks. Moreover, We also identify key findings and insights of pre-trained LLMs in Table 1 and 2 that improve their performance.

3.1.1. General Purpose

T5 [10]: An encoder-decoder model employing a unified text-to-text training for all NLP problems is shown in Figure 7. T5 places layer normalization outside the residual path in a conventional transformer model [64]. It uses masked language modeling as a pre-training objective where spans (consecutive tokens) are replaced with a single mask instead of separate masks for each token. This type of masking speeds up the training as it produces shorter sequences. After pre-training, the model is fine-tuned using adapter layers [106] for downstream tasks.

GPT-3 [6]: The GPT-3 architecture is the same as the GPT-2 [5] but with dense and sparse attention in transformer layers similar to the Sparse Transformer [67]. It shows that large models can train on larger batch sizes with a lower learning rate to decide the batch size during training, GPT-3 uses the gradient noise scale as in [107]. Overall, GPT-3 increases model parameters to 175B showing that the performance of large language

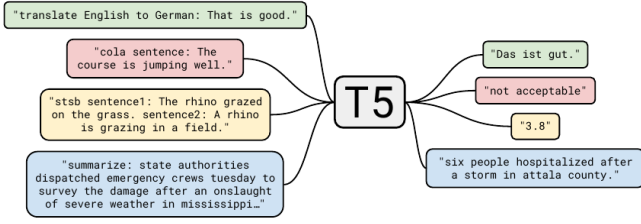


Figure 7: Unified text-to-text training example, source image from [10].

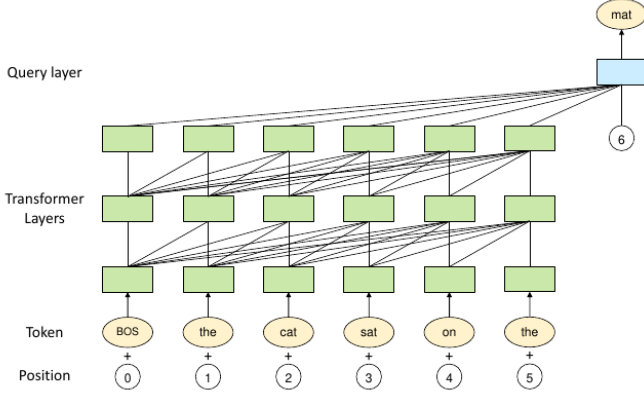


Figure 8: The image is the article of [108], showing an example of PanGu-α architecture.

models improves with the scale and is competitive with the fine-tuned models.

mT5 [11]: A multilingual T5 model [10] trained on the mC4 dataset with 101 languages. The dataset is extracted from the public common crawl scrape. The model uses a larger vocabulary size of 250,000 to cover multiple languages. To avoid over-fitting or under-fitting for a language, mT5 employs a data sampling procedure to select samples from all languages. The paper suggests using a small amount of pre-training datasets, including all languages when fine-tuning for a task using English language data. This allows the model to generate correct non-English outputs.

PanGu-α [108]: An autoregressive model that has a query layer at the end of standard transformer layers, example shown in Figure 8, to predict the next token. Its structure is similar to the transformer layer but with an additional embedding for the next position in the attention mechanism, given in Eq. 3.

$$a = p_n W_h^q W_h^k T H_L^T \quad (3)$$

CPM-2 [12]: Cost-efficient Pre-trained language Models (CPM-2) pre-trains bilingual (English and Chinese) 11B and 198B mixture-of-experts (MoE) models on the WuDaoCorpus [109] dataset. The tokenization process removes “_” white space tokens in the sentencepiece tokenizer. The models are trained with knowledge inheritance, starting with only the Chinese language in the first stage and then adding English and Chinese data. This trained model gets duplicated multiple times to initialize the 198B MoE model. Moreover, to use the model for downstream tasks, CPM-2 experimented with both com-

plete fine-tuning and prompt fine-tuning as in [40] where only prompt-related parameters are updated by inserting prompts at various positions, front, middle, and back. CPM-2 also proposes the INFMOE, a memory-efficient framework with a strategy to dynamically offload parameters to the CPU for inference at a 100B scale. It overlaps data movement with inference computation for lower inference time.

ERNIE 3.0 [110]: ERNIE 3.0 takes inspiration from multi-task learning to build a modular architecture using Transformer-XL [111] as the backbone. The universal representation module is shared by all the tasks, which serve as the basic block for task-specific representation modules, which are all trained jointly for natural language understanding, natural language generation, and knowledge extraction. This LLM is primarily focused on the Chinese language. It claims to train on the largest Chinese text corpora for LLM training, and achieved state-of-the-art in 54 Chinese NLP tasks.

Jurassic-1 [112]: A pair of auto-regressive language models, including a 7B-parameter J1-Large model and a 178B-parameter J1-Jumbo model. The training vocabulary of Jurassic-1 comprise word pieces, complete words, and multi-word expressions without any word boundaries, where possible out-of-vocabulary instances are interpreted as Unicode bytes. Compared to the GPT-3 counterparts, the Jurassic-1 models apply a more balanced depth-to-width self-attention architecture [113] and an improved tokenizer for a faster prediction based on broader resources, achieving a comparable performance in zero-shot learning tasks and a superior performance in few-shot learning tasks given the ability to feed more examples as a prompt.

HyperCLOVA [114]: A Korean language model with GPT-3 architecture.

Yuan 1.0 [115]: Trained on a Chinese corpus with 5TB of high-quality text collected from the Internet. A Massive Data Filtering System (MDFS) built on Spark is developed to process the raw data via coarse and fine filtering techniques. To speed up the training of Yuan 1.0 to save energy expenses and carbon emissions, various factors that improve the performance of distributed training are incorporated in architecture and training: like increasing the hidden state size improves pipeline and tensor parallelism performance, larger micro batches improve pipeline parallelism performance, and larger global batch size improve data parallelism performance. In practice, the Yuan 1.0 model performs well on text classification, Winograd Schema, natural language inference, and reading comprehension tasks.

Gopher [116]: The Gopher family of models ranges from 44M to 280B parameters in size to study the effect of *scale* on the LLMs performance. The 280B model beats GPT-3 [6], Jurassic-1 [112], MT-NLG [117], and others on 81% of the evaluated tasks.

ERNIE 3.0 TITAN [35]: ERNIE 3.0 Titan extends ERNIE 3.0 by training a larger model with 26x the number of parameters of the latter. This bigger model outperformed other state-of-the-art models in 68 NLP tasks. LLMs produce text with incorrect facts. In order to have control of the generated text with factual consistency, ERNIE 3.0 Titan adds another task, *Credible and Controllable Generations*, to its multi-task learning setup.

It introduces additional self-supervised adversarial and controllable language modeling losses to the pre-training step, which enables ERNIE 3.0 Titan to beat other LLMs in their manually selected Factual QA task set evaluations.

GPT-NeoX-20B [118]: An auto-regressive model that largely follows GPT-3 with a few deviations in architecture design, trained on the Pile dataset without any data deduplication. GPT-NeoX has parallel attention and feed-forward layers in a transformer block, given in Eq. 4, that increases throughput by 15%. It uses rotary positional embedding [66], applying it to only 25% of embedding vector dimension as in [119]. This reduces the computation without performance degradation. As opposed to GPT-3, which uses dense and sparse layers, GPT-NeoX-20B uses only dense layers. The hyperparameter tuning at this scale is difficult; therefore, the model chooses hyperparameters from the method [6] and interpolates values between 13B and 175B models for the 20B model. The model training is distributed among GPUs using both tensor and pipeline parallelism.

$$x + \text{Attn}(\text{LN}_1(x)) + \text{FF}(\text{LN}_2(x)) \quad (4)$$

OPT [14]: It is a clone of GPT-3, developed to open-source a model that replicates GPT-3 performance. Training of OPT employs dynamic loss scaling [120] and restarts from an earlier checkpoint with a lower learning rate whenever loss divergence is observed. Overall, the performance of OPT-175B models is comparable to the GPT-3-175B model.

BLOOM [13]: A causal decoder model trained on the ROOTS corpus to open-source an LLM. The architecture of BLOOM is shown in Figure 9, with differences like ALiBi positional embedding, an additional normalization layer after the embedding layer as suggested by the bitsandbytes¹ library. These changes stabilize training with improved downstream performance.

GLaM [91]: Generalist Language Model (GLaM) represents a family of language models using a sparsely activated decoder-only mixture-of-experts (MoE) structure [121, 90]. To gain more model capacity while reducing computation, the experts are sparsely activated where only the best two experts are used to process each input token. The largest GLaM model, GLaM (64B/64E), is about 7× larger than GPT-3 [6], while only part of the parameters are activated per input token. The largest GLaM (64B/64E) model achieves better overall results as compared to GPT-3 while consuming only one-third of GPT-3’s training energy.

MT-NLG [117]: A 530B causal decoder based on the GPT-2 architecture that has roughly 3× GPT-3 model parameters. MT-NLG is trained on filtered high-quality data collected from various public datasets and blends various types of datasets in a single batch, which beats GPT-3 on several evaluations.

Chinchilla [96]: A causal decoder trained on the same dataset as the Gopher [116] but with a little different data sampling distribution (sampled from MassiveText). The model architecture is similar to the one used for Gopher, with the exception of AdamW optimizer instead of Adam. Chinchilla identifies the

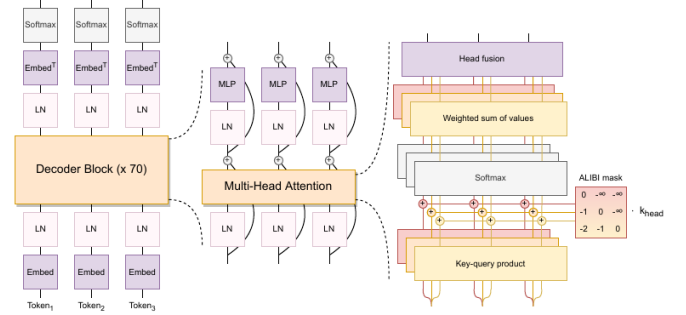


Figure 9: The BLOOM architecture example sourced from [13].

relationship that model size should be doubled for every doubling of training tokens. Over 400 language models ranging from 70 million to over 16 billion parameters on 5 to 500 billion tokens are trained to get the estimates for compute-optimal training under a given budget. The authors train a 70B model with the same compute budget as Gopher (280B) but with 4 times more data. It outperforms Gopher [116], GPT-3 [6], and others on various downstream tasks, after fine-tuning.

AlexaTM [122]: An encoder-decoder model, where encoder weights and decoder embeddings are initialized with a pre-trained encoder to speed up training. The encoder stays frozen for the initial 100k steps and is later unfrozen for end-to-end training. The model is trained on a combination of denoising and causal language modeling (CLM) objectives, concatenating a [CLM] token at the beginning for mode switching. During training, the CLM task is applied for 20% of the time, which improves the in-context learning performance.

PaLM [15]: A causal decoder with parallel attention and feed-forward layers similar to Eq. 4, speeding up training by a factor of 15. Additional changes to the conventional transformer model include SwiGLU activation, RoPE embeddings, multi-query attention that saves computation cost during decoding, and shared input-output embeddings. During training, loss spiking was observed, and to fix it, model training was restarted from a 100-step earlier checkpoint by skipping 200-500 batches around the spike. Moreover, the model was found to memorize around 2.4% of the training data at the 540B model scale, whereas this number was lower for smaller models.

PaLM-2 [123]: A smaller multi-lingual variant of PaLM, trained for larger iterations on a better quality dataset. PaLM-2 shows significant improvements over PaLM, while reducing training and inference costs due to its smaller size. To lessen toxicity and memorization, it appends special tokens with a fraction of pre-training data, which shows a reduction in generating harmful responses.

U-PaLM [124]: This method trains PaLM for 0.1% additional compute with the UL2 (also named as UL2Restore) objective [125], using the same dataset it outperforms the baseline significantly on various NLP tasks, including zero-shot, few-shot, commonsense reasoning, CoT, etc. Training with UL2R involves converting a causal decoder PaLM to a non-causal decoder PaLM and employing 50% sequential denoising, 25% regular denoising, and 25% extreme denoising loss functions.

¹<https://github.com/TimDettmers/bitsandbytes>

UL2 [125]: An encoder-decoder architecture trained using a mixture of denoisers (MoD) objective. Denoisers include 1) R-Denoiser: a regular span masking, 2) S-Denoiser: which corrupts consecutive tokens of a large sequence and 3) X-Denoiser: which corrupts a large number of tokens randomly. During pre-training, UL2 includes a denoiser token from R, S, X to represent a denoising setup. It helps improve fine-tuning performance for downstream tasks that bind the task to one of the upstream training modes. This MoD style of training outperforms the T5 model on many benchmarks.

GLM-130B [33]: GLM-130B is a bilingual (English and Chinese) model trained using an auto-regressive mask infilling pre-training objective similar to the GLM [126]. This training style makes the model bidirectional as compared to GPT-3, which is unidirectional. As opposed to GLM, the training of GLM-130B includes a small amount of multi-task instruction pre-training data (5% of the total data) along with self-supervised mask infilling. To stabilize the training, it applies embedding layer gradient shrink.

LLaMA [127, 21]: A set of decoder-only language models varying from 7B to 70B parameters. LLaMA models series is the most famous among the community for parameter efficiency and instruction tuning.

LLaMA-1 [127]: Implements efficient causal attention [128] by not storing and computing masked attention weights and key/query scores. Another optimization is reducing the number of activations recomputed in the backward pass, as in [129].

LLaMA-2 [21]: This work is more focused on fine-tuning a safer and better LLaMA-2-Chat model for dialogue generation. The pre-trained model has 40% more training data with a larger context length and grouped-query attention.

LLaMA-3/3.1 [130]: A collection of models trained on a seven times larger dataset as compared to LLaMA-2 with double the context length, outperforming its previous variants and other models.

PanGu- Σ [92]: An autoregressive model with parameters copied from PanGu- α and extended to a trillion scale with Random Routed Experts (RRE), the architectural diagram is shown in Figure 10. RRE is similar to the MoE architecture, with distinctions at the second level, where tokens are randomly routed to experts in a domain instead of using a learnable gating method. The model has bottom layers densely activated and shared across all domains, whereas top layers are sparsely activated according to the domain. This training style allows for extracting task-specific models and reduces catastrophic forgetting effects in the case of continual learning.

Mixtral8x22b [131]: A mixture-of-experts (MoE) model with eight distinct experts routes each token to two experts at each layer and combines the outputs additively.

Snowflake Arctic [132]: Arctic LLM is a hybrid of dense and mixture-of-experts (MoE) architecture. The MoE (128×3.66B MLP experts) is parallel to the dense transformer (10B) with only two experts activated. The model has many experts, compared to other MoE LLMs [131, 133], to increase the model capacity and provide an opportunity to choose among many experts for a diverse configuration. The model has 480B parameters, and only 17B are active during a forward pass, reducing

the computation significantly.

Grok [133, 134]: Grok is a family of LLMs including Grok-1 and Grok-1.5, released by XAI.

Grok-1 [133]: Grok-1 is a 314B parameters language MoE model (eight experts), where two experts are activated per token.

Grok-1.5 [134]: Grok-1.5 is a multi-modal LLM with a larger context length and improved performance.

Gemini [135, 136]: Gemini replaces Bard (based on PaLM) with multi-modal capabilities and significant language modeling performance improvements.

Gemini-1 [135]: The first-ever auto-regressive model to achieve human-level capabilities on the MMLU benchmark.

Gemini-1.5 [136]: A multi-modal LLM with MoE architecture builds on the findings of Gemini-1. The model has a 2M context window and can reason over information up to 10M tokens. Such large context windows were never achieved previously and shown to have a huge impact on performance gain.

Nemotron-4 340B [137]: A decoder-only model that has been aligned on 98% synthetic data and only 2% manually annotated data. Utilizing synthetic data at a large proportion improves the model performance significantly. The paper suggested introducing alignment data with a smaller subset of previously seen data during the late stage of the model pre-training, enabling the smooth transition from the pre-trained stage to the final training stage. To train better instruction-following models, weaker models are trained into stronger models iteratively. The synthetic data generated by the weaker instruction-tuned model is used to train a base model which is later supervised fine-tuned outperforming the weaker model.

DeepSeek [138]: DeepSeek studies the LLMs scaling laws in detail to determine the optimal non-embedding model size and training data. The experiments were performed for 8 budgets ranging from $1e^{17}$ to $3e^{20}$ training FLOPs. Each compute budget was tested against ten different models/data scales. The batch size and learning rates were also fitted for the given compute budget finding that the batch size should increase with the increased compute budget while decreasing the learning rate. Following are the equations for the optimal batch-size (B), learning rate (η), model size (M), and data (D):

$$\begin{aligned} B_{opt} &= 0.2920 \cdot C^{0.3271} \\ \eta_{opt} &= 0.3118 \cdot C^{-0.1250} \\ M_{opt} &= M_{base} \cdot C^a \\ D_{opt} &= D_{base} \cdot C^b \end{aligned} \tag{5}$$

$$M_{base} = 0.1715, D_{base} = 5.8316, a = 0.5243, b = 0.4757$$

DeepSeek-v2 [139]: An MoE model that introduces multi-head latent attention (MLA) to reduce inference costs, by compressing Key-Value (KV) cache into a latent vector. MLA achieves better performance than multi-head attention (MHA), and other efficient attention mechanisms such as grouped query attention (GQA), multi-query attention (MQA), etc. Because of MLA, DeepSeek-v2 achieves 5.76 times faster inference throughput as compared to DeepSeek [138].

3.1.2. Coding

CodeGen [140]: CodeGen has a similar architecture to PaLM [15], i.e., parallel attention, MLP layers, and RoPE embeddings. The model is trained on both natural language and programming language data sequentially (trained on the first dataset, then the second, and so on) on the following datasets 1) PILE, 2) BIGQUERY, and 3) BIGPYTHON. CodeGen proposed a multi-step approach to synthesizing code. The purpose is to simplify the generation of long sequences where the previous prompt and generated code are given as input with the next prompt to generate the next code sequence. CodeGen open-source a Multi-Turn Programming Benchmark (MTPB) to evaluate multi-step program synthesis.

Codex [141]: This LLM is trained on a subset of public Python Github repositories to generate code from docstrings. Computer programming is an iterative process where the programs are often debugged and updated before fulfilling the requirements. Similarly, Codex generates 100 versions of a program by repetitive sampling for a given description, which produces a working solution for 77.5% of the problems passing unit tests. Its powerful version powers Github Copilot².

AlphaCode [142]: A set of large language models, ranging from 300M to 41B parameters, designed for competition-level code generation tasks. It uses the multi-query attention [143] to reduce memory and cache costs. Since competitive programming problems highly require deep reasoning and an understanding of complex natural language algorithms, the AlphaCode models are pre-trained on filtered GitHub code in popular languages and then fine-tuned on a new competitive programming dataset named CodeContests. The CodeContests dataset mainly contains problems, solutions, and test cases collected from the Codeforces platform³. The pre-training employs standard language modeling objectives, while GOLD [144] with tempering [145] serves as the training objective for the fine-tuning on CodeContests data. To evaluate the performance of AlphaCode, simulated programming competitions are hosted on the Codeforces platform: overall, AlphaCode ranks at the top 54.3% among over 5000 competitors, where its Codeforces rating is within the top 28% of recently participated users.

CodeT5+ [34]: CodeT5+ is based on CodeT5 [146], with shallow encoder and deep decoder, trained in multiple stages initially unimodal data (code) and later bimodal data (text-code pairs). Each training stage has different training objectives and activates different model blocks encoder, decoder, or both according to the task. The unimodal pre-training includes span denoising and CLM objectives, whereas bimodal pre-training objectives contain contrastive learning, matching, and CLM for text-code pairs. CodeT5+ adds special tokens with the text to enable task modes, for example, [CLS] for contrastive loss, [Match] for text-code matching, etc.

StarCoder [147]: A decoder-only model with the SantaCoder architecture, employing Flash attention to scale up the context length to 8k. The StarCoder trains an encoder to filter names,

emails, and other personal data from the training data. Its fine-tuned variant outperforms PaLM, LLaMA, and LAMDA on HumanEval and MBPP benchmarks.

3.1.3. Scientific Knowledge

Galactica [148]: A large curated corpus of human scientific knowledge with 48 million papers, textbooks, lecture notes, millions of compounds and proteins, scientific websites, encyclopedias, and more are trained using the metaseq library³, which is built on PyTorch and fairscale [149]. The model wraps reasoning datasets with the `< work >` token to provide step-by-step reasoning context to the model, which has been shown to improve the performance on reasoning tasks.

3.1.4. Dialog

LaMDA [150]: A decoder-only model pre-trained on public dialog data, public dialog utterances, and public web documents, where more than 90% of the pre-training data is in English. LaMDA is trained with the objective of producing responses that exhibit high levels of quality, safety, and groundedness. To achieve this, discriminative and generative fine-tuning techniques are incorporated to enhance the model’s safety and quality aspects. As a result, the LaMDA models can be utilized as a general language model performing various tasks.

3.1.5. Finance

BloombergGPT [151]: A non-causal decoder model trained using both financial (“FINPILE” from the Bloomberg archive) and general-purpose datasets. The model’s architecture is similar to the BLOOM [13] and OPT [14]. It allocates 50B parameters to different blocks of the model using the approach [113]. For effective training, BloombergGPT packs documents together with `< [endoftext] >` to use the maximum sequence length, uses warmup batch size starting from 1024 to 2048, and manually reduces the learning rate multiple times during the training.

Xuan Yuan 2.0 [152]: A Chinese financial chat model with BLOOM’s [13] architecture trained on a combination of general purpose, financial, general purpose instructions, and financial institutions datasets. Xuan Yuan 2.0 combined the pre-training and fine-tuning stages to avoid catastrophic forgetting.

3.2. Fine-Tuned LLMs

Pre-trained LLMs have excellent generalization abilities to unseen tasks. However, because they are generally trained with the objective of next token prediction, LLMs have limited capacity to follow user intent and are prone to generate unethical, toxic or inaccurate responses [20]. For their effective utilization, LLMs are fine-tuned to follow instructions [16, 17, 97] and generate safe responses [20], which also results in increasing zero-shot, few-shot, and cross-task generalization [97, 16, 18], with minimal compute increment, e.g., 0.2% of the total pre-training for PaLM 540B [16].

We review various fine-tuned LLMs and strategies for effective fine-tuning in this section.

²<https://github.com/features/copilot>

³<https://codeforces.com/>

Table 1: Noteworthy findings and insights of *pre-trained* Large Language Models.

Models	Findings & Insights
T5	- Encoder and decoder with shared parameters perform equivalently when parameters are not shared - Fine-tuning model layers (adapter layers) work better than the conventional way of training on only classification layers
GPT-3	Few-shot performance of LLMs is better than the zero-shot, suggesting that LLMs are meta-learners
mT5	Large multi-lingual models perform equivalently to single language models on downstream tasks. However, smaller multi-lingual models perform worse
PanGu- α	LLMs have good few shot capabilities
CPM-2	- Prompt fine-tuning requires updating very few parameters while achieving performance comparable to full model fine-tuning - Prompt fine-tuning takes more time to converge as compared to full model fine-tuning - Inserting prompt tokens in-between sentences can allow the model to understand relations between sentences and long sequences - In an analysis, CPM-2 finds that prompts work as a provider (additional context) and aggregator (aggregate information with the input text) for the model
ERNIE 3.0	- A modular LLM architecture with a universal representation module and task-specific representation module helps in the finetuning phase - Optimizing the parameters of a task-specific representation network during the fine-tuning phase is an efficient way to take advantage of the powerful pre-trained model
Jurassic-1	- The performance of LLM is highly related to the network size - To improve runtime performance, more operations can be performed in parallel (width) rather than sequential (depth) - To efficiently represent and fit more text in the same context length, the model uses a larger vocabulary to train a SentencePiece tokenizer without restricting it to word boundaries. This further benefits in few-shot learning tasks
HyperCLOVA	By employing prompt-based tuning, the performances of models can be improved, often surpassing those of state-of-the-art models when the backward gradients of inputs are accessible
Yuan 1.0	The model architecture that excels in pre-training and fine-tuning cases may exhibit contrasting behavior in zero-shot and few-shot learning
Gopher	Relative encodings enable the model to evaluate for longer sequences than training.
ERNIE 3.0 Titan	Additional self-supervised adversarial loss to distinguish between real and generated text improves the model performance as compared to ERNIE 3.0
GPT-NeoX-20B	- Parallel attention + FF layers speed-up training 15% with the same performance as with cascaded layers - Initializing feed-forward output layers before residuals with scheme in [153] avoids activations from growing with increasing depth and width - Training on Pile outperforms GPT-3 on five-shot

Table Continued on Next Page

Models	Findings & Insights
OPT	• Restart training from an earlier checkpoint with a lower learning rate if loss diverges • Model is prone to generate repetitive text and stuck in a loop
Galactica	• Galactica’s performance has continued to improve across validation set, in-domain, and out-of-domain benchmarks, even with multiple repetitions of the corpus, which is superior to existing research on LLMs • A working memory token approach can achieve strong performance over existing methods on mathematical MMLU and MATH benchmarks. It sets a new state-of-the-art on several downstream tasks such as PubMedQA (77.6%) and MedMCQA dev (52.9%)
GLaM	• The model capacity can be maintained at reduced computation by replacing the feed-forward layer in each transformer layer with a mixture-of-experts (MoE) • The model trained on filtered data shows consistently better performances on both NLG and NLU tasks, where the effect of filtering is more significant on the former tasks • Filtered pretraining corpora play a crucial role in the generation capability of LLMs, especially for the downstream tasks • The scaling of GLaM MoE models can be achieved by increasing the size or number of experts in the MoE layer. Given a fixed budget of computation, more experts contribute to a better performance
LaMDA	• The model can be fine-tuned to learn to call different external information resources and tools
AlphaCode	• For higher effectiveness and efficiency, a transformer model can be asymmetrically constructed with a shallower encoder and a deeper decoder • To achieve better performances, it is necessary to employ strategies such as massively scaling upsampling, followed by the filtering and clustering of samples into a compact set • The utilization of novel sampling-efficient transformer architectures designed to facilitate large-scale sampling is crucial • Simplifying problem descriptions can effectively improve the model’s performance
Chinchilla	• The model size and the number of training tokens should be scaled proportionately: for each doubling of the model size, the number of training tokens should be doubled as well
PaLM	• English-centric models produce better translations when translating to English as compared to non-English • Generalized models can have equivalent performance for language translation to specialized small models • Larger models have a higher percentage of training data memorization • Performance has not yet saturated even at 540B scale, which means larger models are likely to perform better
AlexaTM	• Encoder-decoder architecture is more suitable to train LLMs given bidirectional attention to the context than decoder-only • Causal Language Modeling (CLM) task can be added to benefit the model with efficient in-context learning • Placing layer norm at the beginning of each transformer layer improves the training stability

Table Continued on Next Page

Models	Findings & Insights
U-PaLM	• Training with a mixture of denoisers outperforms PaLM when trained further for a few more FLOPs • Training with a mixture of denoisers improves the infilling ability and open-ended text generation diversity
UL2	• Mode switching training enables better performance on downstream tasks • CoT prompting outperforms standard prompting for UL2
GLM-130B	• Pre-training data with a small proportion of multi-task instruction data improves the overall model performance
CodeGen	• Multi-step prompting for code synthesis leads to a better user intent understanding and code generation
LLaMA	• A constant performance improvement is observed when scaling the model • Smaller models can achieve good performances with more training data and computing time
PanGu- Σ	• Sparse models provide the benefits of large models at a lower computation cost • Randomly Routed Experts reduces catastrophic forgetting effects which in turn is essential for continual learning • Randomly Routed Experts allow extracting a domain-specific sub-model in deployment which is cost-efficient while maintaining a performance similar to the original
BloombergGPT	• Pre-training with general-purpose and task-specific data improves task performance without hurting other model capabilities
XuanYuan 2.0	• Combining pre-training and fine-tuning stages in single training avoids catastrophic forgetting
CodeT5+	• Causal LM is crucial for a model’s generation capability in encoder-decoder architectures • Multiple training objectives like span corruption, Causal LM, matching, etc complement each other for better performance
StarCoder	• HHH prompt by Anthropic allows the model to follow instructions without fine-tuning
LLaMA-2	• Model trained on unfiltered data is more toxic but may perform better on downstream tasks after fine-tuning • Model trained on unfiltered data requires fewer samples for safety alignment
PaLM-2	• Data quality is important to train better models • Model and data size should be scaled with 1:1 proportions • Smaller models trained for larger iterations outperform larger models
LLaMA-3/3.1	• Increasing batch size gradually stabilizes the training without loss spikes • High-quality data at the final stages of training improves the model performance • Increasing model context length windows step-wise allows it to better adapt to various sequence lengths
Nemotron-40B	• Model aligned iteratively on synthetic data with data generated from the previously aligned model achieves competitive performance
DeepSeek	• Batch size should increase with the increase in compute budget while decreasing the learning rate
DeepSeek-v2	• Multi-head latent attention (MLA) performs better than multi-head attention (MHA) while requiring a significantly smaller KV cache, therefore achieving faster data generation

Table 2: Key insights and findings from the study of *instruction-tuned* Large Language Models.

Models	Findings & Insights
T0	• Multi-task prompting enables zero-shot generalization and outperforms baselines • Even a single prompt per dataset task is enough to improve performance
WebGPT	• To aid the model in effectively filtering and utilizing relevant information, human labelers play a crucial role in answering questions regarding the usefulness of the retrieved documents • Interacting a fine-tuned language model with a text-based web-browsing environment can improve end-to-end retrieval and synthesis via imitation learning and reinforcement learning • Generating answers with references can make labelers easily judge the factual accuracy of answers
Tk-INSTRUCT	• Instruction tuning leads to a stronger generalization of unseen tasks • More tasks improve generalization whereas only increasing task instances does not help • Supervised trained models are better than generalized models • Models pre-trained with instructions and examples perform well for different types of inputs
mT0 and BLOOMZ	• Instruction tuning enables zero-shot generalization to tasks never seen before • Multi-lingual training leads to even better zero-shot generalization for both English and non-English • Training on machine-translated prompts improves performance for held-out tasks with non-English prompts • English only fine-tuning on multilingual pre-trained language model is enough to generalize to other pre-trained language tasks
OPT-IML	• Creating a batch with multiple task examples is important for better performance • Only example proportional sampling is not enough, training datasets should also be proportional for better generalization/performance • Fully held-out and partially supervised tasks performance improves by scaling tasks or categories whereas fully supervised tasks have no effect • Including small amounts i.e. 5% of pretraining data during fine-tuning is effective • Only 1% reasoning data improves the performance, adding more deteriorates performance • Adding dialogue data makes the performance worse
Sparrow	• Labelers’ judgment and well-defined alignment rules help the model generate better responses • Good dialogue goals can be broken down into detailed natural language rules for the agent and the raters • The combination of reinforcement learning (RL) with reranking yields optimal performance in terms of preference win rates and resilience against adversarial probing
Flan	• Finetuning with CoT improves performance on held-out tasks • Fine-tuning along with CoT data improves reasoning abilities • CoT tuning improves zero-shot reasoning • Performance improves with more tasks • Instruction fine-tuning improves usability which otherwise is challenging for pre-trained models • Improving the model’s performance with instruction tuning is compute-efficient • Multitask prompting enables zero-shot generalization abilities in LLM
WizardCoder	• Fine-tuning with re-written instruction-tuning data into a complex set improves performance
LLaMA-2-Chat	• Model learns to write safe responses with fine-tuning on safe demonstrations, while additional RLHF step further improves model safety and make it less prone to jailbreak attacks
LIMA	• Less high quality data is enough for fine-tuned model generalization

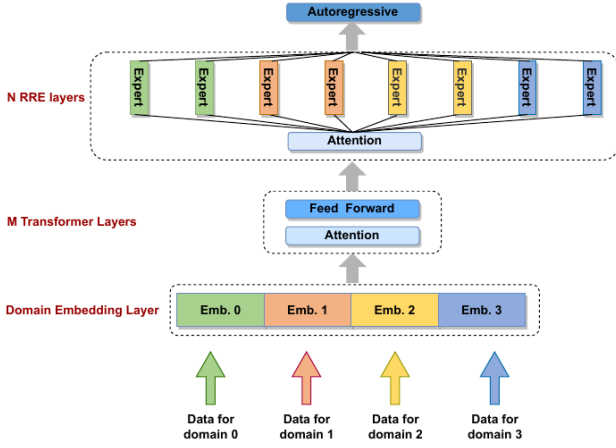


Figure 10: This example illustrates the PanGu- Σ architecture, as depicted in the image sourced from [92].

3.2.1. Instruction-Tuning with Manually Created Datasets

Numerous hand-crafted instruction-tuning datasets with different design choices are proposed in the literature to instruction-tune LLMs. The performance of fine-tuned LLMs depends on multiple factors, such as dataset, instruction diversity, prompting templates, model size, and training objectives. Keeping this in view, diverse fine-tuned models have emerged in the literature using manually created datasets.

The models T0 [17] and mT0 (multi-lingual) [154] employ templates to convert existing datasets into prompt datasets. They have shown improvements in generalization to zero-shot and held-out tasks. Tk-Instruct [18] fine-tuned the T5 model with in-context instructions to study generalization on unseen tasks when given in-context instructions during test time. The model outperformed Instruct-GPT, despite being smaller in size, i.e., 11B parameters as compared to 175B of GPT-3.

Increasing Tasks and Prompt Setups: Zero-shot and few-shot performance improves significantly by expanding task collection and prompt styles. OPT-IML [97] and Flan [16] curated larger 2k and 1.8k task datasets, respectively. While increasing task size alone is not enough, OPT-IML and Flan add more prompting setups in their datasets, zero-shot, few-shot, and CoT. In continuation, CoT Collection [101] fine-tunes Flan-T5 further on 1.88M CoT samples. Another method [102] uses symbolic tasks with tasks in T0, Flan, etc.

3.2.2. Instruction-Tuning with LLMs Generated Datasets

Generating an instruction-tuning dataset requires carefully writing instructions and input-output pairs, which are often written by humans, smaller in size, and less diverse. To overcome this, self-instruct [19] proposed an approach to prompt available LLMs to generate instruction-tuning datasets. Self-instruct outperformed models trained on manually created dataset SUPER-NATURALINSTRUCTIONS (a dataset with 1600+ tasks) [18] by 33%. It starts with a seed of 175 tasks, 1 instruction, and 1 sample per task and iteratively generates new instructions (52k) and instances (82k input-output pairs) using

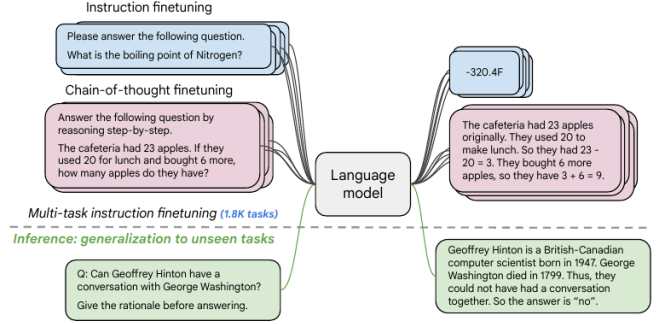


Figure 11: An example image shows an instance of the Flan training paradigm, taken from [16].

GPT-3 [6]. Contrary to this, Dynosaur [155] uses the meta-data of datasets on Huggingface to prompt LLMs to generate multiple task instruction-tuning datasets.

LLaMA Tuned: Various models in the literature instruction-tune LLaMA [156] with GPT-3 [6] or GPT-4 [157] generated datasets. Among these, Alpaca [158], Vicuna [159], and LLaMA-GPT-4 [160] are a few general-purpose fine-tuned models, where Alpaca is trained on 52k samples from text-davinci-003, Vicuna on 70k samples from ShareGPT.com, and LLaMA-GPT-4 by re-creating Alpaca instructions from GPT-4. Goat [161] fine-tunes LLaMA for arithmetic tasks (1 million samples) by generating data from ChatGPT and outperforms GPT-4, PaLM, BLOOM, OPT, etc., attributing its success to the LLaMA’s consistent tokenization of numbers. HuaTuo [162] is a medical knowledge model, fine-tuned with a generated QA dataset of 8k instructions.

Complex Instructions: Evol-Instruct [163, 164] prompts LLMs to convert given instructions into a more complex set. The instructions are iteratively evolved with re-writing instructions in complex wording and creating new instructions. With this style of automated instruction generation, WizardLM [163] (fine-tuned LLaMA on 250k instructions), outperforms Vicuna and Alpaca, and WizardCoder [164] (fine-tuned StarCoder) beats Claude-Plus, Bard, and others.

3.2.3. Aligning with Human Preferences

Incorporating human preferences into LLMs presents a significant advantage in mitigating undesirable behaviors and ensuring accurate outputs. The initial work on alignment, such as InstructGPT [20] aligns GPT-3 using a 3-step approach, instruction-tuning, reward modeling, and fine-tuning with reinforcement learning (RL). The supervised fine-tuned GPT-3 on demonstrations is queried to generate responses, which human labelers rank according to human values, and a reward model is trained on the ranked data. Lastly, the GPT-3 is trained with proximal policy optimization (PPO) using rewards on the generated data from the reward model. LLaMA 2-Chat [21] improves alignment by dividing reward modeling into helpfulness and safety rewards and using rejection sampling in addition to PPO. The initial four versions of LLaMA 2-Chat are fine-tuned with rejection sampling and then with PPO on

top of rejection sampling.

Aligning with Supported Evidence: This style of alignment allows the model to generate responses with proofs and facts, reduces hallucination, and assists humans more effectively, which increases trust in the model’s output. Similar to the RLHF training style, a reward model is trained to rank generated responses containing web citations in answers to questions, which is later used to train the model, as in GopherCite [165], WebGPT [166], and Sparrow [167]. The ranking model in Sparrow [167] is divided into two branches, preference reward and rule reward, where human annotators adversarial probe the model to break a rule. These two rewards together rank a response to train with RL.

Aligning Directly with SFT: The PPO in the RLHF pipeline is complex, memory-intensive, and unstable, requiring multiple models, reward, value, policy, and reference models. Avoiding this sophisticated alignment pipeline is possible by incorporating minimal changes in the supervised fine-tuning (SFT) pipeline as in [168, 169, 170], with better or comparable performance to PPO. Direct preference optimization (DPO) [168] trains a model directly on the human-preferred responses to maximize the likelihood of preferred against unpreferred responses, with per-sample importance weight. Reward ranked fine-tuning RAFT [169] fine-tunes the model on ranked responses by the reward model. Preference ranking optimization (PRO) [171] and RRHF [170] penalize the model to rank responses with human preferences and supervised loss. On the other hand, chain-of-hindsight (CoH) [172] provides feedback to the model in language rather than reward, to learn good versus bad responses.

Aligning with Synthetic Feedback: Aligning LLMs with human feedback is slow and costly. The literature suggests a semi-automated process to align LLMs by prompting LLMs to generate helpful, honest, and ethical responses to the queries, and fine-tuning using the newly created dataset. Constitutional AI [173] replaces human feedback in RLHF with AI, calling it RL from AI feedback (RLAIF). AlpacaFarm [174] designs prompts to imitate human feedback using LLMs APIs. Opposite to constitutional AI, AlpacaFarm injects noise in feedback to replicate human mistakes. Self-Align [98] prompts the LLM with ICL examples, instructing the LLM about what the response should contain to be considered useful and ethical. The same LLM is later fine-tuned with the new dataset.

Aligning with Prompts: LLMs can be steered with prompts to generate desirable responses without training [175, 176]. The self-correction prompting in [176] concatenates instructions and CoT with questions, guiding the model to answer its instruction following a strategy to ensure moral safety before the actual answer. This strategy is shown to reduce the harm in generated responses significantly.

Red-Teaming/Jailbreaking/Adversarial Attacks: LLMs exhibit harmful behaviors, hallucinations, leaking personal information, and other shortcomings through adversarial probing. The models are susceptible to generating harmful responses even though they are aligned for safety [177, 178]. Red-teaming is a common approach to address illicit outputs, where the LLMs are prompted to generate harmful outputs [178, 179].

The dataset collected through red-teaming is used to fine-tune models for safety. While red-teaming largely relies on human annotators, another work [180] red-team LLMs to find prompts that lead to harmful outputs for other LLMs.

3.2.4. Continue Pre-Training

Although fine-tuning boosts a model’s performance, it leads to catastrophic forgetting of previously learned information. Concatenating fine-tuning data with a few randomly selected pre-training samples in every iteration avoids network forgetting [181, 152]. This is also effective in adapting LLMs for cases where fine-tuning data is small and the original capacity is to be maintained. Prompt-based continued pre-training (PCP) [182] trains the model with text and instructions related to tasks and then finally instruction-tunes the model for downstream tasks.

3.2.5. Sample Efficiency

While fine-tuning data is generally many-fold smaller than the pre-training data, it still has to be large enough for acceptable performance [16, 97, 18] and requires proportional computing resources. Studying the effects on performance with less data, existing literature [183, 184] finds that models trained on less data can outperform models trained with more data. In [183], 25% of the total downstream data is found enough for state-of-the-art performance. Selecting coreset-based 0.5% of the total instruction-tuning data improves the model performance by 2% in [184], as compared to the complete data tuning. Less is more for alignment (LIMA) [185] uses only 1000 carefully created demonstrations to fine-tune the model and has achieved comparable performance to GPT-4.

3.3. Increasing Context Window

LLMs are trained with limited context windows due to expensive attention and high memory requirements. A model trained on limited sequence lengths fails to generalize to unseen lengths at inference time [186, 49]. Alternatively, LLMs with ALiBi [65] positional encodings can perform zero-shot length extrapolation. However, ALiBi has less expressive power [66] and inferior performance on multiple benchmarks [46], and many LLMs use RoPE positional embedding that is unable to perform zero-shot extrapolation. A larger context length has benefits such as a better understanding of longer documents, more samples in in-context learning, execution of bigger reasoning processes, etc. Expanding context length during fine-tuning is slow, inefficient, and computationally expensive [49]. Therefore, researchers employ various context window extrapolation techniques discussed below.

Position Interpolation: Rather than extrapolating, [49] shows that interpolating position encodings within the pre-trained context window are more effective. The work demonstrates that only 1000 steps of fine-tuning are enough to achieve better results on larger windows without reducing performance compared to the original context size. Giraffe [46] uses power scaling in RoPE, and YaRN [47] proposed NTK-aware interpolation.

Efficient Attention Mechanism: Dense global attention is one of the major constraints in training larger context window LLMs. Using efficient attention variants, such as local, sparse, and dilated attention, reduces the computation cost significantly. LongT5 [48] proposes transient global attention (TGlobal), applying attention to local and global tokens (windowed token averaging). The model replaces attention in T5 [10] with TGlobal attention, pre-trains the model on 4098 sequence length, fine-tunes on larger window sizes, as large as 16k, and improves task performance on longer inputs. This shows the extrapolation ability of TGlobal attention with only fine-tuning. COLT5 [187] uses two branches, one with lightweight and the other with heavyweight attention and feed-forward layers. All tokens are processed from the lightweight branch, and only important tokens are routed to the heavyweight branch. LongNet [188] replaces standard attention with dilated attention, expanding sequence length to 1 billion tokens. LongLoRA [189] proposes shift-short attention, used during fine-tuning to reduce dense attention costs. However, the model during inference uses dense attention and achieves similar performance as full attention fine-tuning.

Extrapolation without Training: LM-Infinite [186] and parallel context windows (PCW) [190] show length extrapolation is possible using pre-trained LLMs. LM-Infinite suggested Λ -shaped attention applied within the original context window limits. Likewise, PCW chunks larger inputs into the pre-trained context lengths and applies the same positional encodings to each chunk.

3.4. Augmented LLMs

LLMs are capable of learning from the examples concatenated with the input, known as context augmentation, in-context learning (ICL), or few-shot prompting. They show excellent generalization to unseen tasks with few-shot prompting, enabling LLMs to answer queries beyond the capacity acquired during training [6, 55]. These emergent abilities allow for adapting the model without fine-tuning—a costly process. Aside from this, hallucination, producing inaccurate, unsafe, or factually incorrect responses, is common for LLMs, which is avoided by augmenting contextual data. While the user can provide in-context samples in the query [54, 32], here we specifically refer to the methods that access external storage programmatically, calling them augmented LLMs.

The literature suggests various external memory designs to augment LLMs, long-term [191, 192, 193, 194], short-term [195], symbolic [196], and non-symbolic [197, 198]. The memory can be maintained in different formats such as documents, vectors, or databases. A few systems maintain intermediate memory representations to retain information across multiple iterations [194, 192], while others extract important information from the datasets and save it in memory for recall [199]. The memory read and write operations are performed either with or without LLMs cooperation [192, 200, 194, 201], acting as a feedback signal in [195]. We discuss different types of augmented LLMs below.

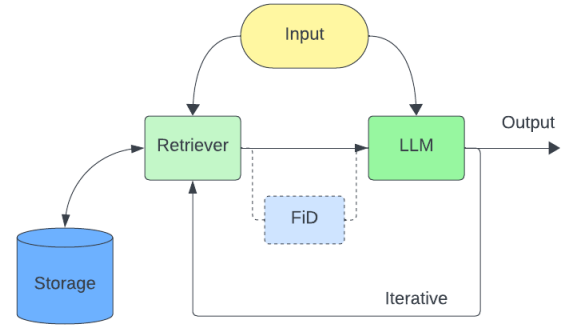


Figure 12: A flow diagram of Retrieval Augmented LLMs. The retriever extracts a similar context to the input and forwards it to the LLM either in simple language or encoded through Fusion-in-Decoder (FiD). Depending on the task, retrieval and generation may repeat multiple times.

3.4.1. Retrieval Augmented LLMs

LLMs may have limited memory and outdated information, leading to inaccurate responses. Retrieving relevant information from external up-to-date storage enables the LLMs to accurately answer with references and utilize more information. With retrieval augmentation, smaller models have been shown to perform at par with larger models. For instance, the 11B model can become competitive to 540B PaLM in [25] and 7.5B to 280B Gopher in [193]. Retrieval augmented language modeling (RALM) has two major components, shown in Figure 12, namely: 1) retriever and 2) language model. In RALM, the retriever plays a crucial role in driving LLM response, where incorrect information can steer LLMs to false behavior. This leads to the development of various methods to retrieve accurate information and fuse with the query for better performance.

Zero-Shot Retrieval Augmentation: This kind of augmentation keeps the original LLM architecture and weights unchanged and uses BM25 [202], nearest neighbors, or frozen pre-trained models like Bert [7] as a retriever. The retrieved information is provided as input to the model for response generation, shown to improve performance over LLMs without retrieval [198, 203]. In some scenarios, multiple retrieval iterations are required to complete the task. The output generated in the first iteration is forwarded to the retriever to fetch similar documents. Forward-looking active retrieval (FLARE) [197] initially generates the response and corrects the output by retrieving relevant documents if the response contains low-confidence tokens. Similarly, RepoCoder [204] fetches code snippets recursively for code completion.

Training with Retrieval Augmentation: To reduce failures in retrieval augmentation generation (RAG), researchers train or fine-tune retrievers and LLMs with a retrieval augmentation pipeline. We discuss the literature below based on their focus on the respective training processes of the pipeline.

Training LLM: Retrieval-enhanced transformer (RETRO) [193] shows pre-training smaller LLMs with RAG pipeline outperforms larger LLMs, such as GPT-3 trained without RAG. RETRO uses a 2-trillion token subset of MassiveText as

a database. The retrieval pipeline divides the input query into subsets and retrieves relevant chunks from the database for each subset, encoded together with input intermediate representations for generating tokens. It uses cross-chunked attention to attend to previous chunks auto-regressively. A study on RETRO [205] shows models pre-trained without RAG but fine-tuned using RAG lack the performance gains obtained by pre-training with RAG.

Training Retriever: Quality of responses generated by LLMs is highly dependent on the in-context examples. Therefore, [206, 207, 208, 209] train retrievers to retrieve accurate few-shot samples while keeping the LLM frozen for generation. Retrieved samples are ranked to build ground-truth data to train retrievers with contrastive learning in [206, 208]. RoBERTa is trained for downstream tasks in [207] for ICL samples retrieval. REPLUG [209] trains the retriever with supervised signals from the frozen LLM-generated outputs.

Training Retriever and LLM: Further benefits are achieved by training both the retriever and the model in [25, 210, 211]. In this case, the error propagates back to the retriever, updating both the language model and the retriever. While masked language modeling (MLM) is a common pre-training objective [25, 211], retrieval pre-trained transformer (RPT) [210] used document chunk prediction as a pre-training objective for long text modeling.

Encoded Context Augmentation: Concatenating retrieved documents with the query becomes infeasible as the sequence length and sample size grow. Encoding the context and fusing it with the decoder (Fusion-in-Decoder) using cross-attention makes it possible to augment more samples without increasing computation costs significantly [212, 193, 210, 25].

Web Augmented: Locally stored memory, but external to LLM, has limited information. However, a large amount of information is available on the internet, which gets updated regularly. Rather than storing information locally, various methods retrieve query-related context through a web search and forward it to LLMs [213, 214, 166].

3.4.2. Tool Augmented LLMs

While RAG relies on the retriever to provide context to the LLM to answer queries, tool augmented LLMs capitalize on the reasoning abilities of LLMs to iteratively plan by dividing tasks into sub-tasks, selecting necessary tools, and taking actions to complete the task [215, 216, 217, 27]. A generic pipeline of tool-augmented LLMs is shown in Figure 13, where different modules in Figure 13 are selected in a loop until the task completion.

Zero-Shot Tool Augmentation: LLMs in-context learning and reasoning abilities enable them to interact with tools without training. Automatic reasoning and tool-use (ART) [217] builds a task library with demonstrations of reasoning steps and calling external tools. It retrieves similar task examples and provides the context to the LLM for inference. Aside from this, [218] shows tool documentation is enough to teach LLMs to use tools without demonstrations. RestGPT [219] integrates LLMs with RESTful APIs by decomposing tasks into planning

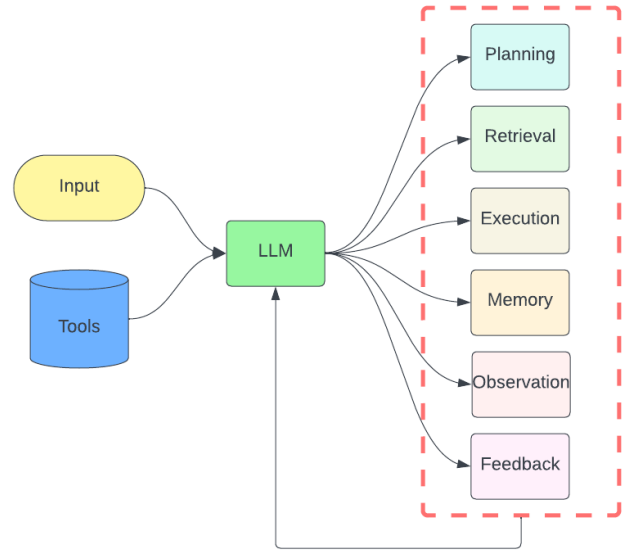


Figure 13: A basic flow diagram of tool augmented LLMs. Given an input and a set of available tools, the model generates a plan to complete the task. The tool augmented LLMs utilize different modules iteratively, such as retriever, tool execution, read-write to memory, feedback, etc., depending on the task.

and API selection steps. The API selector understands the API documentation to select a suitable API for the task and plan the execution. ToolkenGPT [220] uses tools as tokens by concatenating tool embeddings with other token embeddings. During inference, the LLM generates the tool tokens representing the tool call, stops text generation, and restarts using the tool execution output.

Training with Tool Augmentation: LLMs are trained to interact with diverse tools, enhancing planning abilities to overcome the limitations of zero-shot tool augmentation [221, 27, 222, 223]. Gorilla [221] instruction-tunes LLaMA with information retrieval from API documentation. It uses the self-instruct [19] data generation pipeline with GPT-4 by providing in-context examples retrieved from API documentation. Tool augmented language model (TALM) [27] fine-tunes T5 [10] for tool use with a self-play approach, where it iteratively completes tool manipulation tasks and includes them back in the training set. ToolLLM [223] collects 16k APIs from RapidAPI. It samples APIs from the list to generate an instruction-tuning dataset using ChatGPT in single-tool and multi-tool scenarios. For high-quality datasets, ToolLLM suggested a depth-first search-based decision tree (DFS DT) method to generate ground-truths with diverse reasoning and planning.

Multimodal Tool Augmentation: The compositional reasoning capacity of LLMs allows them to manipulate tools in multimodal settings [215, 216, 224]. Following the pipeline shown in Figure 13, the LLM outlines a plan, generally executing in a sequence: Plan → Tool selection → Execute → Inspect → Generate, to respond to the user query. Here, the database of tools is rich in modalities, including text, images, etc. Many of the multimodal tool augmentation systems employ multimodal LLMs [31, 225, 224, 216], while others utilize single modality

LLMs and generate a plan on using different modality tools to solve multimodal queries [226].

3.5. LLMs-Powered Agents

AI agents are autonomous entities, capable of planning, decision-making, and performing actions to achieve complex goals. In the early days, AI agents were rule-based, designed for narrow tasks, and had limited capabilities, such as Clippy [227] and Deep Blue [228]. In contrast to this, LLMs abilities to respond to dynamic scenarios have made it possible to incorporate them in diverse applications, including LLMs-powered agents [224, 216], where LLMs behave as the brain of agents. LLMs have been incorporated in web agents [166, 167], coding agents [229], tool agents [27, 223], embodied agents [26], and conversational agents [195], requiring minimal to no fine-tuning". Below we summarize the research in LLMs-based autonomous agents. For a more detailed discussion, please refer to [230, 231].

LLMs Steering Autonomous Agents: LLMs are the cognitive controllers of the autonomous agents. They generate plans, reason about tasks, incorporate memory to complete tasks, and adapt the outline depending on the feedback from the environment. Depending on the acquired capabilities of LLMs, many methods fine-tune, propose a better prompting approach, or utilize different modules to enhance agents' performance. Modules and strategies employed in autonomous agents are briefly discussed below.

Planning and Reasoning: Completing a complex task requires human-like logical thinking, planning necessary steps, and reasoning current and future directions. Prompting methods like chain-of-thoughts [103], tree-of-thoughts [105], and self-consistency [104] are central to agents, eliciting LLMs to reason its actions and choose among different paths for task completion. When LLMs are prompted with a task description and a sequence of actions, they can accurately generate plan actions without any fine-tuning [232]. Reasoning via planning (RAP) [233] incorporates a re-purposed LLM as a world model to reason about future outcomes and explore alternative paths for task completion. Retroformer [234] uses a retrospective LLM to improve main LLM planning and reasoning capabilities by providing helpful task cues.

Feedback: LLMs in open-loop systems generate plans and assume that the agent will complete them successfully. However, the actual scenario is different with failures and variable responses from the environment. To correctly complete tasks, many methods use LLMs in a closed-loop where the action response is provided as feedback to the LLMs to re-assess and update the plan as required [235, 236, 237, 195]. Another direction of research exploits LLMs as reward functions to train reinforcement learning (RL) policies instead of humans [238].

Memory: LLMs can learn from the context provided in the prompt. In addition to internal memory, various systems employ external memory to save the response history. Reflexion [195] maintains an episodic memory to use previous responses as feedback to improve future decision-making. Retroformer [234] improves its responses by employing short-term

and long-term memory, where short-term memory contains recent responses and long-term memory keeps summarized failed attempts to add in the prompt as reflection.

Multi-Agents Systems: LLMs can play user-defined roles and behave like a specific domain expert. In multi-agent systems, each LLM is assigned a unique role, simulating human behavior and collaborating with other agents to complete a complex task [229, 239].

LLMs in Physical Environment: LLMs are good at instruction-following, however, utilizing them for physically grounded tasks requires adaptation, as they lack real-world knowledge. This could lead to generating illogical responses for a particular physical situation [240, 26]. SayCan [240] make LLMs aware of the available low-level task operations. LLM (Say) builds a high-level plan to complete the task and a learned affordance function (Can) explores the possibility of executing the plan in the real world. SayCan uses RL to train the language-conditioned affordance function. PaLM-E enables the LLM to solve grounded tasks by training multi-modal LLM feeding inputs directly from the sensors.

Manipulation: In the area of manipulation [236, 241], LLMs enhance a robot's dexterity and adaptability, excelling in tasks like object recognition, grasping, and collaboration. They analyze visual and spatial information to determine the most effective approach to interact with objects.

Navigation: LLMs enhance a robot's ability to navigate complex environments with precision and adaptability [242, 243, 244, 245]. They generate feasible paths and trajectories for robots, accounting for intricate environmental details [246]. This ability is valuable in scenarios requiring precise and dynamically adaptable navigation in environments like warehouses, transport, healthcare facilities, and residences.

3.6. Efficient LLMs

Deploying LLMs in production is expensive. Reducing their running costs while preserving performance is an appealing area of research. This section summarizes the approaches suggested to enhance LLMs' efficiency.

3.6.1. Parameter Efficient Fine-Tuning

Fine-tuning LLMs with tens or hundreds of billions of parameters, such as GPT-3 (175B), BLOOM (176B), MT-NLG (540B), etc., is computationally intensive and time-consuming. To avoid complete model fine-tuning, numerous parameter-efficient fine-tuning (PEFT) techniques [40, 247, 41, 38, 39] try to achieve acceptable model fine-tuning performance at reduced costs. As compared to full fine-tuning [248], PEFT performs better in low-resource setups, achieves comparable performance on medium-resource scenarios, and performs worse than full fine-tuning under high-resource availability. An overview of different PEFT approaches is shown in Figure 14.

Adapter Tuning: Adds a few trainable parameters within the transformer block. The adapter layer is a sequence of feature downscaling, non-linearity, and upscaling [106]. Variants of adapter tuning inject adapter layers sequentially [106] and in parallel [38], whereas the mixture of adapter (AdaMix) [249]

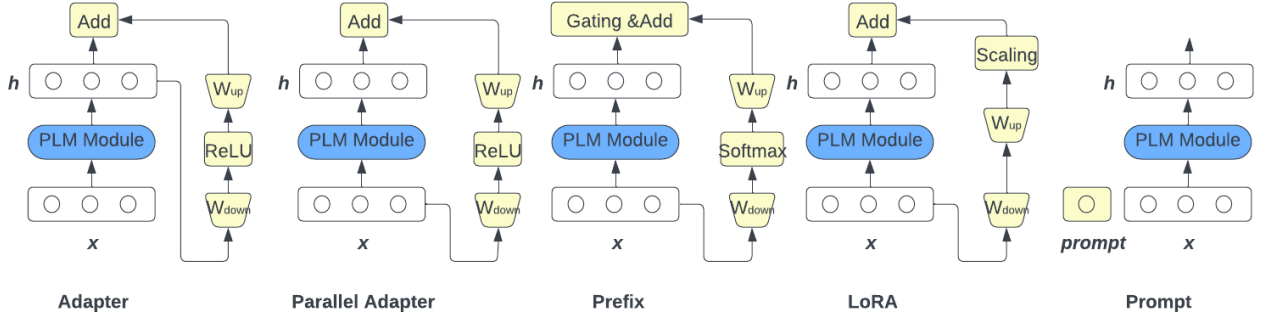


Figure 14: Illustration of parameter-efficient fine-tuning paradigms, where x is input and h is hidden state, figure courtesy [38]. Parallel adapter and LoRA fall in the adapter tuning category.

employs multiple adapter modules in a single layer. AdaMix routes input instances randomly to one of the multiple down-scale and up-scale modules. The mixture of adapters is averaged out for inference to avoid additional latency. Low-Rank Adaptation (LoRA) [250] learns low-rank decomposed matrices to freeze original weights. The learned weights are fused with the original weights for inference, avoiding latency.

Prompt Tuning: Prompting is an effective way to adapt a pre-trained LLM for the downstream task. However, manual prompts bring uncertainty in the model’s prediction, where a change in a single word drops the performance [247]. Prompt tuning alleviates this problem by fine-tuning only 0.001%-3% additional parameters [251]. It concatenates trainable prompt parameters with the model embeddings [247, 40, 251]. Task-specific fixed discrete prompts are concatenated with input embeddings in [40]. As discrete prompts bring instability, prompts are encoded through a learnable mapping in P-Tuning [247], naming continuous prompts, which are appended with the discrete prompts. Only the prompt encoder is trainable in the model. In an extension of P-Tuning, continuous prompts are concatenated with each layer of the network in [251]. Progressive prompts [252] avoid catastrophic forgetting and transfer previously learned knowledge by sequentially adding trainable prompt embeddings to the previously frozen task embeddings.

Prefix Tuning: A set of trainable task-specific prefix vectors are appended to the frozen transformer layers in prefix tuning [41]. The prefix vectors are virtual tokens attended by the context tokens on the right. In addition, adaptive prefix tuning [253] applies a gating mechanism to control the information from the prefix and actual tokens.

Bias Tuning: Fine-tuning only bias terms in small to medium training data has been found effective in BitFit [254]. This method achieves full fine-tuning performance for tasks with less training data and comparable performance with more training data.

3.6.2. Quantization

LLMs require extensive computing and memory for inference. Deploying a 175B parameter GPT-3 model needs at least five 80GB A100 GPUs and 350GB of memory to store in

FP16 format [44]. Such demanding requirements for deploying LLMs make it harder for smaller organizations to utilize them. Model compression is an effective solution but comes at the cost of degraded performance, especially at large scales greater than 6B. These models exhibit very large magnitude outliers that do not exist in smaller models [255], making it challenging and requiring specialized methods for quantizing LLMs [44, 256].

Post-Training Quantization: Minimal or no training is required in this type of quantization, without significantly compromising the model performance. LLM-8-bit [255] uses full-precision matrix multiplication for weights associated with outlier features and 8-bit for remaining features. The lower precision multiplication outputs are converted to FP-16 and concatenated with others. The quantized models have homogenous word embeddings, which may degrade their performance. To fix this, token-level knowledge distillation is employed in [45] along with independent quantization scaling factors for each module due to varying weight distribution. Feature distributions are asymmetric and appear in different channels; outlier suppression [257] shifts and scales per-channel activation distributions for effective quantization. SmoothQuant [44] quantizes activations and weights to INT8 format by smoothing activations and migrating the quantization difficulty toward weights. It multiplies the inverse of the smoothing factor with weights, which introduces a few outliers in the weights but is easier to quantify than unsmoothed activations. OPTQ [256] uses the optimal brain compression (OBC) [258] algorithm to quantize the model layer-by-layer and update weights to compensate for quantization error. To improve speed and performance, OPTQ updates weights in arbitrary order, employs lazy updates, and uses better Cholesky kernels. Outlier-aware weight quantization (OWQ) [259] uses the OPTQ algorithm for quantization but assigns higher precision to vulnerable weights, causing outliers and lower precision for others.

Quantization-Aware Training: To compensate for performance degradation, a quantized model is fine-tuned in quantization-aware training (QAT) [260, 261, 262]. Alpha Tuning quantizes the model using binary coding quantization (BCQ) [263] and fine-tunes only quantization scaling factors. This approach improves performance over

parameter-efficient fine-tuning of the pre-trained model. Similarly, parameter-efficient and quantization-aware adaptation (PEQA) [264] reduces the precision of fully-connected layers and fine-tunes only quantization scaling parameters. LLM-QAT [262] generates training data from the pre-trained network and trains a quantized student model with knowledge distillation. QLoRA [261] fine-tunes 4-bit quantized pre-trained LLM with LoRA [250] using a 4-bit normal float, which shows better performance over a 4-bit integer and float.

3.6.3. Pruning

Pruning is an alternative approach to quantization to compress model size, thereby reducing LLMs deployment costs significantly. Compared to task-agnostic pruning, task-specific pruning is easily achievable with good performance, where a model is fine-tuned on the downstream task and pruned for faster inference. It is possible to prune LLMs for individual tasks, but the cost of pruning and deploying task-specific models is high. To overcome this, many structured and unstructured pruning methods for LLMs have been proposed to maintain reasonable performance across all tasks while shrinking the model size [265, 42, 266].

Unstructured Pruning: This kind of pruning removes less important weights without maintaining any structure. Existing LLM pruning methods take advantage of the unique characteristics of LLMs, uncommon for smaller models, where a small subset of hidden states are activated with large magnitude [255]. Pruning by weights and activations (Wanda) [265] prunes weights in every row based on importance, calculated by multiplying the weights with the norm of input. The pruned model does not require fine-tuning, thereby saving computational costs. Outlier weighed layerwise sparsity (OWL) [267] extends Wanda with non-uniform layer pruning. It shows that the number of outliers varies for different layers; therefore, the model should have variable pruning ratios for better performance for every layer. Contrastive pruning (CAP) [43] iteratively prunes the model by training the sparse model using contrastive loss between pre-trained, fine-tuned, and snapshots of previous sparse models to learn task-specific and task-agnostic knowledge.

Structured Pruning: Here, the parameters are removed in groups, rows, columns, or matrices, which speeds up the inference because of effective hardware tensor core utilization [265]. LLM-Pruner [42] employs a 3-stage structured pruning strategy, identifying the groups of hidden states causing each other to activate during the forward-pass, keeping important groups and removing less important ones, and fine-tuning the pruned model with LoRA. Sparsity-induced mask learning (SIMPLE) [268] prunes the network using learnable masks. Similarly, another method prunes LLMs by learning masks and removing unimportant rank-1 components of the factorized weight matrix [266].

3.7. Multimodal LLMs

Inspired by the success of LLMs in natural language processing applications, an increasing number of research works are

now facilitating LLMs to perceive different modalities of information like image [269, 270, 271], video [272, 273, 274], audio [275, 274, 276], etc. Multimodal LLMs (MLLMs) present substantial benefits compared to standard LLMs that process only text. By incorporating information from various modalities, MLLMs can achieve a deeper understanding of context, leading to more intelligent responses infused with a variety of expressions. Importantly, MLLMs align closely with human perceptual experiences, leveraging the synergistic nature of our multisensory inputs to form a comprehensive understanding of the world [276, 26]. Coupled with a user-friendly interface, MLLMs can offer intuitive, flexible, and adaptable interactions, allowing users to engage with intelligent assistants through a spectrum of input methods. According to the ways of constructing models, current MLLMs can be generally divided into three streams: pre-training, fine-tuning, and prompting. In this section, we will discuss more details of these main streams, as well as the important application of MLLMs in visual reasoning.

Pre-training: This stream of MLLMs intends to support different modalities using unified end-to-end models. For instance, Flamingo [269] applies gated cross-attention to fuse vision and language modalities, which are collected from pre-trained and frozen visual encoder and LLM, respectively. Moreover, BLIP-2 [270] proposes a two-stage strategy to pre-train a Querying Transformer (Q-Former) for the alignment between vision and language modalities: in the first stage, vision-language representation learning is bootstrapped from a frozen visual encoder; and in the second stage, a frozen LLM bootstraps vision-to-language generative learning for zero-shot image-to-text generation. Similarly, MiniGPT-4 [277] deploys pre-trained and frozen ViT [278], Q-Former and Vicuna LLM [159], only training the linear projection layer for vision and language modalities alignment.

Fine-tuning: Derived from instruction tuning [16] for NLP tasks [20, 16, 97], researchers are fine-tune pre-trained LLMs using multimodal instructions. Following this method, LLMs can be easily and effectively extended as multimodal chatbots [277, 271, 29] and multimodal task solvers [279, 30, 280]. The key issue of this stream of MLLMs is to collect multimodal instruction-following data for fine-tuning [58]. To address this issue, the solutions of benchmark adaptation [279, 281, 282], self-instruction [19, 31, 283], and hybrid composition [284, 280] are employed, respectively. To mitigate the gap between the original language modality and additional modalities, the learnable interface is introduced to connect different modalities from frozen pre-trained models. Particularly, the learnable interface is expected to work in a parameter-efficient tuning manner: e.g., LLaMA-Adapter [285] applies an efficient transformer-based adapter module for training, and LaVIN [284] dynamically learns the multimodal feature weights using a mixture-of-modality adapter. Different from the learnable interface, the expert models can directly convert multimodalities into language: e.g., VideoChat-Text [272] incorporates Whisper [286], a speech recognition expert model, to generate the captions of given videos for the understanding of following LLMs.

Prompting: Different from the fine-tuning technique that

directly updates the model parameters given task-specific datasets, the prompting technique provides certain context, examples, or instructions to the model, fulfilling specialized tasks without changing the model parameters. Since prompting can significantly reduce the need for large-scale multimodal data, this technique is widely used to construct MLLMs. Particularly, to solve multimodal Chain of Thought (CoT) problems [103], LLMs are prompted to generate both the reasoning process and the answer given multimodal inputs [287]. On this front, different learning paradigms are exploited in practice: for example, Multimodal-CoT [287] involves two stages of rationale generation and answer inference, where the input of the second stage is a combination of the original input and the output of the first stage; and CoT-PT [288] applies both prompt tuning and specific visual bias to generate a chain of reasoning implicitly. In addition to CoT problems, LLMs can also be prompted with multimodal descriptions and tools, effectively dividing complex tasks into sub-tasks [289, 290].

Visual Reasoning Application: Recent visual reasoning systems [291, 292, 216, 293] tend to apply LLMs for better visual information analysis and visual-language integration. Different from previous works [294, 295] that rely on limited VQA datasets and small-scale neural networks, current LLM-aided methods offer benefits of stronger generalization ability, emergent ability, and interactivity [58]. To realize visual reasoning with the help of LLMs, prompting and fine-tuning techniques can also be utilized: for example, PointClip V2 [292] applies LLMs to generate 3D-specific prompts, which are encoded as textual features and then combined with visual features for 3D recognition; and GPT4Tools [31] employs LoRA [250] to fine-tune LLMs following tool-related instructions. Serving as a controller [293], decision maker [296], or semantics refiner [291, 297], LLMs significantly facilitates the progress of visual reasoning research.

3.8. Summary and Discussion

3.8.1. Architecture

Due to the gigantic scale of LLMs, minor changes in architecture and training strategies have a big impact on performance and stability. Here, we summarize key architectural modules used in various LLMs, leading to better performance, reduced training time and memory, and better training stability.

Layer Normalization: The performance and training stability of LLMs are affected significantly by layer normalization. Pre-norm, that is normalizing inputs rather than outputs, is more common among LLMs stabilizing the training [6, 127, 108]. BLOOM [13] and AlexaTM [122] utilize an additional layer normalization before embedding layer to stabilize the training of large-scale models, while the model’s zero-shot generalization ability can be negatively impacted [13]. However, another study [33] finds that pre-norm degrades fine-tuned model performance as compared to post-norm, and there are no stability benefits of pre-norm beyond the 100B scale. Therefore, GLM-130B [33] used deep-norm which is a variant of post-norm for better downstream task performance after fine-tuning.

Positional Encoding: Like other building blocks of the model,

positional encoding also affects the performance and training stability of LLMs. BLOOM [13] finds ALiBi outperforms learned and rotary positional encodings. Contrary to this, GLM-130B [33] identifies rotary positional encoding as being better than ALiBi. So, there is no conclusion in the literature about positional encodings yet.

Parallel Attention: In this type of attention, feed-forward and attention layers are parallel to each other rather than sequential in a transformer block. It has been shown to reduce training time by 15%. There is no evidence of performance drop due to this change in the literature and it is used by the models PaLM [15], GPT-NeoX [118], and CodeGen [140].

Multi-Query Attention It has shared key and value attention heads in a transformer block while query attention heads are projected as usual. This reduces memory usage and speeds up sampling in autoregressive decoding. No performance degradation has been observed with this change and it makes the training efficient allowing larger batch sizes. Multi-query attention is used in [15, 142].

Mixture of Experts: This type of architecture enables easily scaling models to trillions of parameters [92, 91]. Only a few experts are activated during the computation making them compute-efficient. The performance of MoE models is better than dense models for the same amount of data and requires less computation during fine-tuning to achieve performance similar to dense models as discussed in [91]. MoE architectures are less prone to catastrophic forgetting, therefore are more suited for continual learning [92]. Extracting smaller sub-models for downstream tasks is possible without losing any performance, making MoE architecture hardware-friendly [92].

Sparse vs Dense Activated: GPT-3 [6] uses sparse transformers [67] whereas GLaM [91] and PanGu- Σ [92] use MoE [121] architectures to lower computational costs and increase the model size and capacity. According to the literature, sparse modules do not degrade the model’s performance [67]. However, more experiments are required to verify this statement.

3.8.2. Training Strategies

Training models at a huge scale require tricks to reduce training costs, avoid loss divergence, and achieve better performance. We summarize and discuss some of these key tricks used in different LLMs.

Mixed Precision: It is a famous method for LLMs to reduce memory usage and improve training efficiency. In mixed precision, forward and backward passes are performed in FP16 format whereas optimizer states and master weights are kept in FP32 format [120]. A drawback associated with this format change is training instability due to a smaller value range resulting in loss spikes [33]. An alternative to FP16 is BF16 which has a comparatively larger range and performs precision-sensitive operations like gradient accumulation and softmax in FP32 [13]. BF16 has better performance and training stability but uses more memory and is supported on specific hardware, for example, A100 GPUs. Therefore, its adoption in LLMs is limited.

Training Instability: Loss divergence or spiking is a common issue in LLMs that occurs multiple times during training. This

happens in the presence of gradient clipping [15]. To mitigate this problem, many approaches suggest restarting training from an earlier checkpoint [15, 33, 91], skipping 200-500 earlier data batches at the point of divergence in [15] and re-shuffling batches in [91]. The embedding layer gradient shrink proves to further stabilize the training as its gradient norm is significantly larger than the other layers [33]. Another suggestion to improve training stability for larger models is not to use **biases** in dense and norm layers as in [15].

Weight Initialization: It plays a significant role in model convergence and training stability. GPT-NeoX [118] initializes feed-forward layers before residuals with $\frac{2}{L\sqrt{d}}$ as in [153] and other layers with the small initialization scheme [298]. This avoids activations growing exponentially with increasing depth. MT-NLG [117] found higher variance for weight initialization leads to unstable training, hence validating small initialization scheme [298]. Various models perform random weight initialization which can cause bad initialization, Galactica [148] suggests a longer warmup to negate the effect.

Learning Rate: A suitable learning rate is important for stable training. It is suggested to use a lower value [13, 15, 124] with warmup and decay (cosine or linear). Usually, the learning rate is within the range $1e^{-4}$ to $8e^{-4}$. Moreover, MT-NLG (530B) [117] and GPT-NeoX (20B) [118] suggest interpolating learning rates based on the model size using the GPT-3 [6] models ranging between 13B and 175B. This avoids tuning the learning rate hyperparameter.

Training Parallelism: 3D parallelism, a combination of data, pipeline, and tensor parallelism, is the most utilized training parallelism approach in LLMs [33, 15, 14, 13, 117, 115, 112]. In addition to 3D parallelism, BLOOM [13] uses a zero optimizer [37] to shard optimizer states. PanGu- α [108] and PanGu- Σ [92] go beyond 3D parallelism and apply 5D parallelism which additionally contains optimizer parallelism and rematerialization.

Mode Switching: It adds task-related tokens at the beginning of the text during training. These tokens refer to the natural language understanding and natural language generation tasks which are shown to improve downstream task performance in [125, 124, 122]. During fine-tuning and inference, tokens are appended based on the downstream tasks.

Controllable Text Generation: Generating credible and controlled text from a pre-trained model is challenging. GPT-3 [6] and other LLMs use in-context learning to control generated text. While in-context learning helps in controlling the generated text, ERNIE 3.0 Titan [35] suggests using adversarial loss to rank its generated text for credibility and soft prompts such as genre, topic, keywords, sentiment, and length for better control on generated text.

3.8.3. Supervised Models vs Generalized Models

Although generalized models are capable of performing diverse tasks with good performance they have not yet outperformed models trained in supervised settings. The supervised trained models are still state-of-the-art in various NLP tasks by a large margin as shown in [6, 15, 18].

3.8.4. Zero-Shot vs Few-Shot

LLMs perform well in zero-shot and few-shot settings. But the performance difference between zero-shot and few-shot is large for pre-trained models [6, 15], naming LLMs as meta-learners [6]. LLMs zero-shot evaluations underperform unsupervised methods in neural machine translation [6]. The literature shows pre-training is not enough for good zero-shot performance [15, 16]. To improve the zero-shot performance the literature suggests using instruction fine-tuning that improves the zero-shot performance significantly and outperforms baselines. Instruction fine-tuning has also been shown to improve zero-shot generalization to unseen tasks. Another model, Flan-PaLM [16], unlocks zero-shot reasoning with CoT training.

3.8.5. Encoder vs Decoder vs Encoder-Decoder

Traditionally, these architectures perform well for different tasks, for example, encoder-only for NLU tasks, decoder-only for NLG, and encoder-decoder for sequence2sequence modeling. Encoder-only models are famous for smaller models such as Bert [7], RoBERTa [299], etc., whereas LLMs are either decoder-only [6, 118, 13] or encoder-decoder [10, 11, 122]. While decoder-only models are good at NLG tasks, various LLMs, PaLM [15], OPT [14], GPT-3 [6], BLOOM [13], LLaMA [156], are decoder-only models with significant performance gains on both NLU and NLG tasks. In contradiction to this, T5 [10] and UL2 [125] identify encoder-decoder models out-performing decoder-only models. In another study, PaLM [15] finds increasing the size of decoder-only models can reduce the performance gap between decoder-only and encoder-decoder architectures.

Although decoder-only architectures have become a trend for LLMs, many recently proposed approaches [125, 122] use mode-switching tokens in text with encoder-decoder architectures to enable task-specific modes. Similarly, CodeT5+ [34] uses an encoder-decoder architecture with multiple training objectives for different tasks, activating the encoder, decoder, or both according to the tasks. These variations in architecture and training objectives allow a model to perform well in different settings. Because of this dynamic configuration, the future of LLMs can be attributed to encoder-decoder architectures.

4. Model Configurations

We provide different statistics of pre-trained and instruction-tuned models in this section. This includes information such as publication venue, license type, model creators, steps trained, parallelism, etc in Table 3 and Table 4. Architecture details of pre-trained LLMs are available in Table 5. Providing these details for instruction-tuned models is unnecessary because it fine-tunes pre-trained models for instruction datasets. Hence, architectural details are the same as the baselines. Moreover, optimization settings for various LLMs are available in Table 6 and Table 7. We do not include details on precision, warmup, and weight decay in Table 7. These details are not as important as others to mention for instruction-tuned models, and are not provided by the papers.

Table 3: Summary of pre-trained LLMs (>10B). Only the LLMs discussed individually in the previous sections are summarized. “Data/Tokens” is the model’s pre-training data, which is either the number of tokens or data size. “Data Cleaning” indicates whether data cleaning is performed or not. This includes heuristics (Heur), deduplication (Dedup), quality filtering (QF), and privacy filtering (PF). “Cost” is the calculated training cost obtained by multiplying the GPUs/TPUs hourly rate with the number of GPUs and the training time. The actual cost may vary due to many reasons such as using in-house GPUs or getting a discounted rate, re-training, number of employees working on the problem, etc. “Training Parallelism” indicates distributed training using data parallelism (D), tensor parallelism (T), pipeline parallelism (P), context parallelism (C), model parallelism (M), optimizer parallelism (OP), and rematerialization (R), where for “Library” column, “DS” is a short form for Deep Speed. In column “Commercial Use”, we assumed a model is for non-commercial purposes if its license is unavailable.

Models	Publication Venue	License Type	Model Creators	Purpose	No. of Params	Commercial Use	Steps Trained	Data/ Tokens	Data Cleaning	No. of Processing Units	Processing Unit Type	Training Time	Calculated Train. Cost	Training Parallelism	Library
T5 [10]	JMLR'20	Apache-2.0	Google	General	11B	✓	1M	1T	Heur+Dedup	1024	TPU v3	-	-	D+M	Mesh TensorFlow
GPT-3 [6]	NeurIPS'20	-	OpenAI	General	175B	×	-	300B	Dedup+QF	-	V100	-	-	M	-
mT5 [11]	NAACL'21	Apache-2.0	Google	General	13B	✓	1M	1T	-	-	-	-	-	-	-
PanGu- α [108]	arXiv'21	Apache-2.0	Huawei	General	200B	✓	260k	1.1TB	Heur+Dedup	2048	Ascend 910	-	-	D+OP+P+O+R	MindSpore
CPM-2 [12]	AI Open'21	MIT	Tsinghua	General	198B	✓	1M	2.6TB	Dedup	-	-	-	-	D+M	JAXFormer
Codex [141]	arXiv'21	-	OpenAI	Coding	12B	×	-	100B	Heur	-	-	-	-	-	-
ERNIE 3.0 [110]	arXiv'21	-	Baidu	General	10B	×	120k*	375B	Heur+Dedup	384	V100	-	-	M*	PaddlePaddle
Jurassic-1 [112]	White-Paper'21	Apache-2.0	AI21	General	178B	✓	-	300B	-	800	GPU	-	-	D+M+P	Megatron+DS
HyperCLOVA [114]	EMNLP'21	-	Naver	General	82B	×	-	300B	Clf+Dedup+PF	1024	A100	321h	1.32 Mil	M	Megatron
Yuan 1.0 [115]	arXiv'21	Apache-2.0	-	General	245B	✓	26k*	180B	Heur+Clf+Dedup	2128	GPU	-	-	D+T+P	-
Gopher [116]	arXiv'21	-	Google	General	280B	×	-	300B	QF+Dedup	4096	TPU v3	920h	13.19 Mil	D+M	JAX+Haiku
ERNIE 3.0 Titan [35]	arXiv'21	-	Baidu	General	260B	×	-	300B	Heur+Dedup	-	Ascend 910	-	-	D+M+P+D*	PaddlePaddle
GPT-NeoX-20B [118]	BigScience'22	Apache-2.0	EleutherAI	General	20B	✓	150k	825GB	None	96	40G A100	-	-	M	Megatron+DS+PyTorch
OPT [14]	arXiv'22	MIT	Meta	General	175B	✓	150k	180B	Dedup	992	80G A100	-	-	D+T	Megatron
BLOOM [13]	arXiv'22	RAIL-1.0	BigScience	General	176B	✓	-	366B	Dedup+PR	384	80G A100	2520h	3.87 Mil	D+T+P	Megatron+DS
Galactica [148]	arXiv'22	Apache-2.0	Meta	Science	120B	×	225k	106B	Dedup	128	80GB A100	-	-	-	Metaseq
GLaM [91]	ICML'22	-	Google	General	1.2T	×	600k*	600B	Clf	1024	TPU v4	-	-	M	GSPMD
LaMDA [150]	arXiv'22	-	Google	Dialog	137B	×	3M	2.81T	Filtered	1024	TPU v3	1384h	4.96 Mil	D+M	Lingvo
MT-NLG [117]	arXiv'22	Apache-v2.0	MS.+Nvidia	General	530B	×	-	270B	-	4480	80G A100	-	-	D+T+P	Megatron+DS
AlphaCode [142]	Science'22	Apache-v2.0	Google	Coding	41B	✓	205k	967B	Heur+Dedup	-	TPU v4	-	-	M	JAX+Haiku
Chinchilla [96]	arXiv'22	-	Google	General	70B	×	-	1.4T	QF+Dedup	-	TPUv4	-	-	-	JAX+Haiku
PaLM [15]	arXiv'22	-	Google	General	540B	×	255k	780B	Heur	6144	TPU v4	-	-	D+M	JAX+T5X
AlexaTM [122]	arXiv'22	Apache v2.0	Amazon	General	20B	×	500k	1.1T	Filtered	128	A100	2880h	1.47 Mil	M	DS
U-PaLM [124]	arXiv'22	-	Google	General	540B	×	20k	-	-	512	TPU v4	120h	0.25 Mil	-	-
UL2 [125]	ICLR'23	Apache-2.0	Google	General	20B	✓	2M	1T	-	512	TPU v4	-	-	M	JAX+T5X
GLM [33]	ICLR'23	Apache-2.0	Multiple	General	130B	×	-	400B	-	768	40G A100	1440h	3.37 Mil	M	-
CodeGen [140]	ICLR'23	Apache-2.0	Salesforce	Coding	16B	✓	650k	577B	Heur+Dedup	-	TPU v4	-	-	D+M	JAXFormer
LLaMA [127]	arXiv'23	-	Meta	General	65B	×	350k	1.4T	Clf+Heur+Dedup	2048	80G A100	504h	4.12 Mil	D+M	xFormers
PanGu Σ [92]	arXiv'23	-	Huawei	General	1.085T	×	-	329B	-	512	Ascend 910	2400h	-	D+OP+P+O+R	MindSpore
BloombergGPT [151]	arXiv'23	-	Bloomberg	Finance	50B	×	139k	569B	Dedup	512	40G A100	1272h	1.97 Mil	M	PyTorch
Xuan Yuan 2.0 [152]	arXiv'23	RAIL-1.0	Du Xiaoman	Finance	176B	✓	-	366B	Filtered	-	80GB A100	-	-	P	DS
CodeT5+ [34]	arXiv'23	BSD-3	Salesforce	Coding	16B	✓	110k	51.5B	Dedup	16	40G A100	-	-	-	DS
StarCoder [147]	arXiv'23	OpenRAIL-M	BigCode	Coding	15.5B	✓	250k	1T	Dedup+QF+PF	512	80G A100	624h	1.28 Mil	D+T+P	Megatron-LM
LLaMA-2 [21]	arXiv'23	LLaMA-2.0	Meta	General	70B	✓	500k	2T	Minimal Filtering	-	80G A100	1.7Mh	-	-	-
PaLM-2 [123]	arXiv'23	-	Google	General	-	×	-	-	Ddedup+PF+QF	-	-	-	-	-	-
LLaMA-3.1 [130]	arXiv'24	LLaMA-3.0	Meta	General	405B	✓	1.2M	15T	Dedup+QF	16k	80G H100	30.84Mh	-	D+T+P+C	PyTorch
Mixtral 8x22B [131]	web'24	Apache-2.0	Mistral AI	General	141B	✓	-	-	-	-	-	-	-	-	-
Snowflake Arctic [132]	web'24	Apache-2.0	Snowflake	General	480B	✓	-	3.5T	-	-	-	-	-	T+P	DS
Nemotron-4 340B [137]	web'24	Nvidia	Nvidia	General	340B	✓	-	9T	-	6144	80G H100	-	-	D+T+P	-
DeepSeek [138]	arXiv'24	MIT	DeepSeek	General	67B	✓	-	2T	Dedup+QF	-	-	300.6Kh	-	D+T+P	DS
DeepSeek-v2 [139]	arXiv'24	MIT	DeepSeek	General	67B	✓	-	8.1T	QF	-	H800	172.8Kh	-	D+P	HAI-LLM

Table 4: Summary of instruction tuned LLMs (>10B). All abbreviations are the same as Table 3. Entries in “Data/Tokens” starting with “S-” represent the number of training samples.

Models	Publication Venue	License Type	Model Creators	Purpose	No. of Params	Commercial Use	Pre-trained Models	Steps Trained	Data/ Tokens	No. of Processing Units	Processing Unit Type	Training Time	Calculated Train. Cost	Train. Parallelism	Library
WebGPT [166]	arXiv'21	-	OpenAI	General	175B	×	GPT-3	-	-	-	-	-	-	-	-
T0 [17]	ICLR'22	Apache-2.0	BigScience	General	11B	✓	T5	-	250B	512	TPU v3	270h	0.48 Mil	-	-
Tk-Instruct [18]	EMNLP'22	MIT	AI2+	General	11B	✓	T5	1000	-	256	TPU v3	4h	0.0036 Mil	-	Google T5
OPT-IML [97]	arXiv'22	-	Meta	General	175B	×	OPT	8k	2B	128	40G A100	-	-	D+T	Megatron
Flan-U-PaLM [16]	ICLR'22	Apache-2.0	Google	General	540B	✓	U-PaLM	30k	-	512	TPU v4	-	-	-	JAX+T5X
mT0 [154]	ACL'23	Apache-2.0	HuggingFace+	General	13B	✓	mT5	-	-	-	-	-	-	-	-
Sparrow [167]	arXiv'22	-	Google	Dialog	70B	×	Chinchilla	-	-	64	TPU v3	-	-	M	-
WizardCoder [164]	arXiv'23	Apache-2.0	HK Bapt.	Coding	15B	×	StarCoder	200	S-78k	-	-	-	-	-	-
Alpaca [158]	Github'23	Apache-2.0	Stanford	General	13B	✓	LLaMA	3-Epoch	S-52k	8	80G A100	3h	600	FSDP	PyTorch
Vicuna [159]	Github'23	Apache-2.0	LMSYS	General	13B	✓	LLaMA	3-Epoch	S-125k	-	-	-	-	FSDP	PyTorch
LIMA [185]	arXiv'23	-	Meta+	General	65B	-	LLaMA	15-Epoch	S-1000	-	-	-	-	-	-
Koala [300]	Github'23	Apache-2.0	UC-Berkley	General	13B	×	LLaMA	2-Epoch	S-472k	8	A100	6h	100	-	JAX/FLAX

5. Datasets and Evaluation

5.1. Training Datasets

Generating training and evaluation datasets is expensive because of the large-scale data demand of LLMs. Hence, datasets for training and benchmarking these models are topics of key importance. A summary of datasets commonly used by LLMs is provided next.

The performance of LLMs largely depends on the training data’s quality, size, and diversity. Preparing training datasets of high quality at a large scale is laborious. Researchers have suggested various pre-training and fine-tuning datasets to enhance LLMs capabilities. We summarize these efforts in Table 8. While numerous training datasets are available in the literature, we cover the most widely used ones in our summary.

Table 5: Architecture details of LLMs. Here, “PE” is the positional embedding, “nL” is the number of layers, “nH” is the number of attention heads, “HS” is the size of hidden states.

Models	Type	Training Objective	Attention	Vocab	Tokenizer	Norm	PE	Activation	Bias	nL	nH	HS
T5 (11B)	Enc-Dec	Span Corruption	Standard	32k	SentencePiece	Pre-RMS Layer	Relative Learned	ReLU	×	24	128	1024
GPT3 (175B)	Causal-Dec	Next Token	Dense+Sparse	-	-	-	-	GeLU	✓	96	96	12288
mT5 (13B)	Enc-Dec	Span Corruption	Standard	250k	SentencePiece	Pre-RMS Layer	Relative	ReLU	-	-	-	-
PanGu- α (200B)	Causal-Dec	Next Token	Standard	40k	BPE	-	-	-	-	64	128	16384
CPM-2 (198B)	Enc-Dec	Span Corruption	Standard	250k	SentencePiece	Pre-RMS Layer	Relative	ReLU	-	24	64	-
Codex (12B)	Causal-Dec	Next Token	Standard	-	BPE+	Pre-Layer	Learned	GeLU	-	96	96	12288
ERNIE 3.0 (10B)	Causal-Dec	Next Token	Standard	-	WordPiece	Post-Layer	Relative	GeLU	-	48	64	4096
Jurassic-1 (178B)	Causal-Dec	Next Token	Standard	256k	SentencePiece*	Pre-Layer	Learned	GeLU	✓	76	96	13824
HyperCLOVA (82B)	Causal-Dec	Next Token	Dense+Sparse	-	BPE*	Pre-Layer	Learned	GeLU	-	64	80	10240
Yuan 1.0 (245B)	Causal-Dec	Next Token	Standard	-	-	-	-	-	-	76	-	16384
Gopher (280B)	Causal-Dec	Next Token	Standard	32k	SentencePiece	Pre-RMS Layer	Relative	GeLU	✓	80	128	16384
ERNIE 3.0 Titan (260B)	Causal-Dec	Next Token	Standard	-	WordPiece	Post-Layer	Relative	GeLU	-	48	192	12288
GPT-NeoX-20B	Causal-Dec	Next Token	Parallel	50k	BPE	Layer	Rotary	GeLU	✓	44	64	-
OPT (175B)	Causal-Dec	Next Token	Standard	-	BPE	-	-	ReLU	✓	96	96	-
BLOOM (176B)	Causal-Dec	Next Token	Standard	250k	BPE	Layer	ALiBi	GeLU	✓	70	112	14336
Galactica (120B)	Causal-Dec	Next Token	Standard	50k	BPE+custom	Layer	Learned	GeLU	×	96	80	10240
GLaM (1.2T)	MoE-Dec	Next Token	Standard	256k	SentencePiece	Layer	Relative	GeLU	✓	64	128	32768
LaMDA (137B)	Causal-Dec	Next Token	Standard	32k	BPE	Layer	Relative	GeLU	-	64	128	8192
MT-NLG (530B)	Causal-Dec	Next Token	Standard	50k	BPE	Pre-Layer	Learned	GeLU	✓	105	128	20480
AlphaCode (41B)	Enc-Dec	Next Token	Multi-query	8k	SentencePiece	-	-	-	-	64	128	6144
Chinchilla (70B)	Causal-Dec	Next Token	Standard	32k	SentencePiece-NFKC	Pre-RMS Layer	Relative	GeLU	✓	80	64	8192
PaLM (540B)	Causal-Dec	Next Token	Parallel+Multi-query	256k	SentencePiece	Layer	RoPE	SwiGLU	×	118	48	18432
AlexaTM (20B)	Enc-Dec	Denosing	Standard	150k	SentencePiece	Pre-Layer	Learned	GeLU	✓	78	32	4096
Sparrow (70B)	Causal-Dec	Pref.&Rule RM	-	32k	SentencePiece-NFKC	Pre-RMS Layer	Relative	GeLU	✓	16*	64	8192
U-PaLM (540B)	Non-Causal-Dec	MoD	Parallel+Multi-query	256k	SentencePiece	Layer	RoPE	SwiGLU	×	118	48	18432
UL2 (20B)	Enc-Dec	MoD	Standard	32k	SentencePiece	-	-	-	-	64	16	4096
GLM (130B)	Non-Causal-Dec	AR Blank Infilling	Standard	130k	SentencePiece	Deep	RoPE	GeGLU	✓	70	96	12288
CodeGen (16B)	Causal-Dec	Next Token	Parallel	-	BPE	Layer	RoPE	-	-	34	24	-
LLaMA (65B)	Causal-Dec	Next Token	Standard	32k	BPE	Pre-RMS Layer	RoPE	SwiGLU	-	80	64	8192
PanGu- Σ (1085B)	Causal-Dec	Next Token	Standard	-	BPE	Fused Layer	-	FastGeLU	-	40	40	5120
BloombergGPT (50B)	Causal-Dec	Next Token	Standard	131k	Unigram	Layer	ALiBi	GeLU	✓	70	40	7680
Xuan Yuan 2.0 (176B)	Causal-Dec	Next Token	Self	250k	BPE	Layer	ALiBi	GeLU	✓	70	112	14336
CodeT5+ (16B)	Enc-Dec	SC+NT+Cont.+Match	Standard	-	Code-Specific	-	-	-	-	-	-	-
StarCoder (15.5B)	Causal-Dec	FIM	Multi-query	49k	BPE	-	Learned	-	-	40	48	6144
LLaMA-2 (70B)	Causal-Dec	Next Token	Grouped-query	32k	BPE	Pre-RMS Layer	RoPE	SwiGLUE	-	-	-	-
PaLM-2	-	MoD	Parallel	-	-	-	-	-	-	-	-	-
LLaMA-3.1 (405B)	Causal-Dec	Next Token	Grouped-query	128k	BPE	Pre-RMS Layer	RoPE	SwiGLU	-	126	128	16384
Nemotron-4 (340B)	Causal-Dec	Next Token	Standard	256k	SentencePiece	-	RoPE	ReLU	×	96	96	18432
DeepSeek (67B)	Causal-Dec	Next Token	Grouped-query	100k	BBPE	Pre-RMS Layer	RoPE	SwiGLU	-	95	64	8192
DeepSeek-v2 (67B)	MoE-Dec	Next Token	Multi-Head Latent	100k	BBPE	Pre-RMS Layer	RoPE	SwiGLU	-	60	128	5120

5.2. Evaluation Datasets and Tasks

The evaluation of LLMs is important in gauging their proficiency and limitations. This process measures the model’s ability to comprehend, generate, and interact with human language across a spectrum of tasks. Evaluating a language model (LM) is divided into two broader categories: 1) natural language understanding (NLU) and 2) natural language generation (NLG). It is emphasized that tasks in NLU and NLG are softly categorized and are often used interchangeably in the literature.

Natural Language Understanding: It measures the language understanding capacity of LMs. It encompasses multiple tasks, including sentiment analysis, text classification, natural language inference (NLI), question answering (QA), common-sense reasoning (CR), mathematical reasoning (MR), reading comprehension (RC), etc.

Natural Language Generation: It assesses the language generation capabilities of LLMs by understanding the provided input context. It includes tasks such as summarization, sentence completion, machine translation (MT), dialogue generation, etc. Numerous datasets are proposed for each task, evaluating LLMs against different characteristics. To provide an overview of evaluation datasets, we briefly discuss a few famous datasets within each category and offer a comprehensive list of datasets in Table 9. Moreover, we show a detailed overview of the training datasets and evaluation tasks and benchmarks used by vari-

ous pre-trained LLMs in Table 10 and fine-tuned LLMs in Table 11. We also compare the top-performing LLMs in various NLP tasks in Table 12.

5.2.1. Multi-task

MMLU [307]: A benchmark that measures the knowledge acquired by models during pretraining and evaluates models in zero-shot and few-shot settings across 57 subjects, testing both world knowledge and problem-solving ability.

SuperGLUE [2]: A more challenging and diverse successor to the GLUE [309] benchmark, SuperGLUE includes a variety of language understanding tasks, such as question answering, natural language inference, and co-reference resolution. It is designed to provide a rigorous test of language understanding and requires significant progress in areas like sample-efficient, transfer, multi-task, and unsupervised or self-supervised learning.

BIG-bench [308]: The BIG-bench (Behavior of Intelligent Generative Models Benchmark) is a large-scale benchmark designed to test the abilities of LLMs across a wide range of tasks, including reasoning, creativity, ethics, and understanding of specific domains.

GLUE [309]: The General Language Understanding Evaluation (GLUE) benchmark is a collection of resources for training, evaluating, and analyzing natural language understanding

Table 6: Summary of optimization settings used for pre-trained LLMs. The values for weight decay, gradient clipping, and dropout are 0.1, 1.0, and 0.1, respectively, for most of the LLMs.

Models	Batch Size	Sequence Length	LR	Warmup	LR Decay	Optimizers			Precision			Weight Decay	Grad Clip	Dropout
						AdaFact	Adan	AdamW	FP16	BF16	Mixed			
T5 (11B)	2 ¹¹	512	0.01	×	inverse square root	✓			-	-	-	-	-	✓
GPT3 (175B)	32K	-	6e-5	✓	cosine		✓		✓			✓	✓	-
mT5 (13B)	1024	1024	0.01	-	inverse square root	✓			-	-	-	-	-	✓
PanGu- α (200B)	-	1024	2e-5	-	-	-	-	-	-	✓	-	-	-	-
CPM-2 (198B)	1024	1024	0.001	-	-	✓			-	-	-	-	-	✓
Codex (12B)	-	-	6e-5	✓	cosine		✓		✓			✓	-	-
ERNIE 3.0 (12B)	6144	512	1e-4	✓	linear		✓		-	-	-	✓	-	-
Jurassic-1 (178B)	3.2M	2048	6e-5	✓	cosine		✓		✓			✓	✓	-
HyperCLOVA (82B)	1024	-	6e-5	-	cosine			✓	-	-	-	✓	-	-
Yuan 1.0 (245B)	<10M	2048	1.6e-4	✓	cosine decay to 10%		✓		-	-	-	✓	-	-
Gopher (280B)	3M	2048	4e-5	✓	cosine decay to 10%		✓			✓		-	✓	-
ERNIE 3.0 Titan (260B)	-	512	1e-4	✓	linear		✓		✓			✓	✓	-
GPT-NeoX-20B	1538	2048	0.97e-5	✓	cosine			✓	✓			✓	✓	×
OPT (175B)	2M	2048	1.2e-4	-	linear			✓	✓			✓	✓	✓
BLOOM (176B)	2048	2048	6e-5	✓	cosine		✓			✓		✓	✓	×
Galactica (120B)	2M	2048	7e-6	✓	linear decay to 10%			✓	-	-	-	✓	✓	✓
GLaM (1.2T)	1M	1024	0.01	-	inverse square root	✓			FP32 + ✓			-	✓	×
LaMDA (137B)	256K	-	-	-	-	-	-	-	-	-	-	-	-	-
MT-NLG (530B)	1920	2048	5e-5	✓	cosine decay to 10%		✓			✓		✓	✓	-
AlphaCode (41B)	2048	1536+768	1e-4	✓	cosine decay to 10%			✓		✓		✓	✓	-
Chinchilla (70B)	1.5M	2048	1e-4	✓	cosine decay to 10%			✓		✓		-	-	-
PaLM (540B)	2048	2048	0.01	-	inverse square root	✓			-	-	-	✓	✓	×
AlexaTM (20B)	2M	1024	1e-4	-	linear decay to 5%		✓			✓		✓	-	✓
U-PaLM (540B)	32	2048	1e-4	-	cosine	✓			-	-	-	-	-	-
UL2 (20B)	1024	1024	-	-	inverse square root	-	-	-	-	-	-	×	-	-
GLM (130B)	4224	2048	8e-5	✓	cosine			✓	✓			✓	✓	✓
CodeGen (16B)	2M	2048	5e-5	✓	cosine		✓		-	-	-	✓	✓	-
LLaMA (65B)	4M Tokens	2048	1.5e-4	✓	cosine decay to 10%			✓	-	-	-	✓	✓	-
PanGu- Σ (1.085T)	512	1024	2e-5	✓	-		✓				✓	-	-	-
BloombergGPT (50B)	2048	2048	6e-5	✓	cosine			✓			✓	✓	✓	×
Xuan Yuan 2.0 (176B)	2048	2048	6e-5	✓	cosine		✓		✓			✓	✓	-
CodeT5+ (16B)	2048	1024	2e-4	-	linear			✓			✓	✓	-	-
StarCoder (15.5B)	512	8k	3e-4	✓	cosine		✓			✓		✓	-	-
LLaMA-2 (70B)	4M Tokens	4k	1.5e-4	✓	cosine			✓		✓		✓	✓	-
LLaMA-3.1 (405B)	16M	8192	8e-5	✓	linear+cosine			✓		✓		-	-	-
Nemotron-4 (340B)	2304	4096	-	-	linear	-	-	-		✓		-	-	×
DeepSeek (67B)	4608	4096	3.2e-4	✓	cosine			✓		✓		✓	✓	-
DeepSeek-v2 (67B)	9216	4k	2.4e-4	✓	step-decay			✓	-	-	-	✓	✓	-

Table 7: Summary of optimization settings used for instruction-tuned LLMs. Values for gradient clipping and dropout are the same as the pre-trained models, while no model uses weight decay for instruction tuning.

Models	Batch Size	Sequence Length	LR	Warmup	LR Decay	Optimizers			Grad Clip	Dropout
						AdaFactor	Adam	AdamW		
WebGPT (175B)	BC:512, RM:32	-	6e-5	-	-		✓		-	-
T0 (11B)	1024	1280	1e-3	-	-	✓			-	✓
Tk-Instruct (11B)	1024	-	1e-5	-	constant	-	-	-	-	-
OPT-IML (175B)	128	2048	5e-5	×	linear		✓		✓	✓
Flan-U-PaLM (540B)	32	-	1e-3	-	constant	✓			-	✓
Sparrow (70B)	RM: 8+16, RL:16	-	2e-6	✓	cosine decay to 10%	✓			✓	×
WizardCoder (15B)	512	2048	2e-5	✓	cosine	-	-	-	-	-
Alpaca (13B)	128	512	1e-5	✓	cosine	-	-	✓	✓	×
Vicuna (13B)	128	-2048	2e-5	✓	cosine			✓	-	×
LIMA (65B)	32	2048	1e-5	×	linear			✓	-	✓

systems. It includes a variety of tasks that test a wide range of linguistic phenomena, making it a comprehensive tool for evaluating language understanding in AI.

5.2.2. Language Understanding

WinoGrande [354]: A large-scale dataset inspired by the original Winograd [357] Schema Challenge tests models on their ability to resolve pronoun ambiguity and encourages the development of models that understand the broad context in natural

language text.

CoQA [316]: A conversational question-answering dataset, CoQA challenges models with questions that rely on conversation history and require free-form text answers. Its diverse content from seven domains makes it a rigorous test for models’ ability to handle a wide range of topics and conversational contexts.

WiC [317]: This dataset assesses a model’s ability to discern word meanings based on context, aiding in tasks related

Table 8: Details of various well-known pre-training and fine-tuning datasets. Here, alignment means aligning with human preferences.

Dataset	Type	Size/Samples	Tasks	Source	Creation	Comments
C4 [10]	Pretrain	806GB	-	Common Crawl	Automated	A clean, multilingual dataset with billions of tokens
mC4 [11]	Pretrain	38.49TB	-	Common Crawl	Automated	A multilingual extension of the C4 dataset, mC4 identifies over 100 languages using cld3 from 71 monthly web scrapes of Common Crawl.
PILE [301]	Pretrain	825GB	-	Common Crawl, PubMed Central, OpenWebText2, ArXiv, GitHub, Books3, and others	Automated	A massive dataset comprised of 22 constituent sub-datasets
ROOTs [302]	Pretrain	1.61TB	-	498 Hugging Face datasets	Automated	46 natural and 13 programming languages
MassiveText [116]	Pretrain	10.5TB	-	MassiveWeb, Books, News, Wikipedia, Github, C4	Automated	99% of the data is in English
Wikipedia [303]	Pretrain	-	-	Wikipedia	Automated	Dump of wikipedia
RedPajama [304]	Pretrain	5TB	-	CommonCrawl, C4, Wikipedia, Github, Books, StackExchange	Automated	Open-source replica of LLaMA dataset
PushShift.io Reddit	Pretrain	21.1GB	-	Reddit	Automated	Submissions and comments on Reddit from 2005 to 2019
BigPython [140]	Pretrain	5.5TB	Coding	GitHub	Automated	-
Pool of Prompt (P3) [17]	Instructions	12M	62	PromptSource	Manual	A Subset of PromptSource, created from 177 datasets including summarization, QA, classification, etc.
xP3 [154]	Instructions	81M	71	P3+Multilingual datasets	Manual	Extending P3 to total 46 languages
Super-NaturalInstructions (SNI) [18]	Instructions	12.4M	1616	Multiple datasets	Manual	Extending P3 with additional multilingual datasets, total 46 languages
Flan [16]	Instructions	15M	1836	Muffin+T0-SF+NIV2	Manual	Total 60 languages
OPT-IML [97]	Instructions	18.1M	1667	-	Manual	-
Self-Instruct [19]	Instructions	82k	175	-	Automated	Generated 52k instructions with 82k samples from 175 seed tasks using GPT-3
Alpaca [158]	Instructions	52k	-	-	Automated	Employed self-instruct method to generate data from text-davinci-003
Vicuna [159]	Instructions	125k	-	ShareGPT	Automated	Conversations shared by users on ShareGPT using public APIs
LLaMA-GPT-4 [160]	Instructions	52k	-	Alpaca	Automated	Recreated Alpaca dataset with GPT-4 in English and Chinese
Unnatural Instructions [305]	Instructions	68k	-	15-Seeds (SNI)	Automated	-
LIMA [185]	Instructions	1k	-	Multiple datasets	Manual	Carefully created samples to test performance with fine-tuning on less data
Anthropic-HH-RLHF [306]	Alignment	142k	-	-	Manual	-
Anthropic-HH-RLHF-2 [178]	Alignment	39k	-	-	Manual	-

to Word Sense Disambiguation.

Wikitext103 [318]: With over 100 million tokens from Wikipedia’s top articles, this dataset is a rich resource for tasks that require understanding long-term dependencies, such as language modeling and translation.

PG19 [319]: This is a digital library of diverse books from Project Gutenberg. It is specifically designed to facilitate research in unsupervised learning and language modeling, with a special focus on long-form content.

C4 [10]: A clean, multilingual dataset, C4 offers billions of tokens from web-crawled data. It is a comprehensive resource for training advanced Transformer models on various languages.

LCQMC [320]: The Large-scale Chinese Question Matching Corpus (LCQMC) is a dataset for evaluating the performance of models in semantic matching tasks. It contains pairs of questions in Chinese and their matching status, making it a valuable resource for research in Chinese language understanding.

5.2.3. Story Cloze and Sentence Completion

StoryCloze [334]: It introduces a new “StoryCloze Test”, a commonsense reasoning framework for evaluating story understanding, generation, and script learning. It considers a model’s

ability to understand and generate coherent and sensible stories.

LAMBADA [335]: This dataset evaluates contextual text understanding through a word prediction task. Models must predict the last word of a passage, which is easy for humans when given the whole passage, but not when given only the last sentence.

5.2.4. Physical Knowledge and World Understanding

PIQA [340]: A dataset that probes the physical knowledge of models, aiming to understand how well they are learning about the real world.

TriviaQA [341]: A dataset that tests models on reading comprehension and open domain question answering (QA) tasks, with a focus on Information Retrieval (IR)-style QA.

ARC [342]: A larger version of the ARC-Challenge, this dataset contains both easy and challenging grade-school level, multiple-choice science questions. It is a comprehensive test of a model’s ability to understand and answer complex questions.

ARC-Easy [342]: A subset of the ARC dataset, ARC-Easy, contains questions that are answered correctly by either a retrieval-based algorithm or a word co-occurrence algorithm.

Table 9: Categorized evaluation datasets used in evaluating LLMs.

Type	Datasets/Benchmarks
Multi-Task	MMLU [307], SuperGLUE [2], BIG-bench [308], GLUE [309], BBH [308], CUGE [310], Zero-CLUE [311], FewCLUE [312], Blended Skill Talk [313], HELM [314], KLUE-STs [315]
Language Understanding	CoQA [316], WiC [317], Wikitext103 [318], PG19 [319], LCQMC [320], QQP [321], WinoGender [322], CB [323], FinRE [324], SanWen [325], AFQMC [311], BQ Corpus [326], CNSS [327], CKBQA 13 [328], CLUENER [311], Weibo [329], AQuA [330], OntoNotes [331], HeadQA [332], Twitter Dataset [333]
Story Cloze and Sentence Completion	StoryCloze [334], LAMBADA [335], LCSTS [336], AdGen [337], E2E [338], CHID [339], CHID-FC [312]
Physical Knowledge and World Understanding	PIQA [340], TriviaQA [341], ARC [342], ARC-Easy [342], ARC-Challenge [342], PROST [343], Open-BookQA [344], WebNLG [345], DogWhistle Insider & Outsider [346]
Contextual Language Understanding	RACE [347], RACE-Middle [347], RACE-High [347], QuAC [348], StrategyQA [349], Quiz Bowl [350], cMedQA [351], cMedQA2 [352], MATINF-QA [353]
Commonsense Reasoning	WinoGrande [354], HellaSwag [355], COPA [356], WSC [357], CSQA [358], SIQA [359], C ³ [360], CLUEWSC2020 [311], CLUEWSC [311], CLUEWSC-FC [312], ReCoRD [361]
Reading Comprehension	SQuAD [362], BoolQ [363], SQuADv2 [364], DROP [365], RTE [366], WebQA [367], CMRC2017 [368], CMRC2018 [369], CMRC2019 [370], COTE-BD [371], COTE-DP [371], COTE-MFW [371], MultiRC [372], Natural Questions [373], CNSE [327], DRCD [374], DuReader [375], Dureader _{robust} [376], DuReader-QG [375], SciQ [377], Sogou-log [378], Dureader _{robust} -QG [376], QA4MRE [379], KorQuAD 1.0 [380], CAIL2018-Task1 & Task2 [381]
Mathematical Reasoning	MATH [382], Math23k [383], GSM8K [384], MathQA [385], MGSM [386], MultiArith [387], AS-Div [388], MAWPS [389], SVAMP [390]
Problem Solving	HumanEval [141], DS-1000 [391], MBPP [392], APPS [382], CodeContests [142]
Natural Language Inference & Logical Reasoning	ANLI [393], MNLI-m [394], MNLI-mm [394], QNLI [362], WNLI [357], OCNLI [311], CMNLI [311], ANLI R1 [393], ANLI R2 [393], ANLI R3 [393], HANS [395], OCNLI-FC [312], LogiQA [396], StrategyQA [349]
Cross-Lingual Understanding	MLQA [397], XNLI [398], PAWS-X [399], XSum [400], XCOPA [401], XWinograd [402], TyDiQA-GoldP [403], MLSum [404]
Truthfulness and Fact Checking	TruthfulQA [405], MultiFC [406], Fact Checking on Fever [407]
Biases and Ethics in AI	ETHOS [408], StereoSet [409], BBQ [410], Winobias [411], CrowS-Pairs [412]
Toxicity	RealToxicityPrompts [413], CivilComments toxicity classification [414]
Language Translation	WMT [415], WMT20 [416], WMT20-enzh [416], EPRSTMT [312], CCPM [417]
Scientific Knowledge	AminoProbe [148], BioLAMA [148], Chemical Reactions [148], Galaxy Clusters [148], Mineral Groups [148]
Dialogue	Wizard of Wikipedia [418], Empathetic Dialogues [419], DPC-generated [96] dialogues, ConvAI2 [420], KdConv [421]
Topic Classification	TNEWS-FC [312], YNAT [315], KLUE-TC [315], CSL [311], CSL-FC [312], IFLYTEK [422]

It is a great starting point for models beginning to explore advanced question-answering.

ARC-Challenge [342]: A rigorous question-answering dataset, ARC-Challenge includes complex, grade-school level questions that demand reasoning beyond simple retrieval, testing the true comprehension capabilities of models.

5.2.5. Contextual Language Understanding

RACE [347]: The RACE dataset is a reading comprehension dataset collected from English examinations in China, which benchmarks AI models for understanding and answering questions on long and complex passages, simulating the challenge of a real-world examination.

RACE-Middle [347]: Another subset of the RACE [347] dataset, RACE-Middle, contains middle school-level English exam questions. It offers a slightly less challenging but academically oriented evaluation of a model’s comprehension skills.

RACE-High [347]: A subset of the RACE [347] dataset, RACE-High consists of high school-level English exam ques-

tions. It is designed to evaluate the comprehension ability of models in a more academic and challenging context.

QuAC [348]: This dataset simulates an information-seeking dialog between students and teachers using hidden Wikipedia text. It introduces unique challenges not found in machine comprehension datasets, making it a valuable resource for advancing dialog systems.

5.2.6. Commonsense Reasoning

HellaSwag [355]: A dataset that challenges models to pick the best ending to a context uses Adversarial Filtering to create a ‘Goldilocks’ zone of complexity, where generated text is absurd to humans but often misclassified by models.

COPA [401]: This dataset evaluates a model’s progress in open-domain commonsense causal reasoning. Each question comprises a premise and two alternatives, and the model must select the more plausible alternative, testing a model’s ability to understand and reason about cause and effect.

WSC [357]: The Winograd Schema Challenge (WSC) is a

Table 10: An illustration of training datasets and evaluation tasks employed by pre-trained LLMs. Here, “QA” is question-answering, “Clf” is classification, “NLI” is natural language inference, “MT” is machine translation, “RC” is reading comprehension, “CR” is commonsense reasoning, “MR” is mathematical reasoning, “Mem.” is memorization.

Models	Training Dataset	Benchmark													Truthful/ Bias/ Toxicity/ Mem.
		BIG- bench	MMLU	Super GLUE	QA	Clf	NLI	MT	Cloze/ Completion	RC	CR	MR	Coding		
T5	C4 [10]			✓	✓		✓	✓	✓	✓	✓	✓			
GPT-3	Common Crawl, WebText, Books Cor- pora, Wikipedia			✓	✓			✓	✓	✓				✓	
mT5	mC4 [11]				✓		✓	✓							
PanGu- α	1.1TB Chinese Text Corpus				✓		✓		✓	✓					
CPM-2	WuDaoCorpus [109]									✓		✓			
Codex	54 million public repositories from Github												✓		
ERNIE-3.0	Chinese text corpora, Baidu Search, Web text, QA-long, QA-short, Poetry and Cou- plet Domain-specific data from medical, law, and financial area Baidu knowledge graph with more than 50 million facts			✓	✓	✓	✓	✓	✓	✓		✓			
Jurassic-1	Wikipedia, OWT, Books, C4, Pile [301], arXiv, GitHub				✓		✓		✓	✓					
HyperCLOVA	Korean blogs, Community sites, News, KiN Korean Wikipedia, Wikipedia (En- glish and Japanese), Modu-Corpus: Mes- senger, News, Spoken and written lan- guage corpus, Web corpus							✓							
Yuan 1.0	Common Crawl, SogouT, Sogou News, Baidu Baike, Wikipedia, Books				✓	✓	✓			✓					
Gopher	subsets of MassiveWeb Books, C4, News, GitHub and Wikipedia samples from Mas- siveText	✓	✓	✓	✓						✓	✓		✓	
ERNIE-3.0 TITAN	Same as ERNIE 3.0 and ERNIE 3.0 ad- versarial dataset, ERNIE 3.0 controllable dataset				✓	✓	✓		✓	✓					
GPT-NeoX-20B	Pile [301]			✓	✓		✓		✓		✓	✓			
OPT	RoBERTa [299], Pile [301], PushShift.io Reddit [423]				✓	✓					✓			✓	
BLOOM	ROOTs [13]			✓			✓	✓	✓				✓	✓	
Galactica	arXiv, PMC, Semantic Scholar, Wikipedia, StackExchange, LibreText, Open Text- books, RefSeq Genome, OEIS, LIPID MAPS, NASAExoplanet, Common Crawl, ScientificCC, AcademicCC, GitHub repos- itories Khan Problems, GSM8K, OneS- mallStep	✓	✓		✓							✓		✓	
GLaM	Filtered Webpages, Social media conversa- tions Wikipedia, Forums, Books, News				✓		✓		✓	✓	✓				
LaMDA	Infiniset : Public documents, Dialogs, Ut- terances													✓	
MT-NLG	Two snapshots of Common Crawl and Books3, OpenWebText2, Stack Exchange, PubMed Abstracts, Wikipedia, PG-19 [242], BookCorpus2, NIH ExPorter, Pile, CC-Stories, RealNews						✓		✓	✓	✓			✓	
AlphaCode	Selected GitHub repositories, CodeCon- tests: Codeforces, Description2Code, Co- deNet												✓		
Chinchilla	MassiveWeb, MassiveText Books, C4, News, GitHub, Wikipedia	✓	✓		✓					✓	✓			✓	
PaLM	webpages, books, Wikipedia, news, arti- cles, source code, social media conversa- tions	✓			✓			✓			✓		✓	✓	
AlexaTM	Wikipedia, mC4			✓			✓	✓			✓			✓	
U-PaLM	Same as PaLM	✓		✓	✓		✓		✓	✓	✓				
UL2	-			✓	✓	✓	✓					✓		✓	
GLM-130B	-	✓	✓						✓						
CodeGen	Pile, BigQuery, BigPython												✓		
LLaMA	CommonCrawl, C4, Github, Wikipedia, Books, arXiv, StackExchange		✓		✓					✓	✓	✓	✓	✓	
PanGu- Σ	WuDaoCorpora, CLUE, Pile, C4, Python code				✓	✓	✓	✓	✓				✓		
BloombergGPT	inPile, Pile, C4, Wikipedia	✓	✓				✓		✓	✓	✓			✓	
CodeT5+	CodeSearchNet, Github Code											✓	✓		
StarCoder	The Stack v1.2		✓									✓	✓	✓	
LLaMA-2	✓	✓		✓						✓	✓	✓	✓		
PaLM-2	Web documents, Code, Books, Maths, Conversation			✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	

Table 11: An illustration of training datasets and evaluation benchmarks used in fine-tuned LLMs. “SNI” is a short of Super-NaturalInstructions.

Models	Training Dataset	BIG-bench	MMLU	BBH	RAFT	FLAN	SNI	PromptSource	TyDiQA	HumanEval	MBPP	Truthful/Bias/Toxicity
T0	Pool of Prompts	✓										
WebGPT	ELI5 [424], ELI5 fact-check [166], TriviaQA [341], ARC-Challenge [342], ARC-Easy [342], Hand-written data, Demonstrations of humans, Comparisons between model-generated answers											✓
Tk-INSTRUCT	SNI [18]						✓					
mT0	xP3 [154]											
OPT-IML	PromptSource [17], FLAN [16], SNI [425], UnifiedSKG [426], CrossFit [427], ExMix [428], T5 [10], Reasoning		✓	✓	✓	✓	✓	✓				
Flan	Muffin, T0-SF, Niv2, CoT		✓	✓					✓			
WizardCoder	Code Alpaca									✓	✓	

reading comprehension task in which a system must resolve references in a text, often requiring world knowledge and reasoning about the text.

CSQA [358]: The CommonsenseQA is a question-answering dataset that requires commonsense knowledge to evaluate the ability of AI models to understand and answer questions.

5.2.7. Reading Comprehension

BoolQ [363]: A dataset derived from Google search queries, BoolQ challenges models to answer binary (yes/no) questions. The questions are naturally occurring and are paired with a paragraph from a Wikipedia article containing the answer. It is a test of reading comprehension and reasoning.

SQuADv2 [364]: The Stanford Question Answering Dataset (SQuAD) [362] is a collection of questions posed by crowd workers on a set of Wikipedia articles, where the answer to every question is a segment of text from the corresponding reading passage. SQuADv2 combines the original SQuAD1.1 dataset with over 50,000 unanswerable questions. The aim is to evaluate a model’s ability to understand and answer questions based on a given context and to determine when a question is unanswerable.

DROP [365]: DROP, or Discrete Reasoning Over the content of Paragraphs, is designed to test a model’s ability to understand a wide variety of reading phenomena. It encourages comprehensive and reliable evaluation of reading comprehension capabilities.

RTE [366]: The Recognizing Textual Entailment (RTE) datasets come from a series of annual competitions on textual entailment, predicting whether a given sentence logically follows from another and evaluating a model’s understanding of logical relationships in a text.

WebQA [367]: A dataset for open-domain question answering, WebQA offers a large collection of web-based question-answer pairs. It is designed to assess the ability of AI models to understand and answer questions based on web content.

CMRC2018 [369]: This dataset is a test of Chinese language models’ ability to reason comprehensively and is designed with a challenging span-extraction format that pushes the boundaries

of machine performance.

5.2.8. Mathematical Reasoning

MATH [382]: This dataset is a platform for evaluating the mathematical problem-solving abilities of AI models. It contains a diverse set of math problems, ranging from arithmetic to calculus, and is designed to test the model’s ability to understand and solve complex mathematical problems.

Math23k [383]: This one challenges a model’s ability to understand and solve mathematical word problems. It contains 23,000 Chinese arithmetic word problems that require models to perform reasoning and computation based on the problem description.

GSM8K [384]: A dataset of diverse grade school math word problems, testing a model’s ability to perform multi-step mathematical reasoning.

5.2.9. Problem Solving and Logical Reasoning

ANLI [393]: A large-scale dataset designed to test the robustness of machine learning models in Natural Language Inference (NLI) is created through an iterative, adversarial process where humans try to generate examples that models cannot correctly classify.

HumanEval [141]: A dataset for evaluating the problem-solving ability of AI models, which includes a diverse set of tasks that require various cognitive abilities, making it a comprehensive tool for assessing general intelligence in AI.

StrategyQA [349]: A question-answering dataset that requires reasoning over multiple pieces of evidence to evaluate the strategic reasoning ability of AI models, pushing the boundaries of what machines can understand and answer.

5.2.10. Cross-Lingual Understanding

XNLI [398]: A cross-lingual benchmark, XNLI extends the MultiNLI [429] corpus to 15 languages, including low-resource ones like Urdu. It tests models on cross-lingual sentence understanding, with 112,500 annotated pairs across three categories: entailment, contradiction, and neutral.

PAWS-X [399]: PAWS-X, or Cross-lingual Paraphrase Adversaries from Word Scrambling, is a multilingual version of the

PAWS [430] dataset for paraphrase identification. It includes examples in seven languages and is designed to evaluate the performance of cross-lingual paraphrase identification models.

5.2.11. Truthfulness

Truthful-QA [405]: A unique benchmark that measures a language model’s truthfulness when generating answers. The dataset includes questions across various categories like health, law, and politics, some designed to test the model against common human misconceptions.

5.2.12. Biases and Ethics in AI

ETHOS [408]: ETHOS is a hate speech detection dataset built from YouTube and Reddit comments. It is a tool in the fight against online hate speech, offering binary and multi-label variants for robust content moderation.

StereoSet [409]: StereoSet is a comprehensive dataset designed to measure and evaluate the presence of stereotypical biases in language models. It focuses on four key domains: gender, profession, race, and religion. Contrasting stereotypical bias against language modeling ability provides a valuable tool for understanding and mitigating biases in large language models.

6. Applications

Applying Large Language Models (LLMs) to a variety of downstream tasks has become a popular trend in both AI-related research communities and industries, with many emerging uses being discovered and explored daily. LLMs, which are capable of understanding and generating human-like text, have found meaningful applications across a variety of fields. This section provides an overview of LLM applications in medicine, education, science, mathematics, law, finance, robotics, and coding. While each of these domains pose different challenges, LLMs open up opportunities to make significant contributions to these domains through their generalizability.

General Purpose: LLMs are being widely considered as general-purpose tools for a wide variety of tasks [431]. This is due to their inherent ability to understand, generate, and manipulate human-like text in a contextually relevant manner. This allows them to perform tasks ranging from simple language translation and question-answering to more complex tasks like summarization, text generation, and even programming help [432]. The utility of LLMs is further enhanced by their ability to adapt to the specific style and tone of the text they are processing, making the outputs more user-friendly and context-aware. In everyday applications, LLMs can be used as personal assistants, helping users draft emails or schedule appointments [433]; they can also be deployed in customer service to handle common questions or applied to generate content for digital platforms like websites by creating human-like text based on given prompts [434]. Moreover, LLMs play a crucial role in data analysis, where they can filter large volumes of text data, summarize key points, and find patterns that would take humans much longer to identify [435]. Despite their wide-ranging applications, it is essential to remember that LLMs,

similar to any AI system, are only as good as the data they have been trained on.

Medicine: The application of LLMs in the field of medicine is reshaping healthcare delivery and research. For example, LLMs are increasingly used in clinical decision support systems to provide physicians with evidence-based treatment recommendations [436, 437, 438]. By analyzing patient data and medical literature, they can help identify potential diagnoses, suggest appropriate tests, and recommend optimal treatment strategies. Moreover, LLMs can also enhance patient interactions with healthcare systems; e.g., they can be used in chatbot applications [439, 440, 441] to answer patient queries about symptoms or medications, schedule appointments, and even provide essential health advice. For medical research, LLMs are used to extract and filter information from a considerable amount of medical literature, identify relevant studies, summarize findings, and even predict future research trends [442, 443, 444]. For medical education, LLMs can help create training materials, generate exam questions, provide detailed explanations of complex medical topics, and offer personalized feedback to students [445, 446, 447, 448]. They can also simulate patient interactions, enabling students to practice and improve their clinical skills. At a broader level, LLMs can assist in public health initiatives by analyzing media data to detect disease outbreaks, monitor public sentiment towards health policies, and disseminate health information in a clear and understandable manner [449]. LLMs can be employed to support public health initiatives, addressing related issues such as data privacy, the necessity for explainability, and the potential risk of propagating biases [450, 451].

Education: The integration of LLMs into the educational sector offers opportunities to enhance learning experiences, teacher support, and educational content development. For students, by analyzing their learning styles, performance, and preferences, LLMs can provide customized study materials and practice questions to develop personalized learning experiences [452]. For teachers, LLMs can help to create lesson plans and grade assignments and generate diverse and inclusive educational content, significantly saving more time for teaching and student interaction [453, 454]. In language learning, LLMs serve as advanced conversational partners capable of simulating conversations in multiple languages, correcting grammar, enhancing vocabulary, and aiding pronunciation for the needs of fluency in practice [455]. Furthermore, LLMs improve accessibility in education by providing support for students with disabilities. They can generate real-time transcriptions for the hearing impaired, offer reading assistance for the visually impaired, and simplify complex texts for those with learning disabilities [451]. As LLMs continue to evolve, their applications in education can benefit more students and teachers from different perspectives in practice.

Science: Similar to medical applications, LLMs can expedite the research process by quickly analyzing and summarizing scientific literature. By briefing comprehensible and accessible research summaries, LLMs can assist researchers in staying up-to-date with the latest findings, even in fields outside their area of expertise [456, 457]. In addition, LLMs can aid scientists

Table 12: Performance comparison of top performing LLMs across various NLU and NLG tasks. Here, “N-Shots” indicate the number of example prompts provided to the model during the evaluation, representing its capability in few-shot or zero-shot learning settings, “f” represents the fine-tuned version, and “B” represents the benchmark.

Task	Dataset/Benchmark	Top-1		Top-2		Top-3	
		Model (Size)	Score (N-shots)	Model (Size)	Score (N-shots)	Model (Size)	Score (N-shots)
Multi-Task	BIG-bench (B)	Chinchilla (70B)	65.1 (5-shot)	Gopher (280B)	53.97 (5-shot)	PaLM (540B)	53.7 (5-shot)
	MMLU (B)	GPT-4 (-)	86.4 (5-shot)	Gemini (Ultra)	83.7 (5-shot)	Flan-PaLM-2 _(f) (Large)	81.2 (5-shot)
Language Understanding	SuperGLUE (B)	ERNIE 3.0 (12B)	90.6 (-)	PaLM _(f) (540B)	90.4 (-)	T5 (11B)	88.9 (-)
Story Comprehension and Generation	HellaSwag	GPT-4 (-)	95.3 (10-shot)	Gemini (Ultra)	87.8 (10-shot)	PaLM-2 (Large)	86.8 (one shot)
	StoryCloze	GPT3 (175B)	87.7 (few shot)	PaLM-2 (Large)	87.4 (one shot)	OPT (175B)	79.82 (-)
Physical Knowledge and World Understanding	PIQA	PaLM-2 (Large)	85.0 (one shot)	LLaMa (65B)	82.8 (zero shot)	MT-NLG (530B)	81.99 (zero shot)
	TriviaQA	PaLM-2 (Large)	86.1 (one shot)	LLaMA-2 (70B)	85.0 (one shot)	PaLM (540B)	81.4 (one shot)
Contextual Language Understanding	LAMBADA	PaLM (540B)	89.7 (few shot)	MT-NLG (530B)	87.15 (few shot)	PaLM-2 (Large)	86.9 (one shot)
Commonsense Reasoning	WinoGrande	GPT-4 (-)	87.5 (5-shot)	PaLM-2 (Large)	83.0 (one shot)	PaLM (540B)	81.1 (zero shot)
	SIQA	LLaMA (65B)	52.3 (zero shot)	Chinchilla (70B)	51.3 (zero shot)	Gopher (280B)	50.6 (zero shot)
Reading Comprehension	BoolQ	PaLM _(f) (540B)	92.2 (-)	T5 (11B)	91.2 (-)	PaLM-2 (Large)	90.9 (one shot)
Truthfulness	Truthful-QA	LLaMA (65B)	57 (-)				
Mathematical Reasoning	MATH	Gemini (Ultra)	53.2 (4-shot)	PaLM-2 (Large)	34.3 (4-shot)	LLaMa-2 (65B)	13.5 (4-shot)
	GSM8K	GPT-4 (-)	92.0 (5-shot)	PaLM-2 (Large)	80.7 (8-shot)	U-PaLM (540B)	58.5 (-)
Problem Solving and Logical Reasoning	HumanEval	Gemini _(f) (Ultra)	74.4 (zero shot)	GPT-4 (-)	67.0 (zero shot)	Code Llama (34B)	48.8 (zero shot)

in formulating new hypotheses and research questions since their ability to process large-scale datasets allows them to unveil insights that might not be immediately apparent to human researchers [458]. Moreover, for scientific writing, LLMs can help researchers draft documents, suggest improvements, and ensure adherence to specific formatting guidelines [459, 460]. This not only saves time but also improves the clarity of scientific communication, enabling interdisciplinary teams to work together more effectively.

Maths: In addition to providing mathematical research and education support, LLMs can assist in solving mathematical problems by giving step-by-step explanations and guiding users through complex proofs and calculations. They can help identify errors in reasoning or computation and suggest corrections, serving as an invaluable tool for both learning and verification purposes [461, 462]. LLMs can be employed to check the validity of mathematical proofs, offering a preliminary filter before human review. While they are not a substitute for the meticulous work of mathematicians, they can help simplify the process of proof verification [463, 464]. Moreover, LLMs enhance accessibility to mathematics by translating complex concepts and findings into understandable language for non-specialists [465], where the gap between theoretical mathematics and applied contexts such as physics, engineering, and economics can be bridged.

Law: LLMs can assist with the thematic analysis of legal documents, including generating initial coding for datasets, identifying themes, and classifying data according to these themes. This collaborative effort between legal experts and LLMs has proved to be effective in analyzing legal texts such as court opinions on theft, improving both the efficiency and quality of the research [466]. Additionally, LLMs have been evaluated for their ability to generate explanations of legal terms, focusing on improving factual accuracy and relevance by incorporating sentences from case law. By feeding relevant case law into the LLM, the augmented models can generate higher-quality explanations with less factually incorrect information [467]. Moreover, LLMs can be trained with specialized domain knowledge

to perform legal reasoning tasks [468] and answer legal questions [469].

Finance: LLMs like BloombergGPT [151], trained on extensive proprietary financial datasets, exhibit superior performance on financial tasks. This indicates the value of domain-specific training in creating LLMs that can more accurately understand and process industry-specific language and concepts. The introduction of FinGPT [470] as an open-source model offers transparent and accessible resources to develop novel applications such as robo-advising, algorithmic trading, and low-code solutions, ultimately expanding the capabilities of financial services. Both BloombergGPT and FinGPT show the adaptability of LLMs to the financial domain, with the former showing the power of custom datasets and the latter emphasizing a data-centric approach and low-rank adaptation techniques for customization. Moreover, LLMs demonstrate an ability to break down complex financial tasks into actionable plans, enabling end-to-end solutions that were previously unfeasible with a single model [471].

Robotics: In robotics research, LLMs have promising applications, such as enhancing human-robot interaction [28, 472, 473, 474], task planning [237], motion planning [246], navigation [246, 475], object manipulation [236], personalized robots [476], etc. LLMs enable robots to understand the environment effectively and generate plans to complete tasks collaboratively [240, 26]. They can facilitate continuous learning by allowing robots to access and integrate information from a wide range of sources, helping robots acquire new skills, adapt to changes, and refine their paths [224, 233, 234].

7. Challenges and Future Directions

LLMs such as GPT-4 and its predecessors have significantly advanced natural language processing. Nevertheless, they also bring along a set of challenges. The computational cost, adversarial robustness, and interpretability are among the technical challenges that are intrinsic to these models. Furthermore, as these models are scaled up to handle more complex

tasks or to operate in more complex or dynamic environments, new challenges in scalability, privacy, and real-time processing emerge. On the frontier of foundational research, integrating multi-modality and the effectiveness of transfer learning are being keenly explored. Additionally, the continuous learning aspect of these models, which aims to have models that can adapt to new information over time, presents a fresh set of challenges. These challenges not only underscore the technical intricacies involved but also highlight the broader impact and the future trajectory of LLMs in real-world applications. The following sections delve into these challenges, shedding light on the ongoing and potential efforts to address them.

Computational Cost: Training LLMs require extensive computational resources, which increases production costs and raises environmental concerns due to substantial energy consumption during large-scale training. Improved performance occurs as computational resources increase, but the rate of improvement gradually decreases when both the model and dataset size remain fixed, following the power law of diminishing returns [477].

Bias and Fairness: LLMs can inherit and amplify societal biases in their training data. These biases can manifest in the model’s outputs, leading to potential ethical and fairness issues [478].

Overfitting: Although LLMs possess substantial learning capabilities, they are susceptible to overfitting noisy and peculiar patterns within their extensive training data. Consequently, this may cause them to generate illogical responses [479]. The debate about Memorization vs. Generalization in LLMs is about finding the right balance. Memorization allows the model to remember specific details from its training data, ensuring it can provide accurate answers to precise questions. However, generalization enables the model to make inferences and produce responses for inputs it has not seen before, which is essential for handling various real-world tasks. Striking the right balance is the challenge: too much memorization can lead to overfitting, making the model inflexible and struggling with new inputs [480].

Economic and Research Inequality: The high cost of training and deploying LLMs may make their development concentrated within well-funded organizations, potentially worsening economic and research inequalities in AI [481].

Reasoning and Planning: Some reasoning and planning tasks, even as seemingly simple as common-sense planning, which humans find easy, remain well beyond the current capabilities of LLMs evaluated using an assessment framework. This is not entirely unexpected, considering that LLMs primarily generate text completions based on likelihood and offer no solid guarantees in terms of reasoning abilities [482].

Hallucinations: LLMs exhibit “hallucinations”, where they generate responses that, while sounding plausible, are incorrect or do not align with the provided information [483]. Hallucinations can be categorized into three categories.

- Input-conflicting hallucination, wherein LLMs produce content that diverges from the input given by users.
- Context-conflicting hallucination, where LLMs generate

content that contradicts information they have generated earlier.

- Fact-conflicting hallucination involves LLM’s generation of content that does not align with established world knowledge.

Prompt Engineering: Prompts serve as inputs to LLMs, and their syntax and semantics play a crucial role in determining the model’s output. The prompt variations, sometimes counter-intuitive to humans, can result in significant changes in model output and are addressed through prompt engineering, which involves designing natural language queries to guide LLMs responses effectively [484, 32].

Limited Knowledge: Information acquired during pretraining is limited and may become obsolete after some time. Retraining the model using updated data is costly. To generate factually accurate responses, people use a retrieval augmentation pipeline [198]. However, pre-trained models are not trained with retrieval augmentation generation (RAG) [6, 21]; hence, adapting the training pipeline is necessary [193, 25].

Safety and Controllability: Using LLMs comes with the risk of generating harmful, misleading, or inappropriate content, whether by accident or when given specific prompts. Ensuring these models are safely utilized is a significant concern [485].

Security and Privacy: LLMs are prone to leaking personal information and generating false, unethical, misaligned responses. Researchers have explored various security attacks, i.e., backdoor attacks, jailbreaking, prompt injection, and data poisoning, that lead to breaking LLMs security. Therefore, developing better defense mechanisms is essential to ensure LLMs are safe, reliable, and trustworthy for complex AI applications [486].

Multi-Modality: Multi-modal learning, where LLMs are trained on diverse data like text, images, and videos, aims to create models with richer understanding but faces challenges in data alignment, fusion strategies, and higher computational demands.

Catastrophic Forgetting: LLMs are often pre-trained on large datasets and then fine-tuned on domain-specific data, reducing training resources. However, they face issues like domain adaptation and catastrophic forgetting, which hinder the retention of original knowledge when learning new tasks.

Adversarial Robustness: Large Language Models (LLMs) have shown great capabilities in various tasks but are vulnerable to adversarial attacks, where slight, deliberate input alterations can mislead them. Especially with models like BERT, adversarial fine-tuning can enhance robustness, although it sometimes compromises generalization [487]. As LLMs integrate more into complex systems, examining their security properties becomes crucial, given the emerging field of adversarial attacks on LLMs within trustworthy ML [488]. This vulnerability is notable in safety-critical domains, necessitating robust adversarial evaluation tools to ensure LLM reliability [489].

Interpretability and Explainability: The “black-box” nature of LLMs poses challenges in understanding their decision-making, which is crucial for broader acceptance and trust,

especially in sensitive domains. Despite their advanced capabilities, the lack of insight into their operation limits their effectiveness and trustworthiness [490, 491]. Efforts are being made to make LLMs more explainable to promote user trust and to ensure responsible AI usage. Understanding the logic behind LLMs' responses is essential for fostering trust and ensuring they align with human values and legal standards.

Privacy Concerns: Privacy concerns in Large Language Models (LLMs) have escalated with their growth in complexity and size, particularly around data sharing and potential misuse. There is a risk of malicious content creation, filter bypass, and data privacy issues, especially in e-commerce, where protecting customer privacy is crucial. If models are trained on private data, additional concerns arise if such models are made publicly available. LLMs tend to memorize phrases from their training sets, which an adversary could exploit to extract sensitive data, posing a threat to personal privacy [492, 493].

Real-Time Processing: Real-time processing in Large Language Models (LLMs) is pivotal for various applications, especially with the rising popularity of mobile AI applications and concerns regarding information security and privacy. However, LLMs often have hundreds of layers and millions of parameters, which impede real-time processing due to the high computational demands and limited weight storage on hardware platforms, particularly in edge computing environments [494]. While certain efforts like MobileBERT aim to reduce memory requirements, they still face substantial execution overhead due to the large number of model layers, leading to high inference latency.

Long-Term Dependencies: Large Language Models have shown considerable progress in understanding and generating text, yet they often struggle with preserving context and handling long-term dependencies, particularly in complex, multi-turn conversations or long documents. This limitation can lead to incoherent or irrelevant responses.

Hardware Acceleration: The growth of LLMs presents significant hardware challenges due to the increasing computational and memory demands associated with training and deploying these models. GPUs have played a crucial role in meeting the hardware requirements for training LLMs, with the networking industry also evolving to optimize hardware for training workloads. However, the growing size of LLMs, which has been outpacing hardware progress, makes model inference increasingly costly. Model quantization is a promising approach to bridge the widening gap between LLM size and hardware capacity [495]. Although specialized hardware acceleration like GPUs or TPUs can significantly reduce the computational cost, making real-time applications more feasible, they may not fully resolve all limitations, necessitating further advancements in hardware technology.

Regulatory and Ethical Frameworks: The rapid advancements in artificial intelligence have given rise to sophisticated Large Language Models (LLMs) like OpenAI's GPT-4 [157] and Google's Bard. These developments underscore the imperative for regulatory oversight to manage the ethical and social challenges accompanying LLMs' widespread use [496]. For instance, LLMs can generate content that can be used posi-

tively or negatively, emphasizing the need for proactive ethical frameworks and policy measures to guide their responsible use and assign accountability for their outputs [497]. Auditing is identified as a promising governance mechanism to ensure that AI systems, including LLMs, are designed and deployed ethically, legally, and technically robust [498].

8. Conclusion

This article has comprehensively reviewed the developments in LLMs. It contributes to summarizing significant findings of LLMs in the existing literature and provides a detailed analysis of the design aspects, including architectures, datasets, and training pipelines. We identified crucial architectural components and training strategies employed by different LLMs. These aspects are presented as summaries and discussions throughout the article. Moreover, we have discussed the performance differences of LLMs in zero-shot and few-shot settings, explored the impact of fine-tuning, and compared supervised and generalized models and encoder vs. decoder vs. encoder-decoder architectures. A comprehensive review of multi-modal LLMs, retrieval augmented LLMs, LLMs-powered agents, efficient LLMs, datasets, evaluation, applications, and challenges is also provided. This article is anticipated to serve as a valuable resource for researchers, offering insights into the recent advancements in LLMs and providing fundamental concepts and details to develop better LLMs.

Acknowledgement: The author/s would like to acknowledge the support received from Saudi Data and AI Authority (SDAIA) and King Fahd University of Petroleum and Minerals (KFUPM) under SDAIA-KFUPM Joint Research Center for Artificial Intelligence Grant No. JRC-AI-RFP-11.

References

- [1] A. Chernyavskiy, D. Ilvovsky, P. Nakov, Transformers: "the end of history" for natural language processing?, in: Machine Learning and Knowledge Discovery in Databases. Research Track: European Conference, ECML PKDD 2021, Bilbao, Spain, September 13–17, 2021, Proceedings, Part III 21, Springer, 2021, pp. 677–693. [1](#)
- [2] A. Wang, Y. Pruksachatkun, N. Nangia, A. Singh, J. Michael, F. Hill, O. Levy, S. Bowman, Superglue: A stickier benchmark for general-purpose language understanding systems, *Advances in neural information processing systems* 32 (2019). [1](#), [26](#), [29](#)
- [3] D. Adiwardana, M.-T. Luong, D. R. So, J. Hall, N. Fiedel, R. Thoppilan, Z. Yang, A. Kulshreshtha, G. Nemade, Y. Lu, et al., Towards a human-like open-domain chatbot, *arXiv preprint arXiv:2001.09977* (2020). [1](#)
- [4] B. A. y Arcas, Do large language models understand us?, *Daedalus* 151 (2) (2022) 183–197. [2](#)
- [5] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, et al., Language models are unsupervised multitask learners, *OpenAI blog* 1 (8) (2019) 9. [2](#), [7](#)
- [6] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al., Language models are few-shot learners, *Advances in neural information processing systems* 33 (2020) 1877–1901. [2](#), [6](#), [7](#), [8](#), [9](#), [16](#), [18](#), [23](#), [24](#), [25](#), [34](#)
- [7] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, *arXiv preprint arXiv:1810.04805* (2018). [2](#), [18](#), [24](#)

- [8] M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee, L. Zettlemoyer, Deep contextualized word representations, in: NAACL-HLT, Association for Computational Linguistics, 2018, pp. 2227–2237. [2](#)
- [9] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, L. Zettlemoyer, Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension, arXiv preprint arXiv:1910.13461 (2019). [2](#)
- [10] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu, Exploring the limits of transfer learning with a unified text-to-text transformer, *The Journal of Machine Learning Research* 21 (1) (2020) 5485–5551. [2](#), [7](#), [8](#), [18](#), [19](#), [24](#), [25](#), [28](#), [30](#), [31](#)
- [11] L. Xue, N. Constant, A. Roberts, M. Kale, R. Al-Rfou, A. Siddhant, A. Barua, C. Raffel, mt5: A massively multilingual pre-trained text-to-text transformer, arXiv preprint arXiv:2010.11934 (2020). [2](#), [7](#), [8](#), [24](#), [25](#), [28](#), [30](#)
- [12] Z. Zhang, Y. Gu, X. Han, S. Chen, C. Xiao, Z. Sun, Y. Yao, F. Qi, J. Guan, P. Ke, et al., Cpm-2: Large-scale cost-effective pre-trained language models, *AI Open* 2 (2021) 216–224. [2](#), [8](#), [25](#)
- [13] T. L. Scao, A. Fan, C. Akiki, E. Pavlick, S. Ilić, D. Hesslow, R. Castagné, A. S. Luccioni, F. Yvon, M. Gallé, et al., Bloom: A 176b-parameter open-access multilingual language model, arXiv preprint arXiv:2211.05100 (2022). [2](#), [4](#), [9](#), [11](#), [23](#), [24](#), [25](#), [30](#)
- [14] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, et al., Opt: Open pre-trained transformer language models, arXiv preprint arXiv:2205.01068 (2022). [2](#), [9](#), [11](#), [24](#), [25](#)
- [15] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, et al., Palm: Scaling language modeling with pathways, arXiv preprint arXiv:2204.02311 (2022). [2](#), [6](#), [9](#), [11](#), [23](#), [24](#), [25](#)
- [16] H. W. Chung, L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, E. Li, X. Wang, M. Dehghani, S. Brahma, et al., Scaling instruction-finetuned language models, arXiv preprint arXiv:2210.11416 (2022). [2](#), [7](#), [11](#), [16](#), [17](#), [22](#), [24](#), [25](#), [28](#), [31](#)
- [17] V. Sanh, A. Webson, C. Raffel, S. H. Bach, L. Sutawika, Z. Alyafeai, A. Chaffin, A. Stiegler, T. L. Scao, A. Raja, et al., Multitask prompted training enables zero-shot task generalization, arXiv preprint arXiv:2110.08207 (2021). [2](#), [11](#), [16](#), [25](#), [28](#), [31](#)
- [18] Y. Wang, S. Mishra, P. Alipoormolabashi, Y. Kordi, A. Mirzaei, A. Naik, A. Ashok, A. S. Dhanasekaran, A. Arunkumar, D. Stap, et al., Super-naturalinstructions: Generalization via declarative instructions on 1600+ nlp tasks, in: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, 2022, pp. 5085–5109. [2](#), [7](#), [11](#), [16](#), [17](#), [24](#), [25](#), [28](#), [31](#)
- [19] Y. Wang, Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi, H. Hajishirzi, Self-instruct: Aligning language model with self generated instructions, arXiv preprint arXiv:2212.10560 (2022). [2](#), [16](#), [19](#), [22](#), [28](#)
- [20] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al., Training language models to follow instructions with human feedback, *Advances in Neural Information Processing Systems* 35 (2022) 27730–27744. [2](#), [7](#), [11](#), [16](#), [22](#)
- [21] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al., Llama 2: Open foundation and fine-tuned chat models, arXiv preprint arXiv:2307.09288 (2023). [2](#), [7](#), [10](#), [16](#), [25](#), [34](#)
- [22] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, et al., Emergent abilities of large language models, arXiv preprint arXiv:2206.07682 (2022). [2](#)
- [23] T. Webb, K. J. Holyoak, H. Lu, Emergent analogical reasoning in large language models, *Nature Human Behaviour* 7 (9) (2023) 1526–1541. [2](#)
- [24] D. A. Boiko, R. MacKnight, G. Gomes, Emergent autonomous scientific research capabilities of large language models, arXiv preprint arXiv:2304.05332 (2023). [2](#)
- [25] G. Izacard, P. Lewis, M. Lomeli, L. Hosseini, F. Petroni, T. Schick, J. Dwivedi-Yu, A. Joulin, S. Riedel, E. Grave, Few-shot learning with retrieval augmented language models, arXiv preprint arXiv:2208.03299 (2022). [2](#), [18](#), [19](#), [34](#)
- [26] D. Driess, F. Xia, M. S. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, et al., Palm-e: An embodied multimodal language model, arXiv preprint arXiv:2303.03378 (2023). [2](#), [20](#), [22](#), [33](#)
- [27] A. Parisi, Y. Zhao, N. Fiedel, Talm: Tool augmented language models, arXiv preprint arXiv:2205.12255 (2022). [2](#), [19](#), [20](#)
- [28] B. Zhang, H. Soh, Large language models as zero-shot human models for human-robot interaction, arXiv preprint arXiv:2303.03548 (2023). [2](#), [33](#)
- [29] Q. Ye, H. Xu, G. Xu, J. Ye, M. Yan, Y. Zhou, J. Wang, A. Hu, P. Shi, Y. Shi, et al., mplug-owl: Modularization empowers large language models with multimodality, arXiv preprint arXiv:2304.14178 (2023). [2](#), [22](#)
- [30] W. Wang, Z. Chen, X. Chen, J. Wu, X. Zhu, G. Zeng, P. Luo, T. Lu, J. Zhou, Y. Qiao, et al., Visionllm: Large language model is also an open-ended decoder for vision-centric tasks, arXiv preprint arXiv:2305.11175 (2023). [2](#), [22](#)
- [31] R. Yang, L. Song, Y. Li, S. Zhao, Y. Ge, X. Li, Y. Shan, Gpt4tools: Teaching large language model to use tools via self-instruction, arXiv preprint arXiv:2305.18752 (2023). [2](#), [19](#), [22](#), [23](#)
- [32] E. Saravia, Prompt Engineering Guide, <https://github.com/dair-ai/Prompt-Engineering-Guide> (12 2022). [2](#), [7](#), [18](#), [34](#)
- [33] A. Zeng, X. Liu, Z. Du, Z. Wang, H. Lai, M. Ding, Z. Yang, Y. Xu, W. Zheng, X. Xia, et al., Glm-130b: An open bilingual pre-trained model, arXiv preprint arXiv:2210.02414 (2022). [2](#), [10](#), [23](#), [24](#), [25](#)
- [34] Y. Wang, H. Le, A. D. Gotmare, N. D. Bui, J. Li, S. C. Hoi, Codet5+: Open code large language models for code understanding and generation, arXiv preprint arXiv:2305.07922 (2023). [2](#), [11](#), [24](#), [25](#)
- [35] S. Wang, Y. Sun, Y. Xiang, Z. Wu, S. Ding, W. Gong, S. Feng, J. Shang, Y. Zhao, C. Pang, et al., Ernie 3.0 titan: Exploring larger-scale knowledge enhanced pre-training for language understanding and generation, arXiv preprint arXiv:2112.12731 (2021). [2](#), [8](#), [24](#), [25](#)
- [36] J. Rasley, S. Rajbhandari, O. Ruwase, Y. He, Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2020, pp. 3505–3506. [2](#), [5](#)
- [37] S. Rajbhandari, J. Rasley, O. Ruwase, Y. He, Zero: Memory optimizations enable training trillion parameter models, in: SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, IEEE, 2020, pp. 1–16. [2](#), [4](#), [24](#)
- [38] J. He, C. Zhou, X. Ma, T. Berg-Kirkpatrick, G. Neubig, Towards a unified view of parameter-efficient transfer learning, arXiv preprint arXiv:2110.04366 (2021). [2](#), [20](#), [21](#)
- [39] Z. Hu, Y. Lan, L. Wang, W. Xu, E.-P. Lim, R. K.-W. Lee, L. Bing, S. Porra, Llm-adapters: An adapter family for parameter-efficient fine-tuning of large language models, arXiv preprint arXiv:2304.01933 (2023). [2](#), [20](#)
- [40] B. Lester, R. Al-Rfou, N. Constant, The power of scale for parameter-efficient prompt tuning, arXiv preprint arXiv:2104.08691 (2021). [2](#), [8](#), [20](#), [21](#)
- [41] X. L. Li, P. Liang, Prefix-tuning: Optimizing continuous prompts for generation, arXiv preprint arXiv:2101.00190 (2021). [2](#), [20](#), [21](#)
- [42] X. Ma, G. Fang, X. Wang, Llm-pruner: On the structural pruning of large language models, arXiv preprint arXiv:2305.11627 (2023). [2](#), [22](#)
- [43] R. Xu, F. Luo, C. Wang, B. Chang, J. Huang, S. Huang, F. Huang, From dense to sparse: Contrastive pruning for better pre-trained language model compression, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 36, 2022, pp. 11547–11555. [2](#), [22](#)
- [44] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, S. Han, Smoothquant: Accurate and efficient post-training quantization for large language models, in: ICML, Vol. 202 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 38087–38099. [2](#), [21](#)
- [45] C. Tao, L. Hou, W. Zhang, L. Shang, X. Jiang, Q. Liu, P. Luo, N. Wong, Compression of generative pre-trained language models via quantization, arXiv preprint arXiv:2203.10705 (2022). [2](#), [21](#)
- [46] A. Pal, D. Karkhanis, M. Roberts, S. Dooley, A. Sundararajan, S. Naidu, Giraffe: Adventures in expanding context lengths in llms, arXiv preprint arXiv:2308.10882 (2023). [2](#), [17](#)
- [47] B. Peng, J. Quesnelle, H. Fan, E. Shippole, Yarn: Efficient context window extension of large language models, arXiv preprint arXiv:2309.00071 (2023). [2](#), [17](#)
- [48] M. Guo, J. Ainslie, D. Uthus, S. Ontanon, J. Ni, Y.-H. Sung, Y. Yang,

- LongT5: Efficient text-to-text transformer for long sequences, arXiv preprint arXiv:2112.07916 (2021). 2, 18
- [49] S. Chen, S. Wong, L. Chen, Y. Tian, Extending context window of large language models via positional interpolation, arXiv preprint arXiv:2306.15595 (2023). 2, 17
- [50] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, et al., A survey of large language models, arXiv preprint arXiv:2303.18223 (2023). 2, 3, 7
- [51] U. Naseem, I. Razzak, S. K. Khan, M. Prasad, A comprehensive survey on word representation models: From classical to state-of-the-art word representation language models, Transactions on Asian and Low-Resource Language Information Processing 20 (5) (2021) 1–35. 2, 3
- [52] B. Min, H. Ross, E. Sulem, A. P. B. Veyseh, T. H. Nguyen, O. Sainz, E. Agirre, I. Heinz, D. Roth, Recent advances in natural language processing via large pre-trained language models: A survey, arXiv preprint arXiv:2111.01243 (2021). 2, 3
- [53] C. Zhou, Q. Li, C. Li, J. Yu, Y. Liu, G. Wang, K. Zhang, C. Ji, Q. Yan, L. He, et al., A comprehensive survey on pretrained foundation models: A history from bert to chatgpt, arXiv preprint arXiv:2302.09419 (2023). 2, 3
- [54] Q. Dong, L. Li, D. Dai, C. Zheng, Z. Wu, B. Chang, X. Sun, J. Xu, Z. Sui, A survey for in-context learning, arXiv preprint arXiv:2301.00234 (2022). 2, 7, 18
- [55] J. Huang, K. C.-C. Chang, Towards reasoning in large language models: A survey, arXiv preprint arXiv:2212.10403 (2022). 2, 7, 18
- [56] Y. Wang, W. Zhong, L. Li, F. Mi, X. Zeng, W. Huang, L. Shang, X. Jiang, Q. Liu, Aligning large language models with human: A survey, arXiv preprint arXiv:2307.12966 (2023). 2
- [57] X. Zhu, J. Li, Y. Liu, C. Ma, W. Wang, A survey on model compression for large language models, arXiv preprint arXiv:2308.07633 (2023). 2
- [58] S. Yin, C. Fu, S. Zhao, K. Li, X. Sun, T. Xu, E. Chen, A survey on multi-modal large language models, arXiv preprint arXiv:2306.13549 (2023). 2, 22, 23
- [59] J. J. Webster, C. Kit, Tokenization as the initial phase in nlp, in: COLING 1992 volume 4: The 14th international conference on computational linguistics, 1992. 4
- [60] T. Kudo, Subword regularization: Improving neural network translation models with multiple subword candidates, in: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2018, pp. 66–75. 4
- [61] R. Sennrich, B. Haddow, A. Birch, Neural machine translation of rare words with subword units, in: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2016, pp. 1715–1725. 4
- [62] M. Schuster, K. Nakajima, Japanese and korean voice search, in: 2012 IEEE international conference on acoustics, speech and signal processing (ICASSP), IEEE, 2012, pp. 5149–5152. 4
- [63] S. J. Mielke, Z. Alyafei, E. Salesky, C. Raffel, M. Dey, M. Gallé, A. Raja, C. Si, W. Y. Lee, B. Sagot, et al., Between words and characters: A brief history of open-vocabulary modeling and tokenization in nlp, arXiv preprint arXiv:2112.10508 (2021). 4
- [64] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, I. Polosukhin, Attention is all you need, Advances in neural information processing systems 30 (2017). 4, 7
- [65] O. Press, N. Smith, M. Lewis, Train short, test long: Attention with linear biases enables input length extrapolation, in: International Conference on Learning Representations, 2022. URL <https://openreview.net/forum?id=R8sQPpGCv0> 4, 17
- [66] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, Y. Liu, Roformer: Enhanced transformer with rotary position embedding, arXiv preprint arXiv:2104.09864 (2021). 4, 9, 17
- [67] R. Child, S. Gray, A. Radford, I. Sutskever, Generating long sequences with sparse transformers, arXiv preprint arXiv:1904.10509 (2019). 4, 7, 23
- [68] T. Dao, D. Fu, S. Ermon, A. Rudra, C. Ré, Flashattention: Fast and memory-efficient exact attention with io-awareness, Advances in Neural Information Processing Systems 35 (2022) 16344–16359. 4
- [69] K. Hornik, M. Stinchcombe, H. White, Multilayer feedforward networks are universal approximators, Neural networks 2 (5) (1989) 359–366. 4
- [70] V. Nair, G. E. Hinton, Rectified linear units improve restricted boltzmann machines, in: Proceedings of the 27th international conference on machine learning (ICML-10), 2010, pp. 807–814. 4
- [71] D. Hendrycks, K. Gimpel, Gaussian error linear units (gelus), arXiv preprint arXiv:1606.08415 (2016). 4
- [72] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, Dropout: a simple way to prevent neural networks from overfitting, The journal of machine learning research 15 (1) (2014) 1929–1958. 4
- [73] D. Krueger, T. Maharaj, J. Kramár, M. Pezeshki, N. Ballas, N. R. Ke, A. Goyal, Y. Bengio, A. Courville, C. Pal, Zoneout: Regularizing rnns by randomly preserving hidden activations, arXiv preprint arXiv:1606.01305 (2016). 4
- [74] N. Shazeer, Glu variants improve transformer, arXiv preprint arXiv:2002.05202 (2020). 4
- [75] Y. N. Dauphin, A. Fan, M. Auli, D. Grangier, Language modeling with gated convolutional networks, in: International conference on machine learning, PMLR, 2017, pp. 933–941. 4
- [76] J. L. Ba, J. R. Kiros, G. E. Hinton, Layer normalization, arXiv preprint arXiv:1607.06450 (2016). 4
- [77] B. Zhang, R. Sennrich, Root mean square layer normalization, Advances in Neural Information Processing Systems 32 (2019). 4
- [78] A. Baevski, M. Auli, Adaptive input representations for neural language modeling, arXiv preprint arXiv:1809.10853 (2018). 4
- [79] H. Wang, S. Ma, L. Dong, S. Huang, D. Zhang, F. Wei, Deepnet: Scaling transformers to 1,000 layers, arXiv preprint arXiv:2203.00555 (2022). 4
- [80] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, B. Catanzaro, Megatron-lm: Training multi-billion parameter language models using model parallelism, arXiv preprint arXiv:1909.08053 (2019). 4, 5
- [81] "bmtrain: Efficient training for big models.", URL <https://github.com/OpenBMB/BMTrain> 4, 5
- [82] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, et al., Transformers: State-of-the-art natural language processing, in: Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations, 2020, pp. 38–45. 5
- [83] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, et al., Jax: composable transformations of python+ numpy programs (2018). 5
- [84] S. Li, J. Fang, Z. Bian, H. Liu, Y. Liu, H. Huang, B. Wang, Y. You, Colossal-ai: A unified deep learning system for large-scale parallel training, arXiv preprint arXiv:2110.14883 (2021). 5
- [85] J. He, J. Qiu, A. Zeng, Z. Yang, J. Zhai, J. Tang, Fastmoe: A fast mixture-of-expert training system, arXiv preprint arXiv:2103.13262 (2021). 5
- [86] L. Huawei Technologies Co., Huawei mindspore ai development framework, in: Artificial Intelligence Technology, Springer, 2022, pp. 137–162. 5
- [87] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al., Pytorch: An imperative style, high-performance deep learning library, Advances in neural information processing systems 32 (2019). 5
- [88] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, et al., Tensorflow: a system for large-scale machine learning., in: Osdi, Vol. 16, Savannah, GA, USA, 2016, pp. 265–283. 5
- [89] T. Chen, M. Li, Y. Li, M. Lin, N. Wang, M. Wang, T. Xiao, B. Xu, C. Zhang, Z. Zhang, Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems, arXiv preprint arXiv:1512.01274 (2015). 5
- [90] W. Fedus, B. Zoph, N. Shazeer, Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, The Journal of Machine Learning Research 23 (1) (2022) 5232–5270. 5, 9
- [91] N. Du, Y. Huang, A. M. Dai, S. Tong, D. Lepikhin, Y. Xu, M. Krikun, Y. Zhou, A. W. Yu, O. Firat, et al., Glam: Efficient scaling of language models with mixture-of-experts, in: International Conference on Machine Learning, PMLR, 2022, pp. 5547–5569. 5, 9, 23, 24, 25
- [92] X. Ren, P. Zhou, X. Meng, X. Huang, Y. Wang, W. Wang, P. Li, X. Zhang, A. Podolskiy, G. Arshinov, et al., Pangu-Σ: Towards trillion parameter language model with sparse heterogeneous computing, arXiv preprint arXiv:2303.10845 (2023). 5, 10, 16, 23, 24, 25
- [93] T. Wang, A. Roberts, D. Hesslow, T. Le Scao, H. W. Chung, I. Beltagy, J. Launay, C. Raffel, What language model architecture and pretrain-

- ing objective works best for zero-shot generalization?, in: International Conference on Machine Learning, PMLR, 2022, pp. 22964–22984. 5
- [94] L. Dong, N. Yang, W. Wang, F. Wei, X. Liu, Y. Wang, J. Gao, M. Zhou, H.-W. Hon, Unified language model pre-training for natural language understanding and generation, *Advances in neural information processing systems* 32 (2019). 6
- [95] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, D. Amodei, Scaling laws for neural language models, *arXiv preprint arXiv:2001.08361* (2020). 6
- [96] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. d. L. Casas, L. A. Hendricks, J. Welbl, A. Clark, et al., Training compute-optimal large language models, *arXiv preprint arXiv:2203.15556* (2022). 6, 9, 25, 29
- [97] S. Iyer, X. V. Lin, R. Pasunuru, T. Mihaylov, D. Simig, P. Yu, K. Shuster, T. Wang, Q. Liu, P. S. Koura, et al., Opt-1ml: Scaling language model instruction meta learning through the lens of generalization, *arXiv preprint arXiv:2212.12017* (2022). 7, 11, 16, 17, 22, 25, 28
- [98] Z. Sun, Y. Shen, Q. Zhou, H. Zhang, Z. Chen, D. Cox, Y. Yang, C. Gan, Principle-driven self-alignment of language models from scratch with minimal human supervision, *arXiv preprint arXiv:2305.03047* (2023). 7, 17
- [99] A. Askell, Y. Bai, A. Chen, D. Drain, D. Ganguli, T. Henighan, A. Jones, N. Joseph, B. Mann, N. DasSarma, et al., A general language assistant as a laboratory for alignment, *arXiv preprint arXiv:2112.00861* (2021). 7
- [100] D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, G. Irving, Fine-tuning language models from human preferences, *arXiv preprint arXiv:1909.08593* (2019). 7
- [101] S. Kim, S. J. Joo, D. Kim, J. Jang, S. Ye, J. Shin, M. Seo, The cot collection: Improving zero-shot and few-shot learning of language models via chain-of-thought fine-tuning, *arXiv preprint arXiv:2305.14045* (2023). 7, 16
- [102] Q. Liu, F. Zhou, Z. Jiang, L. Dou, M. Lin, From zero to hero: Examining the power of symbolic tasks in instruction tuning, *arXiv preprint arXiv:2304.07995* (2023). 7, 16
- [103] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al., Chain-of-thought prompting elicits reasoning in large language models, *Advances in Neural Information Processing Systems* 35 (2022) 24824–24837. 7, 20, 23
- [104] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, *arXiv preprint arXiv:2203.11171* (2022). 7, 20
- [105] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. L. Griffiths, Y. Cao, K. Narasimhan, Tree of thoughts: Deliberate problem solving with large language models, *arXiv preprint arXiv:2305.10601* (2023). 7, 20
- [106] N. Houshy, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, S. Gelly, Parameter-efficient transfer learning for nlp, in: *International Conference on Machine Learning*, PMLR, 2019, pp. 2790–2799. 7, 20
- [107] S. McCandlish, J. Kaplan, D. Amodei, O. D. Team, An empirical model of large-batch training, *arXiv preprint arXiv:1812.06162* (2018). 7
- [108] W. Zeng, X. Ren, T. Su, H. Wang, Y. Liao, Z. Wang, X. Jiang, Z. Yang, K. Wang, X. Zhang, et al., Pangu- α : Large-scale autoregressive pre-trained chinese language models with auto-parallel computation, *arXiv preprint arXiv:2104.12369* (2021). 8, 23, 24, 25
- [109] S. Yuan, H. Zhao, Z. Du, M. Ding, X. Liu, Y. Cen, X. Zou, Z. Yang, J. Tang, Wudaocorpora: A super large-scale chinese corpora for pre-training language models, *AI Open* 2 (2021) 65–68. 8, 30
- [110] Y. Sun, S. Wang, S. Feng, S. Ding, C. Pang, J. Shang, J. Liu, X. Chen, Y. Zhao, Y. Lu, et al., Ernie 3.0: Large-scale knowledge enhanced pre-training for language understanding and generation, *arXiv preprint arXiv:2107.02137* (2021). 8, 25
- [111] Z. Dai, Z. Yang, Y. Yang, J. Carbonell, Q. V. Le, R. Salakhutdinov, Transformer-xl: Attentive language models beyond a fixed-length context, *arXiv preprint arXiv:1901.02860* (2019). 8
- [112] O. Lieber, O. Sharir, B. Lenz, Y. Shoham, Jurassic-1: Technical details and evaluation, *White Paper. AI21 Labs* 1 (2021). 8, 24, 25
- [113] Y. Levine, N. Wies, O. Sharir, H. Bata, A. Shashua, Limits to depth efficiencies of self-attention, *Advances in Neural Information Processing Systems* 33 (2020) 22640–22651. 8, 11
- [114] B. Kim, H. Kim, S.-W. Lee, G. Lee, D. Kwak, D. H. Jeon, S. Park, S. Kim, S. Kim, D. Seo, et al., What changes can large-scale language models bring? intensive study on hyperclova: Billions-scale korean generative pretrained transformers, *arXiv preprint arXiv:2109.04650* (2021). 8, 25
- [115] S. Wu, X. Zhao, T. Yu, R. Zhang, C. Shen, H. Liu, F. Li, H. Zhu, J. Luo, L. Xu, et al., Yuan 1.0: Large-scale pre-trained language model in zero-shot and few-shot learning, *arXiv preprint arXiv:2110.04725* (2021). 8, 24, 25
- [116] J. W. Rae, S. Borgeaud, T. Cai, K. Millican, J. Hoffmann, F. Song, J. Aslanides, S. Henderson, R. Ring, S. Young, et al., Scaling language models: Methods, analysis & insights from training gopher, *arXiv preprint arXiv:2112.11446* (2021). 8, 9, 25, 28
- [117] S. Smith, M. Patwary, B. Norick, P. LeGresley, S. Rajbhandari, J. Casper, Z. Liu, S. Prabhunoye, G. Zerveas, V. Korthikanti, et al., Using deepspeed and megatron to train megatron-turing nlG 530b, a large-scale generative language model, *arXiv preprint arXiv:2201.11990* (2022). 8, 9, 24, 25
- [118] S. Black, S. Biderman, E. Hallahan, Q. Anthony, L. Gao, L. Golding, H. He, C. Leahy, K. McDonnell, J. Phang, et al., Gpt-neox-20b: An open-source autoregressive language model, *arXiv preprint arXiv:2204.06745* (2022). 9, 23, 24, 25
- [119] W. Ben, K. Aran, Gpt-j-6b: A 6 billion parameter autoregressive language model (2021). 9
- [120] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, et al., Mixed precision training, *arXiv preprint arXiv:1710.03740* (2017). 9, 23
- [121] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, J. Dean, Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, *arXiv preprint arXiv:1701.06538* (2017). 9, 23
- [122] S. Soltan, S. Ananthakrishnan, J. FitzGerald, R. Gupta, W. Hamza, H. Khan, C. Peris, S. Rawls, A. Rosenbaum, A. Rumshisky, et al., Alex-atm 20b: Few-shot learning using a large-scale multilingual seq2seq model, *arXiv preprint arXiv:2208.01448* (2022). 9, 23, 24, 25
- [123] R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen, et al., Palm 2 technical report, *arXiv preprint arXiv:2305.10403* (2023). 9, 25
- [124] Y. Tay, J. Wei, H. W. Chung, V. Q. Tran, D. R. So, S. Shakeri, X. Garcia, H. S. Zheng, J. Rao, A. Chowdhery, et al., Transcending scaling laws with 0.1% extra compute, *arXiv preprint arXiv:2210.11399* (2022). 9, 24, 25
- [125] Y. Tay, M. Dehghani, V. Q. Tran, X. Garcia, J. Wei, X. Wang, H. W. Chung, D. Bahri, T. Schuster, S. Zheng, et al., Ul2: Unifying language learning paradigms, in: *The Eleventh International Conference on Learning Representations*, 2022. 9, 10, 24, 25
- [126] Z. Du, Y. Qian, X. Liu, M. Ding, J. Qiu, Z. Yang, J. Tang, Glm: General language model pretraining with autoregressive blank infilling, in: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022, pp. 320–335. 10
- [127] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al., Llama: Open and efficient foundation language models, *arXiv preprint arXiv:2302.13971* (2023). 10, 23, 25
- [128] M. N. Rabe, C. Staats, Self-attention does not need $o(n^2)$ memory, *arXiv preprint arXiv:2112.05682* (2021). 10
- [129] V. A. Korthikanti, J. Casper, R. Lym, L. McAfee, M. Andersch, M. Shoeybi, B. Catanzaro, Reducing activation recomputation in large transformer models, *Proceedings of Machine Learning and Systems* 5 (2023). 10
- [130] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, et al., The llama 3 herd of models, *arXiv preprint arXiv:2407.21783* (2024). 10, 25
- [131] <https://mistral.ai/news/mixtral-8x22b/>. 10, 25
- [132] <https://github.com/Snowflake-Labs/snowflake-arctic>. 10, 25
- [133] <https://github.com/xai-org/grok-1>. 10
- [134] <https://x.ai/blog/grok-1.5>. 10
- [135] G. Team, R. Anil, S. Borgeaud, Y. Wu, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, et al., Gemini: a family of highly capable multimodal models, *arXiv preprint arXiv:2312.11805* (2023). 10
- [136] M. Reid, N. Savinov, D. Teplyashin, D. Lepikhin, T. Lillicrap, J.-b.

- Alayrac, R. Soricut, A. Lazaridou, O. Firat, J. Schrittwieser, et al., Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, arXiv preprint arXiv:2403.05530 (2024). 10
- [137] B. Adler, N. Agarwal, A. Aithal, D. H. Anh, P. Bhattacharya, A. Brundyn, J. Casper, B. Catanzaro, S. Clay, J. Cohen, et al., Nemotron-4 340b technical report, arXiv preprint arXiv:2406.11704 (2024). 10, 25
- [138] X. Bi, D. Chen, G. Chen, S. Chen, D. Dai, C. Deng, H. Ding, K. Dong, Q. Du, Z. Fu, et al., Deepseek llm: Scaling open-source language models with longtermism, arXiv preprint arXiv:2401.02954 (2024). 10, 25
- [139] DeepSeek-AI, A. Liu, B. Feng, B. Wang, B. Wang, B. Liu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Yang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Chen, J. Yuan, J. Qiu, J. Song, K. Dong, K. Gao, K. Guan, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Pan, R. Xu, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Zheng, T. Wang, T. Pei, T. Yuan, T. Sun, W. L. Xiao, W. Zeng, W. An, W. Liu, W. Liang, W. Gao, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Chen, X. Nie, X. Sun, Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model, CoRR abs/2405.04434 (2024). 10, 25
- [140] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, C. Xiong, Codegen: An open large language model for code with multi-turn program synthesis, arXiv preprint arXiv:2203.13474 (2022). 11, 23, 25, 28
- [141] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al., Evaluating large language models trained on code, arXiv preprint arXiv:2107.03374 (2021). 11, 25, 29, 31
- [142] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, et al., Competition-level code generation with alphacode, Science 378 (6624) (2022) 1092–1097. 11, 23, 25, 29
- [143] N. Shazeer, Fast transformer decoding: One write-head is all you need, arXiv preprint arXiv:1911.02150 (2019). 11
- [144] R. Y. Pang, H. He, Text generation by learning from demonstrations, arXiv preprint arXiv:2009.07839 (2020). 11
- [145] R. Dabre, A. Fujita, Softmax tempering for training neural machine translation models, arXiv preprint arXiv:2009.09372 (2020). 11
- [146] Y. Wang, W. Wang, S. Joty, S. C. Hoi, Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, arXiv preprint arXiv:2109.00859 (2021). 11
- [147] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, et al., Starcoder: may the source be with you!, arXiv preprint arXiv:2305.06161 (2023). 11, 25
- [148] R. Taylor, M. Kardas, G. Cucurull, T. Scialom, A. Hartshorn, E. Saravia, A. Poulton, V. Kerkez, R. Stojnic, Galactica: A large language model for science, arXiv preprint arXiv:2211.09085 (2022). 11, 24, 25, 29
- [149] FairScale authors, FairScale: A general purpose modular pytorch library for high performance and large scale training, <https://github.com/facebookresearch/fairscale> (2021). 11
- [150] R. Thoppilan, D. De Freitas, J. Hall, N. Shazeer, A. Kulshreshtha, H.-T. Cheng, A. Jin, T. Bos, L. Baker, Y. Du, et al., Lamda: Language models for dialog applications, arXiv preprint arXiv:2201.08239 (2022). 11, 25
- [151] S. Wu, O. Irsoy, S. Lu, V. Dabravolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, G. Mann, Bloomberggpt: A large language model for finance, arXiv preprint arXiv:2303.17564 (2023). 11, 25, 33
- [152] X. Zhang, Q. Yang, D. Xu, Xuanyuan 2.0: A large chinese financial chat model with hundreds of billions parameters, arXiv preprint arXiv:2305.12002 (2023). 11, 17, 25
- [153] W. Ben, Mesh-transformer-jax: Model-parallel implementation of transformer language model with jax (2021). 12, 24
- [154] N. Muennighoff, T. Wang, L. Sutawika, A. Roberts, S. Biderman, T. L. Scao, M. S. Bari, S. Shen, Z.-X. Yong, H. Schoelkopf, et al., Crosslingual generalization through multitask finetuning, arXiv preprint arXiv:2211.01786 (2022). 16, 25, 28, 31
- [155] D. Yin, X. Liu, F. Yin, M. Zhong, H. Bansal, J. Han, K.-W. Chang, Dynosaur: A dynamic growth paradigm for instruction-tuning data curation, arXiv preprint arXiv:2305.14327 (2023). 16
- [156] P. Gao, J. Han, R. Zhang, Z. Lin, S. Geng, A. Zhou, W. Zhang, P. Lu, C. He, X. Yue, et al., Llama-adapter v2: Parameter-efficient visual instruction model, arXiv preprint arXiv:2304.15010 (2023). 16, 24
- [157] Openai. gpt-4 technical report (2023). 16, 35
- [158] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, T. B. Hashimoto, Stanford alpaca: An instruction-following llama model, https://github.com/tatsu-lab/stanford_alpaca (2023). 16, 25, 28
- [159] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez, I. Stoica, E. P. Xing, Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality (March 2023). URL <https://lmsys.org/blog/2023-03-30-vicuna/> 16, 22, 25, 28
- [160] B. Peng, C. Li, P. He, M. Galley, J. Gao, Instruction tuning with gpt-4, arXiv preprint arXiv:2304.03277 (2023). 16, 28
- [161] T. Liu, B. K. H. Low, Goat: Fine-tuned llama outperforms gpt-4 on arithmetic tasks, arXiv preprint arXiv:2305.14201 (2023). 16
- [162] H. Wang, C. Liu, N. Xi, Z. Qiang, S. Zhao, B. Qin, T. Liu, Huatuo: Tuning llama model with chinese medical knowledge, arXiv preprint arXiv:2304.06975 (2023). 16
- [163] C. Xu, Q. Sun, K. Zheng, X. Geng, P. Zhao, J. Feng, C. Tao, D. Jiang, Wizardlm: Empowering large language models to follow complex instructions, arXiv preprint arXiv:2304.12244 (2023). 16
- [164] Z. Luo, C. Xu, P. Zhao, Q. Sun, X. Geng, W. Hu, C. Tao, J. Ma, Q. Lin, D. Jiang, Wizardcoder: Empowering code large language models with evol-instruct, arXiv preprint arXiv:2306.08568 (2023). 16, 25
- [165] J. Menick, M. Trebacz, V. Mikulik, J. Aslanides, F. Song, M. Chadwick, M. Glaese, S. Young, L. Campbell-Gillingham, G. Irving, et al., Teaching language models to support answers with verified quotes, arXiv preprint arXiv:2203.11147 (2022). 17
- [166] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders, et al., Webgpt: Browser-assisted question-answering with human feedback, arXiv preprint arXiv:2112.09332 (2021). 17, 19, 20, 25, 31
- [167] A. Glaese, N. McAleese, M. Trebacz, J. Aslanides, V. Firoiu, T. Ewalds, M. Rauh, L. Weidinger, M. Chadwick, P. Thacker, et al., Improving alignment of dialogue agents via targeted human judgements, arXiv preprint arXiv:2209.14375 (2022). 17, 20, 25
- [168] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, C. Finn, Direct preference optimization: Your language model is secretly a reward model, arXiv preprint arXiv:2305.18290 (2023). 17
- [169] H. Dong, W. Xiong, D. Goyal, R. Pan, S. Diao, J. Zhang, K. Shum, T. Zhang, Raft: Reward ranked finetuning for generative foundation model alignment, arXiv preprint arXiv:2304.06767 (2023). 17
- [170] Z. Yuan, H. Yuan, C. Tan, W. Wang, S. Huang, F. Huang, Rrhf: Rank responses to align language models with human feedback without tears, arXiv preprint arXiv:2304.05302 (2023). 17
- [171] F. Song, B. Yu, M. Li, H. Yu, F. Huang, Y. Li, H. Wang, Preference ranking optimization for human alignment, arXiv preprint arXiv:2306.17492 (2023). 17
- [172] H. Liu, C. Sferrazza, P. Abbeel, Languages are rewards: Hindsight finetuning using human feedback, arXiv preprint arXiv:2302.02676 (2023). 17
- [173] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, et al., Constitutional ai: Harmlessness from ai feedback, arXiv preprint arXiv:2212.08073 (2022). 17
- [174] Y. Dubois, X. Li, R. Taori, T. Zhang, I. Gulrajani, J. Ba, C. Guestrin, P. Liang, T. B. Hashimoto, AlpacaFarm: A simulation framework for methods that learn from human feedback, arXiv preprint arXiv:2305.14387 (2023). 17
- [175] C. Si, Z. Gan, Z. Yang, S. Wang, J. Wang, J. Boyd-Graber, L. Wang, Prompting gpt-3 to be reliable, arXiv preprint arXiv:2210.09150 (2022). 17
- [176] D. Ganguli, A. Askell, N. Schiefer, T. Liao, K. Lukosiute, A. Chen, A. Goldie, A. Mirhoseini, C. Olsson, D. Hernandez, et al., The capacity for moral self-correction in large language models, arXiv preprint arXiv:2302.07459 (2023). 17
- [177] A. Wei, N. Haghtalab, J. Steinhardt, Jailbroken: How does llm safety training fail?, arXiv preprint arXiv:2307.02483 (2023). 17

- [178] D. Ganguli, L. Lovitt, J. Kernion, A. Askell, Y. Bai, S. Kadavath, B. Mann, E. Perez, N. Schiefer, K. Ndousse, et al., Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned, arXiv preprint arXiv:2209.07858 (2022). [17](#), [28](#)
- [179] S. Casper, J. Lin, J. Kwon, G. Culp, D. Hadfield-Menell, Explore, establish, exploit: Red teaming language models from scratch, arXiv preprint arXiv:2306.09442 (2023). [17](#)
- [180] E. Perez, S. Huang, F. Song, T. Cai, R. Ring, J. Aslanides, A. Glaese, N. McAleese, G. Irving, Red teaming language models with language models, arXiv preprint arXiv:2202.03286 (2022). [17](#)
- [181] T. Scialom, T. Chakrabarty, S. Muresan, Fine-tuned language models are continual learners, in: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, 2022, pp. 6107–6122. [17](#)
- [182] Z. Shi, A. Lipani, Don't stop pretraining? make prompt-based fine-tuning powerful learner, arXiv preprint arXiv:2305.01711 (2023). [17](#)
- [183] H. Gupta, S. A. Sawant, S. Mishra, M. Nakamura, A. Mitra, S. Mashetty, C. Baral, Instruction tuned models are quick learners, arXiv preprint arXiv:2306.05539 (2023). [17](#)
- [184] H. Chen, Y. Zhang, Q. Zhang, H. Yang, X. Hu, X. Ma, Y. Yanggong, J. Zhao, Maybe only 0.5% data is needed: A preliminary exploration of low training data instruction tuning, arXiv preprint arXiv:2305.09246 (2023). [17](#)
- [185] C. Zhou, P. Liu, P. Xu, S. Iyer, J. Sun, Y. Mao, X. Ma, A. Efrat, P. Yu, L. Yu, et al., Lima: Less is more for alignment, arXiv preprint arXiv:2305.11206 (2023). [17](#), [25](#), [28](#)
- [186] C. Han, Q. Wang, W. Xiong, Y. Chen, H. Ji, S. Wang, Lm-infinite: Simple on-the-fly length generalization for large language models, arXiv preprint arXiv:2308.16137 (2023). [17](#), [18](#)
- [187] J. Ainslie, T. Lei, M. de Jong, S. Ontañón, S. Brahma, Y. Zemlyanskiy, D. Uthus, M. Guo, J. Lee-Thorp, Y. Tay, et al., Colt5: Faster long-range transformers with conditional computation, arXiv preprint arXiv:2303.09752 (2023). [18](#)
- [188] J. Ding, S. Ma, L. Dong, X. Zhang, S. Huang, W. Wang, F. Wei, Longnet: Scaling transformers to 1,000,000,000 tokens, arXiv preprint arXiv:2307.02486 (2023). [18](#)
- [189] Y. Chen, S. Qian, H. Tang, X. Lai, Z. Liu, S. Han, J. Jia, Longlora: Efficient fine-tuning of long-context large language models, arXiv preprint arXiv:2309.12307 (2023). [18](#)
- [190] N. Ratner, Y. Levine, Y. Belinkov, O. Ram, I. Magar, O. Abend, E. Karpas, A. Shashua, K. Leyton-Brown, Y. Shoham, Parallel context windows for large language models, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2023, pp. 6383–6402. [18](#)
- [191] W. Wang, L. Dong, H. Cheng, X. Liu, X. Yan, J. Gao, F. Wei, Augmenting language models with long-term memory, arXiv preprint arXiv:2306.07174 (2023). [18](#)
- [192] X. Xu, Z. Gou, W. Wu, Z.-Y. Niu, H. Wu, H. Wang, S. Wang, Long time no see! open-domain conversation with long-term persona memory, arXiv preprint arXiv:2203.05797 (2022). [18](#)
- [193] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millikan, G. B. Van Den Driessche, J.-B. Lespiau, B. Damoc, A. Clark, et al., Improving language models by retrieving from trillions of tokens, in: International conference on machine learning, PMLR, 2022, pp. 2206–2240. [18](#), [19](#), [34](#)
- [194] W. Zhong, L. Guo, Q. Gao, Y. Wang, Memorybank: Enhancing large language models with long-term memory, arXiv preprint arXiv:2305.10250 (2023). [18](#)
- [195] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan, S. Yao, Reflexion: Language agents with verbal reinforcement learning, arXiv preprint arXiv:2303.11366 14 (2023). [18](#), [20](#)
- [196] C. Hu, J. Fu, C. Du, S. Luo, J. Zhao, H. Zhao, Chatdb: Augmenting llms with databases as their symbolic memory, arXiv preprint arXiv:2306.03901 (2023). [18](#)
- [197] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, G. Neubig, Active retrieval augmented generation, arXiv preprint arXiv:2305.06983 (2023). [18](#)
- [198] O. Ram, Y. Levine, I. Dalmedigos, D. Muhlgay, A. Shashua, K. Leyton-Brown, Y. Shoham, In-context retrieval-augmented language models, arXiv preprint arXiv:2302.00083 (2023). [18](#), [34](#)
- [199] X. Li, X. Qiu, Mot: Pre-thinking and recalling enable chatgpt to self-improve with memory-of-thoughts, arXiv preprint arXiv:2305.05181 (2023). [18](#)
- [200] D. Schuurmans, Memory augmented large language models are computationally universal, arXiv preprint arXiv:2301.04589 (2023). [18](#)
- [201] A. Modarressi, A. Imani, M. Fayyaz, H. Schütze, Ret-llm: Towards a general read-write memory for large language models, arXiv preprint arXiv:2305.14322 (2023). [18](#)
- [202] S. Robertson, H. Zaragoza, et al., The probabilistic relevance framework: Bm25 and beyond, Foundations and Trends® in Information Retrieval 3 (4) (2009) 333–389. [18](#)
- [203] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, D. Zhou, Rationale-augmented ensembles in language models, arXiv preprint arXiv:2207.00747 (2022). [18](#)
- [204] F. Zhang, B. Chen, Y. Zhang, J. Liu, D. Zan, Y. Mao, J.-G. Lou, W. Chen, Repocoder: Repository-level code completion through iterative retrieval and generation, arXiv preprint arXiv:2303.12570 (2023). [18](#)
- [205] B. Wang, W. Ping, P. Xu, L. McAfee, Z. Liu, M. Shoenybi, Y. Dong, O. Kuchaiev, B. Li, C. Xiao, et al., Shall we pretrain autoregressive language models with retrieval? a comprehensive study, arXiv preprint arXiv:2304.06762 (2023). [19](#)
- [206] L. Wang, N. Yang, F. Wei, Learning to retrieve in-context examples for large language models, arXiv preprint arXiv:2307.07164 (2023). [19](#)
- [207] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, W. Chen, What makes good in-context examples for gpt-3?, arXiv preprint arXiv:2101.06804 (2021). [19](#)
- [208] O. Rubin, J. Herzig, J. Berant, Learning to retrieve prompts for in-context learning, arXiv preprint arXiv:2112.08633 (2021). [19](#)
- [209] W. Shi, S. Min, M. Yasunaga, M. Seo, R. James, M. Lewis, L. Zettlemoyer, W.-t. Yih, Replug: Retrieval-augmented black-box language models, arXiv preprint arXiv:2301.12652 (2023). [19](#)
- [210] O. Rubin, J. Berant, Long-range language modeling with self-retrieval, arXiv preprint arXiv:2306.13421 (2023). [19](#)
- [211] K. Guu, K. Lee, Z. Tung, P. Pasupat, M. Chang, Retrieval augmented language model pre-training, in: International conference on machine learning, PMLR, 2020, pp. 3929–3938. [19](#)
- [212] S. Hofstätter, J. Chen, K. Raman, H. Zamani, Fid-light: Efficient and effective retrieval-augmented text generation, in: Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2023, pp. 1437–1447. [19](#)
- [213] M. Komeili, K. Shuster, J. Weston, Internet-augmented dialogue generation, arXiv preprint arXiv:2107.07566 (2021). [19](#)
- [214] A. Lazaridou, E. Gribovskaya, W. Stokowiec, N. Grigorev, Internet-augmented language models through few-shot prompting for open-domain question answering, arXiv preprint arXiv:2203.05115 (2022). [19](#)
- [215] D. Gao, L. Ji, L. Zhou, K. Q. Lin, J. Chen, Z. Fan, M. Z. Shou, Assistgpt: A general multi-modal assistant that can plan, execute, inspect, and learn, arXiv preprint arXiv:2306.08640 (2023). [19](#)
- [216] P. Lu, B. Peng, H. Cheng, M. Galley, K.-W. Chang, Y. N. Wu, S.-C. Zhu, J. Gao, Chameleon: Plug-and-play compositional reasoning with large language models, arXiv preprint arXiv:2304.09842 (2023). [19](#), [20](#), [23](#)
- [217] B. Paranjape, S. Lundberg, S. Singh, H. Hajishirzi, L. Zettlemoyer, M. T. Ribeiro, Art: Automatic multi-step reasoning and tool-use for large language models, arXiv preprint arXiv:2303.09014 (2023). [19](#)
- [218] C.-Y. Hsieh, S.-A. Chen, C.-L. Li, Y. Fujii, A. Ratner, C.-Y. Lee, R. Krishna, T. Pfister, Tool documentation enables zero-shot tool-usage with large language models, arXiv preprint arXiv:2308.00675 (2023). [19](#)
- [219] Y. Song, W. Xiong, D. Zhu, C. Li, K. Wang, Y. Tian, S. Li, Restgpt: Connecting large language models with real-world applications via restful apis, arXiv preprint arXiv:2306.06624 (2023). [19](#)
- [220] S. Hao, T. Liu, Z. Wang, Z. Hu, Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings, arXiv preprint arXiv:2305.11554 (2023). [19](#)
- [221] S. G. Patil, T. Zhang, X. Wang, J. E. Gonzalez, Gorilla: Large language model connected with massive apis, arXiv preprint arXiv:2305.15334 (2023). [19](#)
- [222] Q. Xu, F. Hong, B. Li, C. Hu, Z. Chen, J. Zhang, On the tool manipulation capability of open-source large language models, arXiv preprint arXiv:2305.16504 (2023). [19](#)
- [223] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, et al., Toolllm: Facilitating large language models to master 16000+ real-world apis, arXiv preprint arXiv:2307.16789 (2023). [19](#)

- [224] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, Y. Zhuang, Hugginggpt: Solving ai tasks with chatgpt and its friends in huggingface, arXiv preprint arXiv:2303.17580 (2023). 19, 20, 33
- [225] Y. Liang, C. Wu, T. Song, W. Wu, Y. Xia, Y. Liu, Y. Ou, S. Lu, L. Ji, S. Mao, et al., Taskmatrix. ai: Completing tasks by connecting foundation models with millions of apis, arXiv preprint arXiv:2303.16434 (2023). 19
- [226] D. Suris, S. Menon, C. Vondrick, Vipergpt: Visual inference via python execution for reasoning, arXiv preprint arXiv:2303.08128 (2023). 20
- [227] A. Maedche, S. Morana, S. Schacht, D. Werth, J. Krumeich, Advanced user assistance systems, *Business & Information Systems Engineering* 58 (2016) 367–370. 20
- [228] M. Campbell, A. J. Hoane Jr, F.-h. Hsu, Deep blue, *Artificial intelligence* 134 (1-2) (2002) 57–83. 20
- [229] S. Hong, X. Zheng, J. Chen, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, et al., Metagpt: Meta programming for multi-agent collaborative framework, arXiv preprint arXiv:2308.00352 (2023). 20
- [230] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, et al., The rise and potential of large language model based agents: A survey, arXiv preprint arXiv:2309.07864 (2023). 20
- [231] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, et al., A survey on large language model based autonomous agents, arXiv preprint arXiv:2308.11432 (2023). 20
- [232] W. Huang, P. Abbeel, D. Pathak, I. Mordatch, Language models as zero-shot planners: Extracting actionable knowledge for embodied agents, in: *International Conference on Machine Learning*, PMLR, 2022, pp. 9118–9147. 20
- [233] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, Z. Hu, Reasoning with language model is planning with world model, arXiv preprint arXiv:2305.14992 (2023). 20, 33
- [234] W. Yao, S. Heinecke, J. C. Niebles, Z. Liu, Y. Feng, L. Xue, R. Murthy, Z. Chen, J. Zhang, D. Arpit, et al., Retroformer: Retrospective large language agents with policy gradient optimization, arXiv preprint arXiv:2308.02151 (2023). 20, 33
- [235] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, T. Jackson, N. Brown, L. Luu, S. Levine, K. Hausman, brian ichter, *Inner monologue: Embodied reasoning through planning with language models*, in: 6th Annual Conference on Robot Learning, 2022. URL <https://openreview.net/forum?id=3R3Pz5i0tye> 20
- [236] C. Jin, W. Tan, J. Yang, B. Liu, R. Song, L. Wang, J. Fu, Alphablock: Embodied finetuning for vision-language reasoning in robot manipulation, arXiv preprint arXiv:2305.18898 (2023). 20, 33
- [237] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, A. Garg, Progprompt: Generating situated robot task plans using large language models, in: 2023 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2023, pp. 11523–11530. 20, 33
- [238] W. Yu, N. Gileadi, C. Fu, S. Kirmani, K.-H. Lee, M. G. Arenas, H.-T. L. Chiang, T. Erez, L. Hasenclever, J. Humplik, et al., Language to rewards for robotic skill synthesis, arXiv preprint arXiv:2306.08647 (2023). 20
- [239] X. Tang, A. Zou, Z. Zhang, Y. Zhao, X. Zhang, A. Cohan, M. Gerstein, Medagents: Large language models as collaborators for zero-shot medical reasoning, arXiv preprint arXiv:2311.10537 (2023). 20
- [240] A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, E. Jang, R. Julian, et al., Do as i can, not as i say: Grounding language in robotic affordances, in: *Conference on Robot Learning*, PMLR, 2023, pp. 287–318. 20, 33
- [241] H. Ha, P. Florence, S. Song, Scaling up and distilling down: Language-guided robot skill acquisition, arXiv preprint arXiv:2307.14535 (2023). 20
- [242] A. Rajvanshi, K. Sikka, X. Lin, B. Lee, H.-P. Chiu, A. Velasquez, Saynav: Grounding large language models for dynamic planning to navigation in new environments, arXiv preprint arXiv:2309.04077 (2023). 20
- [243] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, Y. Su, Llm-planner: Few-shot grounded planning for embodied agents with large language models, arXiv preprint arXiv:2212.04088 (2022). 20
- [244] V. S. Dorbala, J. F. Mullen Jr, D. Manocha, Can an embodied agent find your" cat-shaped mug"? Llm-based zero-shot object navigation, arXiv preprint arXiv:2303.03480 (2023). 20
- [245] C. Huang, O. Mees, A. Zeng, W. Burgard, Visual language maps for robot navigation, in: 2023 IEEE International Conference on Robotics and Automation (ICRA), IEEE, 2023, pp. 10608–10615. 20
- [246] Y. Ding, X. Zhang, C. Paxton, S. Zhang, Task and motion planning with large language models for object rearrangement, arXiv preprint arXiv:2303.06247 (2023). 20, 33
- [247] X. Liu, Y. Zheng, Z. Du, M. Ding, Y. Qian, Z. Yang, J. Tang, Gpt understands, too, arXiv preprint arXiv:2103.10385 (2021). 20, 21
- [248] G. Chen, F. Liu, Z. Meng, S. Liang, Revisiting parameter-efficient tuning: Are we really there yet?, arXiv preprint arXiv:2202.07962 (2022). 20
- [249] Y. Wang, S. Mukherjee, X. Liu, J. Gao, A. H. Awadallah, J. Gao, Adamix: Mixture-of-adapters for parameter-efficient tuning of large language models, arXiv preprint arXiv:2205.12410 1 (2) (2022) 4. 20
- [250] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, Lora: Low-rank adaptation of large language models, arXiv preprint arXiv:2106.09685 (2021). 21, 22, 23
- [251] X. Liu, K. Ji, Y. Fu, W. Tam, Z. Du, Z. Yang, J. Tang, P-tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks, in: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2022, pp. 61–68. 21
- [252] A. Razdaibiedina, Y. Mao, R. Hou, M. Khabza, M. Lewis, A. Almahairi, Progressive prompts: Continual learning for language models, arXiv preprint arXiv:2301.12314 (2023). 21
- [253] Z.-R. Zhang, C. Tan, H. Xu, C. Wang, J. Huang, S. Huang, Towards adaptive prefix tuning for parameter-efficient language model fine-tuning, arXiv preprint arXiv:2305.15212 (2023). 21
- [254] E. B. Zaken, S. Ravfogel, Y. Goldberg, Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models, arXiv preprint arXiv:2106.10199 (2021). 21
- [255] T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer, Llm. int8 (): 8-bit matrix multiplication for transformers at scale, arXiv preprint arXiv:2208.07339 (2022). 21, 22
- [256] E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh, Gptq: Accurate post-training quantization for generative pre-trained transformers, arXiv preprint arXiv:2210.17323 (2022). 21
- [257] X. Wei, Y. Zhang, Y. Li, X. Zhang, R. Gong, J. Guo, X. Liu, Outlier suppression+: Accurate quantization of large language models by equivalent and optimal shifting and scaling, arXiv preprint arXiv:2304.09145 (2023). 21
- [258] E. Frantar, D. Alistarh, Optimal brain compression: A framework for accurate post-training quantization and pruning, *Advances in Neural Information Processing Systems* 35 (2022) 4475–4488. 21
- [259] C. Lee, J. Jin, T. Kim, H. Kim, E. Park, Owq: Lessons learned from activation outliers for weight quantization in large language models, arXiv preprint arXiv:2306.02272 (2023). 21
- [260] S. J. Kwon, J. Kim, J. Bae, K. M. Yoo, J.-H. Kim, B. Park, B. Kim, J.-W. Ha, N. Sung, D. Lee, Alpatuning: Quantization-aware parameter-efficient adaptation of large-scale pre-trained language models, arXiv preprint arXiv:2210.03858 (2022). 21
- [261] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer, Qlora: Efficient finetuning of quantized llms, arXiv preprint arXiv:2305.14314 (2023). 21, 22
- [262] Z. Liu, B. Oguz, C. Zhao, E. Chang, P. Stock, Y. Mehdad, Y. Shi, R. Krishnamoorthi, V. Chandra, Llm-qat: Data-free quantization aware training for large language models, arXiv preprint arXiv:2305.17888 (2023). 21, 22
- [263] Y. Guo, A. Yao, H. Zhao, Y. Chen, Network sketching: Exploiting binary structure in deep cnns, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 5955–5963. 21
- [264] J. Kim, J. H. Lee, S. Kim, J. Park, K. M. Yoo, S. J. Kwon, D. Lee, Memory-efficient fine-tuning of compressed large language models via sub-4-bit integer quantization, arXiv preprint arXiv:2305.14152 (2023). 22
- [265] M. Sun, Z. Liu, A. Bair, J. Z. Kolter, A simple and effective pruning approach for large language models, arXiv preprint arXiv:2306.11695 (2023). 22
- [266] Z. Wang, J. Wohlwend, T. Lei, Structured pruning of large language models, arXiv preprint arXiv:1910.04732 (2019). 22

- [267] L. Yin, Y. Wu, Z. Zhang, C.-Y. Hsieh, Y. Wang, Y. Jia, M. Pechenizkiy, Y. Liang, Z. Wang, S. Liu, Outlier weighed layerwise sparsity (owl): A missing secret sauce for pruning llms to high sparsity, arXiv preprint arXiv:2310.05175 (2023). 22
- [268] C. Tao, L. Hou, H. Bai, J. Wei, X. Jiang, Q. Liu, P. Luo, N. Wong, Structured pruning for efficient generative pre-trained language models, in: Findings of the Association for Computational Linguistics: ACL 2023, 2023, pp. 10880–10895. 22
- [269] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds, et al., Flamingo: a visual language model for few-shot learning, Advances in Neural Information Processing Systems 35 (2022) 23716–23736. 22
- [270] J. Li, D. Li, S. Savarese, S. Hoi, Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models, arXiv preprint arXiv:2301.12597 (2023). 22
- [271] H. Liu, C. Li, Q. Wu, Y. J. Lee, Visual instruction tuning, arXiv preprint arXiv:2304.08485 (2023). 22
- [272] K. Li, Y. He, Y. Wang, Y. Li, W. Wang, P. Luo, Y. Wang, L. Wang, Y. Qiao, Videochat: Chat-centric video understanding, arXiv preprint arXiv:2305.06355 (2023). 22
- [273] M. Maaz, H. Rasheed, S. Khan, F. S. Khan, Video-chatgpt: Towards detailed video understanding via large vision and language models, arXiv preprint arXiv:2306.05424 (2023). 22
- [274] H. Zhang, X. Li, L. Bing, Video-llama: An instruction-tuned audio-visual language model for video understanding, arXiv preprint arXiv:2306.02858 (2023). 22
- [275] X. Mei, C. Meng, H. Liu, Q. Kong, T. Ko, C. Zhao, M. D. Plumbley, Y. Zou, W. Wang, Wavcaps: A chatgpt-assisted weakly-labelled audio captioning dataset for audio-language multimodal research, arXiv preprint arXiv:2303.17395 (2023). 22
- [276] C. Lyu, M. Wu, L. Wang, X. Huang, B. Liu, Z. Du, S. Shi, Z. Tu, Macaw-llm: Multi-modal language modeling with image, audio, video, and text integration, arXiv preprint arXiv:2306.09093 (2023). 22
- [277] D. Zhu, J. Chen, X. Shen, X. Li, M. Elhoseiny, Minigt-4: Enhancing vision-language understanding with advanced large language models, arXiv preprint arXiv:2304.10592 (2023). 22
- [278] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al., An image is worth 16x16 words: Transformers for image recognition at scale, arXiv preprint arXiv:2010.11929 (2020). 22
- [279] W. Dai, J. Li, D. Li, A. M. H. Tiong, J. Zhao, W. Wang, B. Li, P. Fung, S. Hoi, Instructblip: Towards general-purpose vision-language models with instruction tuning, arXiv preprint arXiv:2305.06500 (2023). 22
- [280] Z. Xu, Y. Shen, L. Huang, Multiinstruct: Improving multi-modal zero-shot learning via instruction tuning, arXiv preprint arXiv:2212.10773 (2022). 22
- [281] Z. Zhao, L. Guo, T. Yue, S. Chen, S. Shao, X. Zhu, Z. Yuan, J. Liu, Chatbridge: Bridging modalities with large language model as a language catalyst, arXiv preprint arXiv:2305.16103 (2023). 22
- [282] L. Li, Y. Yin, S. Li, L. Chen, P. Wang, S. Ren, M. Li, Y. Yang, J. Xu, X. Sun, et al., M3 it: A large-scale dataset towards multi-modal multilingual instruction tuning, arXiv preprint arXiv:2306.04387 (2023). 22
- [283] R. Pi, J. Gao, S. Diao, R. Pan, H. Dong, J. Zhang, L. Yao, J. Han, H. Xu, L. K. T. Zhang, Detgpt: Detect what you need via reasoning, arXiv preprint arXiv:2305.14167 (2023). 22
- [284] G. Luo, Y. Zhou, T. Ren, S. Chen, X. Sun, R. Ji, Cheap and quick: Efficient vision-language instruction tuning for large language models, arXiv preprint arXiv:2305.15023 (2023). 22
- [285] R. Zhang, J. Han, A. Zhou, X. Hu, S. Yan, P. Lu, H. Li, P. Gao, Y. Qiao, Llama-adapter: Efficient fine-tuning of language models with zero-init attention, arXiv preprint arXiv:2303.16199 (2023). 22
- [286] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, I. Sutskever, Robust speech recognition via large-scale weak supervision, in: International Conference on Machine Learning, PMLR, 2023, pp. 28492–28518. 22
- [287] Z. Zhang, A. Zhang, M. Li, H. Zhao, G. Karypis, A. Smola, Multi-modal chain-of-thought reasoning in language models, arXiv preprint arXiv:2302.00923 (2023). 23
- [288] J. Ge, H. Luo, S. Qian, Y. Gan, J. Fu, S. Zhan, Chain of thought prompt tuning in vision language models, arXiv preprint arXiv:2304.07919 (2023). 23
- [289] C. Wu, S. Yin, W. Qi, X. Wang, Z. Tang, N. Duan, Visual chatgpt: Talking, drawing and editing with visual foundation models, arXiv preprint arXiv:2303.04671 (2023). 23
- [290] Z. Yang, L. Li, J. Wang, K. Lin, E. Azarnasab, F. Ahmed, Z. Liu, C. Liu, M. Zeng, L. Wang, Mm-react: Prompting chatgpt for multimodal reasoning and action, arXiv preprint arXiv:2303.11381 (2023). 23
- [291] T. Wang, J. Zhang, J. Fei, Y. Ge, H. Zheng, Y. Tang, Z. Li, M. Gao, S. Zhao, Y. Shan, et al., Caption anything: Interactive image description with diverse multimodal controls, arXiv preprint arXiv:2305.02677 (2023). 23
- [292] X. Zhu, R. Zhang, B. He, Z. Zeng, S. Zhang, P. Gao, Pointclip v2: Adapting clip for powerful 3d open-world learning, arXiv preprint arXiv:2211.11682 (2022). 23
- [293] T. Gupta, A. Kembhavi, Visual programming: Compositional visual reasoning without training, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 14953–14962. 23
- [294] P. Gao, Z. Jiang, H. You, P. Lu, S. C. Hoi, X. Wang, H. Li, Dynamic fusion with intra-and inter-modality attention flow for visual question answering, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2019, pp. 6639–6648. 23
- [295] Z. Yu, J. Yu, Y. Cui, D. Tao, Q. Tian, Deep modular co-attention networks for visual question answering, in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2019, pp. 6281–6290. 23
- [296] H. You, R. Sun, Z. Wang, L. Chen, G. Wang, H. A. Ayyubi, K.-W. Chang, S.-F. Chang, Idealgpt: Iteratively decomposing vision and language reasoning via large language models, arXiv preprint arXiv:2305.14985 (2023). 23
- [297] R. Zhang, X. Hu, B. Li, S. Huang, H. Deng, Y. Qiao, P. Gao, H. Li, Prompt, generate, then cache: Cascade of foundation models makes strong few-shot learners, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 15211–15222. 23
- [298] T. Q. Nguyen, J. Salazar, Transformers without tears: Improving the normalization of self-attention, CoRR abs/1910.05895 (2019). 24
- [299] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, V. Stoyanov, Roberta: A robustly optimized bert pre-training approach, arXiv preprint arXiv:1907.11692 (2019). 24, 30
- [300] X. Geng, A. Gudibande, H. Liu, E. Wallace, P. Abbeel, S. Levine, D. Song, Koala: A dialogue model for academic research, Blog post (April 2023).
URL <https://bair.berkeley.edu/blog/2023/04/03/koala/> 25
- [301] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, et al., The pile: An 800gb dataset of diverse text for language modeling, arXiv preprint arXiv:2101.00027 (2020). 28, 30
- [302] H. Laurençon, L. Saulnier, T. Wang, C. Akiki, A. Villanova del Moral, T. Le Scao, L. Von Werra, C. Mou, E. González Ponferrada, H. Nguyen, et al., The bigscience roots corpus: A 1.6 tb composite multilingual dataset, Advances in Neural Information Processing Systems 35 (2022) 31809–31826. 28
- [303] Wikipedia.
URL https://en.wikipedia.org/wiki/Main_Page 28
- [304] Together Computer, Redpajama: An open source recipe to reproduce llama training dataset (Apr. 2023).
URL <https://github.com/togethercomputer/RedPajama-Data> 28
- [305] O. Honovich, T. Scialom, O. Levy, T. Schick, Unnatural instructions: Tuning language models with (almost) no human labor, arXiv preprint arXiv:2212.09689 (2022). 28
- [306] Y. Bai, A. Jones, K. Ndousse, A. Askell, A. Chen, N. DasSarma, D. Drain, S. Fort, D. Ganguli, T. Henighan, et al., Training a helpful and harmless assistant with reinforcement learning from human feedback, arXiv preprint arXiv:2204.05862 (2022). 28
- [307] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, J. Steinhardt, Measuring massive multitask language understanding, arXiv preprint arXiv:2009.03300 (2020). 26, 29
- [308] A. Srivastava, A. Rastogi, A. Rao, A. A. M. Sholeh, A. Abid, A. Fisch, A. R. Brown, A. Santoro, A. Gupta, A. Garriga-Alonso, et al., Beyond

- the imitation game: Quantifying and extrapolating the capabilities of language models, arXiv preprint arXiv:2206.04615 (2022). 26, 29
- [309] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, S. R. Bowman, Glue: A multi-task benchmark and analysis platform for natural language understanding, arXiv preprint arXiv:1804.07461 (2018). 26, 29
- [310] Y. Yao, Q. Dong, J. Guan, B. Cao, Z. Zhang, C. Xiao, X. Wang, F. Qi, J. Bao, J. Nie, et al., Cugc: A chinese language understanding and generation evaluation benchmark, arXiv preprint arXiv:2112.13610 (2021). 29
- [311] L. Xu, H. Hu, X. Zhang, L. Li, C. Cao, Y. Li, Y. Xu, K. Sun, D. Yu, C. Yu, et al., Clue: A chinese language understanding evaluation benchmark, arXiv preprint arXiv:2004.05986 (2020). 29
- [312] L. Xu, X. Lu, C. Yuan, X. Zhang, H. Xu, H. Yuan, G. Wei, X. Pan, X. Tian, L. Qin, et al., Fewclue: A chinese few-shot learning evaluation benchmark, arXiv preprint arXiv:2107.07498 (2021). 29
- [313] E. M. Smith, M. Williamson, K. Shuster, J. Weston, Y.-L. Boureau, Can you put it all together: Evaluating conversational agents’ ability to blend skills, arXiv preprint arXiv:2004.08449 (2020). 29
- [314] P. Liang, R. Bommasani, T. Lee, D. Tsipras, D. Soylu, M. Yasunaga, Y. Zhang, D. Narayanan, Y. Wu, A. Kumar, et al., Holistic evaluation of language models, arXiv preprint arXiv:2211.09110 (2022). 29
- [315] S. Park, J. Moon, S. Kim, W. I. Cho, J. Han, J. Park, C. Song, J. Kim, Y. Song, T. Oh, et al., Klue: Korean language understanding evaluation, arXiv preprint arXiv:2105.09680 (2021). 29
- [316] S. Reddy, D. Chen, C. D. Manning, Coqa: A conversational question answering challenge, Transactions of the Association for Computational Linguistics 7 (2019) 249–266. 27, 29
- [317] M. T. Pilehvar, J. Camacho-Collados, Wic: 10,000 example pairs for evaluating context-sensitive representations, arXiv preprint arXiv:1808.09121 6 (2018). 27, 29
- [318] S. Merity, C. Xiong, J. Bradbury, R. Socher, Pointer sentinel mixture models, arXiv preprint arXiv:1609.07843 (2016). 28, 29
- [319] J. W. Rae, A. Potapenko, S. M. Jayakumar, T. P. Lillicrap, Compressive transformers for long-range sequence modelling, arXiv preprint arXiv:1911.05507 (2019). 28, 29
- [320] X. Liu, Q. Chen, C. Deng, H. Zeng, J. Chen, D. Li, B. Tang, Lcqm: A large-scale chinese question matching corpus, in: Proceedings of the 27th international conference on computational linguistics, 2018, pp. 1952–1962. 28, 29
- [321] S. Iyer, N. Dandekar, K. Csernai, First quora dataset release: Question pairs, <https://quoradata.quora.com/First-Quora-Dataset-Release-Question-Pairs>. 29
- [322] R. Rudinger, J. Naradowsky, B. Leonard, B. Van Durme, Gender bias in coreference resolution, arXiv preprint arXiv:1804.09301 (2018). 29
- [323] M.-C. De Marneffe, M. Simons, J. Tonhauser, The commitmentbank: Investigating projection in naturally occurring discourse, in: proceedings of Sinn und Bedeutung, Vol. 23, 2019, pp. 107–124. 29
- [324] Z. Li, N. Ding, Z. Liu, H. Zheng, Y. Shen, Chinese relation extraction with multi-grained information and external linguistic knowledge, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019, pp. 4377–4386. 29
- [325] J. Xu, J. Wen, X. Sun, Q. Su, A discourse-level named entity recognition and relation extraction dataset for chinese literature text, arXiv preprint arXiv:1711.07010 (2017). 29
- [326] J. Chen, Q. Chen, X. Liu, H. Yang, D. Lu, B. Tang, The bq corpus: A large-scale domain-specific chinese corpus for sentence semantic equivalence identification, in: Proceedings of the 2018 conference on empirical methods in natural language processing, 2018, pp. 4946–4951. 29
- [327] B. Liu, D. Niu, H. Wei, J. Lin, Y. He, K. Lai, Y. Xu, Matching article pairs with graphical decomposition and convolutions, arXiv preprint arXiv:1802.07459 (2018). 29
- [328] P. Li, W. Li, Z. He, X. Wang, Y. Cao, J. Zhou, W. Xu, Dataset and neural recurrent sequence labeling model for open-domain factoid question answering, arXiv preprint arXiv:1607.06275 (2016). 29
- [329] N. Peng, M. Dredze, Named entity recognition for chinese social media with jointly trained embeddings, in: Proceedings of the 2015 conference on empirical methods in natural language processing, 2015, pp. 548–554. 29
- [330] W. Ling, D. Yogatama, C. Dyer, P. Blunsom, Program induction by rationale generation: Learning to solve and explain algebraic word problems, arXiv preprint arXiv:1705.04146 (2017). 29
- [331] R. Weischedel, S. Pradhan, L. Ramshaw, M. Palmer, N. Xue, M. Marcus, A. Taylor, C. Greenberg, E. Hovy, R. Belvin, et al., Ontonotes release 4.0, LDC2011T03, Philadelphia, Penn.: Linguistic Data Consortium (2011). 29
- [332] D. Vilares, C. Gómez-Rodríguez, Head-qa: A healthcare dataset for complex reasoning, arXiv preprint arXiv:1906.04701 (2019). 29
- [333] S. L. Blodgett, L. Green, B. O’Connor, Demographic dialectal variation in social media: A case study of african-american english, arXiv preprint arXiv:1608.08868 (2016). 29
- [334] N. Mostafazadeh, N. Chambers, X. He, D. Parikh, D. Batra, L. Vanderwende, P. Kohli, J. Allen, A corpus and evaluation framework for deeper understanding of commonsense stories, arXiv preprint arXiv:1604.01696 (2016). 28, 29
- [335] D. Paperno, G. Kruszewski, A. Lazaridou, Q. N. Pham, R. Bernardi, S. Pezzelle, M. Baroni, G. Boleda, R. Fernández, The lambda dataset: Word prediction requiring a broad discourse context, arXiv preprint arXiv:1606.06031 (2016). 28, 29
- [336] B. Hu, Q. Chen, F. Zhu, Lcsts: A large scale chinese short text summarization dataset, arXiv preprint arXiv:1506.05865 (2015). 29
- [337] Z. Shao, M. Huang, J. Wen, W. Xu, X. Zhu, Long and diverse text generation with planning-based hierarchical variational model, arXiv preprint arXiv:1908.06605 (2019). 29
- [338] J. Novikova, O. Dušek, V. Rieser, The e2e dataset: New challenges for end-to-end generation, arXiv preprint arXiv:1706.09254 (2017). 29
- [339] C. Zheng, M. Huang, A. Sun, Chid: A large-scale chinese idiom dataset for cloze test, arXiv preprint arXiv:1906.01265 (2019). 29
- [340] Y. Bisk, R. Zellers, J. Gao, Y. Choi, et al., Piqa: Reasoning about physical commonsense in natural language, in: Proceedings of the AAAI conference on artificial intelligence, Vol. 34, 2020, pp. 7432–7439. 28, 29
- [341] M. Joshi, E. Choi, D. S. Weld, L. Zettlemoyer, Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension, arXiv preprint arXiv:1705.03551 (2017). 28, 29, 31
- [342] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, O. Tafjord, Think you have solved question answering? try arc, the ai2 reasoning challenge, arXiv preprint arXiv:1803.05457 (2018). 28, 29, 31
- [343] S. Aroca-Ouellette, C. Paik, A. Roncone, K. Kann, Prost: Physical reasoning of objects through space and time, arXiv preprint arXiv:2106.03634 (2021). 29
- [344] T. Mihaylov, P. Clark, T. Khot, A. Sabharwal, Can a suit of armor conduct electricity? a new dataset for open book question answering, arXiv preprint arXiv:1809.02789 (2018). 29
- [345] T. C. Ferreira, C. Gardent, N. Illykh, C. Van Der Lee, S. Mille, D. Moussallem, A. Shimorina, The 2020 bilingual, bi-directional webnlg+ shared task overview and evaluation results (webnlg+ 2020), in: Proceedings of the 3rd International Workshop on Natural Language Generation from the Semantic Web (WebNLG+), 2020. 29
- [346] C. Xu, W. Zhou, T. Ge, K. Xu, J. McAuley, F. Wei, Blow the dog whistle: A chinese dataset for cant understanding with common sense and world knowledge, arXiv preprint arXiv:2104.02704 (2021). 29
- [347] G. Lai, Q. Xie, H. Liu, Y. Yang, E. Hovy, Race: Large-scale reading comprehension dataset from examinations, arXiv preprint arXiv:1704.04683 (2017). 29
- [348] E. Choi, H. He, M. Iyyer, M. Yatskar, W.-t. Yih, Y. Choi, P. Liang, L. Zettlemoyer, Quac: Question answering in context, arXiv preprint arXiv:1808.07036 (2018). 29
- [349] M. Geva, D. Khashabi, E. Segal, T. Khot, D. Roth, J. Berant, Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies, Transactions of the Association for Computational Linguistics 9 (2021) 346–361. 29, 31
- [350] J. Boyd-Graber, B. Satinoff, H. He, H. Daumé III, Besting the quiz master: Crowdsourcing incremental classification games, in: Proceedings of the 2012 joint conference on empirical methods in natural language processing and computational natural language learning, 2012, pp. 1290–1301. 29
- [351] S. Zhang, X. Zhang, H. Wang, J. Cheng, P. Li, Z. Ding, Chinese medical question answer matching using end-to-end character-level multi-scale cnns, Applied Sciences 7 (8) (2017) 767. 29
- [352] S. Zhang, X. Zhang, H. Wang, L. Guo, S. Liu, Multi-scale attentive interaction networks for chinese medical question answer selection, IEEE

- Access 6 (2018) 74061–74071. [29](#)
- [353] C. Xu, J. Pei, H. Wu, Y. Liu, C. Li, Matinf: A jointly labeled large-scale dataset for classification, question answering and summarization, arXiv preprint arXiv:2004.12302 (2020). [29](#)
- [354] K. Sakaguchi, R. L. Bras, C. Bhagavatula, Y. Choi, Winogrande: An adversarial winograd schema challenge at scale, *Communications of the ACM* 64 (9) (2021) 99–106. [27, 29](#)
- [355] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, Y. Choi, Hellaswag: Can a machine really finish your sentence?, arXiv preprint arXiv:1905.07830 (2019). [29](#)
- [356] M. Roemmele, C. A. Bejan, A. S. Gordon, Choice of plausible alternatives: An evaluation of commonsense causal reasoning., in: AAAI spring symposium: logical formalizations of commonsense reasoning, 2011, pp. 90–95. [29](#)
- [357] H. Levesque, E. Davis, L. Morgenstern, The winograd schema challenge, in: Thirteenth international conference on the principles of knowledge representation and reasoning, 2012. [27, 29](#)
- [358] A. Talmor, J. Herzig, N. Lourie, J. Berant, Commonsenseqa: A question answering challenge targeting commonsense knowledge, arXiv preprint arXiv:1811.00937 (2018). [29, 31](#)
- [359] M. Sap, H. Rashkin, D. Chen, R. LeBras, Y. Choi, Socialqa: Commonsense reasoning about social interactions, arXiv preprint arXiv:1904.09728 (2019). [29](#)
- [360] K. Sun, D. Yu, D. Yu, C. Cardie, Investigating prior knowledge for challenging chinese machine reading comprehension, *Transactions of the Association for Computational Linguistics* 8 (2020) 141–155. [29](#)
- [361] S. Zhang, X. Liu, J. Liu, J. Gao, K. Duh, B. Van Durme, Record: Bridging the gap between human and machine commonsense reading comprehension, arXiv preprint arXiv:1810.12885 (2018). [29](#)
- [362] P. Rajpurkar, J. Zhang, K. Lopyrev, P. Liang, Squad: 100,000+ questions for machine comprehension of text, arXiv preprint arXiv:1606.05250 (2016). [29, 31](#)
- [363] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, K. Toutanova, Boolq: Exploring the surprising difficulty of natural yes/no questions, arXiv preprint arXiv:1905.10044 (2019). [29, 31](#)
- [364] P. Rajpurkar, R. Jia, P. Liang, Know what you don’t know: Unanswerable questions for squad, arXiv preprint arXiv:1806.03822 (2018). [29, 31](#)
- [365] D. Dua, Y. Wang, P. Dasigi, G. Stanovsky, S. Singh, M. Gardner, Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs, arXiv preprint arXiv:1903.00161 (2019). [29, 31](#)
- [366] I. Dagan, O. Glickman, B. Magnini, The pascal recognising textual entailment challenge, in: Machine learning challenges workshop, Springer, 2005, pp. 177–190. [29, 31](#)
- [367] Y. Chang, M. Narang, H. Suzuki, G. Cao, J. Gao, Y. Bisk, Webqa: Multitop and multimodal qa, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 16495–16504. [29, 31](#)
- [368] Y. Cui, T. Liu, Z. Chen, W. Ma, S. Wang, G. Hu, Dataset for the first evaluation on chinese machine reading comprehension, arXiv preprint arXiv:1709.08299 (2017). [29](#)
- [369] Y. Cui, T. Liu, W. Che, L. Xiao, Z. Chen, W. Ma, S. Wang, G. Hu, A span-extraction dataset for chinese machine reading comprehension, arXiv preprint arXiv:1810.07366 (2018). [29, 31](#)
- [370] Y. Cui, T. Liu, Z. Yang, Z. Chen, W. Ma, W. Che, S. Wang, G. Hu, A sentence cloze dataset for chinese machine reading comprehension, arXiv preprint arXiv:2004.03116 (2020). [29](#)
- [371] Y. Li, T. Liu, D. Li, Q. Li, J. Shi, Y. Wang, Character-based bilstm-crf incorporating pos and dictionaries for chinese opinion target extraction, in: Asian Conference on Machine Learning, PMLR, 2018, pp. 518–533. [29](#)
- [372] D. Khashabi, S. Chaturvedi, M. Roth, S. Upadhyay, D. Roth, Looking beyond the surface: A challenge set for reading comprehension over multiple sentences, in: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers), 2018, pp. 252–262. [29](#)
- [373] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, et al., Natural questions: A benchmark for question answering research, *Transactions of the Association for Computational Linguistics* 7 (2019) 453–466. [29](#)
- [374] C. C. Shao, T. Liu, Y. Lai, Y. Tseng, S. Tsai, Drcd: A chinese machine reading comprehension dataset, arXiv preprint arXiv:1806.00920 (2018). [29](#)
- [375] W. He, K. Liu, J. Liu, Y. Lyu, S. Zhao, X. Xiao, Y. Liu, Y. Wang, H. Wu, Q. She, et al., Dureader: a chinese machine reading comprehension dataset from real-world applications, arXiv preprint arXiv:1711.05073 (2017). [29](#)
- [376] H. Tang, J. Liu, H. Li, Y. Hong, H. Wu, H. Wang, Dureaderrobust: A chinese dataset towards evaluating the robustness of machine reading comprehension models, arXiv preprint arXiv:2004.11142 (2020). [29](#)
- [377] J. Welbl, N. F. Liu, M. Gardner, Crowdsourcing multiple choice science questions, arXiv preprint arXiv:1707.06209 (2017). [29](#)
- [378] C. Xiong, Z. Dai, J. Callan, Z. Liu, R. Power, End-to-end neural ad-hoc ranking with kernel pooling, in: Proceedings of the 40th International ACM SIGIR conference on research and development in information retrieval, 2017, pp. 55–64. [29](#)
- [379] A. Peñas, E. Hovy, P. Forner, Á. Rodrigo, R. Sutcliffe, R. Morante, Qa4mre 2011–2013: Overview of question answering for machine reading evaluation, in: Information Access Evaluation. Multilinguality, Multimodality, and Visualization: 4th International Conference of the CLEF Initiative, CLEF 2013, Valencia, Spain, September 23–26, 2013. Proceedings 4, Springer, 2013, pp. 303–320. [29](#)
- [380] S. Lim, M. Kim, J. Lee, Korquad1.0: Korean qa dataset for machine reading comprehension, arXiv preprint arXiv:1909.07005 (2019). [29](#)
- [381] C. Xiao, H. Zhong, Z. Guo, C. Tu, Z. Liu, M. Sun, Y. Feng, X. Han, Z. Hu, H. Wang, et al., Cail2018: A large-scale legal dataset for judgment prediction, arXiv preprint arXiv:1807.02478 (2018). [29](#)
- [382] D. Hendrycks, S. Basart, S. Kadavath, M. Mazeika, A. Arora, E. Guo, C. Burns, S. Puranik, H. He, D. Song, et al., Measuring coding challenge competence with apps, arXiv preprint arXiv:2105.09938 (2021). [29, 31](#)
- [383] Y. Wang, X. Liu, S. Shi, Deep neural solver for math word problems, in: Proceedings of the 2017 conference on empirical methods in natural language processing, 2017, pp. 845–854. [29, 31](#)
- [384] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al., Training verifiers to solve math word problems, arXiv preprint arXiv:2110.14168 (2021). [29, 31](#)
- [385] J. Austin, A. Odena, M. I. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. J. Cai, M. Terry, Q. V. Le, C. Sutton, Program synthesis with large language models, *CoRR* abs/2108.07732 (2021). [29](#)
- [386] F. Shi, M. Suzgun, M. Freitag, X. Wang, S. Srivats, S. Vosoughi, H. W. Chung, Y. Tay, S. Ruder, D. Zhou, et al., Language models are multilingual chain-of-thought reasoners, arXiv preprint arXiv:2210.03057 (2022). [29](#)
- [387] S. Roy, D. Roth, Solving general arithmetic word problems, arXiv preprint arXiv:1608.01413 (2016). [29](#)
- [388] S.-Y. Miao, C.-C. Liang, K.-Y. Su, A diverse corpus for evaluating and developing english math word problem solvers, arXiv preprint arXiv:2106.15772 (2021). [29](#)
- [389] R. Koncel-Kedziorski, S. Roy, A. Amini, N. Kushman, H. Hajishirzi, Mawps: A math word problem repository, in: Proceedings of the 2016 conference of the north american chapter of the association for computational linguistics: human language technologies, 2016, pp. 1152–1157. [29](#)
- [390] A. Patel, S. Bhattamishra, N. Goyal, Are nlp models really able to solve simple math word problems?, arXiv preprint arXiv:2103.07191 (2021). [29](#)
- [391] Y. Lai, C. Li, Y. Wang, T. Zhang, R. Zhong, L. Zettlemoyer, W.-t. Yih, D. Fried, S. Wang, T. Yu, Ds-1000: A natural and reliable benchmark for data science code generation, in: International Conference on Machine Learning, PMLR, 2023, pp. 18319–18345. [29](#)
- [392] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le, et al., Program synthesis with large language models, arXiv preprint arXiv:2108.07732 (2021). [29](#)
- [393] Y. Nie, A. Williams, E. Dinan, M. Bansal, J. Weston, D. Kiela, Adversarial nli: A new benchmark for natural language understanding, arXiv preprint arXiv:1910.14599 (2019). [29, 31](#)
- [394] A. Williams, N. Nangia, S. R. Bowman, A broad-coverage challenge corpus for sentence understanding through inference, arXiv preprint arXiv:1704.05426 (2017). [29](#)
- [395] R. T. McCoy, E. Pavlick, T. Linzen, Right for the wrong reasons: Diag-

- nosing syntactic heuristics in natural language inference, arXiv preprint arXiv:1902.01007 (2019). 29
- [396] J. Liu, L. Cui, H. Liu, D. Huang, Y. Wang, Y. Zhang, Logiqa: A challenge dataset for machine reading comprehension with logical reasoning, arXiv preprint arXiv:2007.08124 (2020). 29
- [397] P. Lewis, B. Öğuz, R. Rinott, S. Riedel, H. Schwenk, Mlqa: Evaluating cross-lingual extractive question answering, arXiv preprint arXiv:1910.07475 (2019). 29
- [398] A. Conneau, G. Lample, R. Rinott, A. Williams, S. R. Bowman, H. Schwenk, V. Stoyanov, Xnli: Evaluating cross-lingual sentence representations, arXiv preprint arXiv:1809.05053 (2018). 29, 31
- [399] Y. Yang, Y. Zhang, C. Tar, J. Baldridge, Paws-x: A cross-lingual adversarial dataset for paraphrase identification, arXiv preprint arXiv:1908.11828 (2019). 29, 31
- [400] S. Narayan, S. B. Cohen, M. Lapata, Don't give me the details, just the summary!, Topic-Aware Convolutional Neural Networks for Extreme Summarization. ArXiv, abs (1808). 29
- [401] E. M. Ponti, G. Glavaš, O. Majewska, Q. Liu, I. Vulić, A. Korhonen, Xcopa: A multilingual dataset for causal commonsense reasoning, arXiv preprint arXiv:2005.00333 (2020). 29
- [402] A. Tikhonov, M. Ryabinin, It's all in the heads: Using attention heads as a baseline for cross-lingual transfer in commonsense reasoning, arXiv preprint arXiv:2106.12066 (2021). 29
- [403] J. H. Clark, E. Choi, M. Collins, D. Garrette, T. Kwiatkowski, V. Nikolaev, J. Palomaki, Tydi qa: A benchmark for information-seeking question answering in typologically diverse languages, Transactions of the Association for Computational Linguistics 8 (2020) 454–470. 29
- [404] T. Scialom, P.-A. Dray, S. Lamprier, B. Piwowarski, J. Staiano, Msum: The multilingual summarization corpus, arXiv preprint arXiv:2004.14900 (2020). 29
- [405] S. Lin, J. Hilton, O. Evans, Truthfulqa: Measuring how models mimic human falsehoods, arXiv preprint arXiv:2109.07958 (2021). 29, 32
- [406] I. Augenstein, C. Lioma, D. Wang, L. C. Lima, C. Hansen, C. Hansen, J. G. Simonsen, Multifc: A real-world multi-domain dataset for evidence-based fact checking of claims, arXiv preprint arXiv:1909.03242 (2019). 29
- [407] J. Thorne, A. Vlachos, C. Christodoulopoulos, A. Mittal, Fever: a large-scale dataset for fact extraction and verification, arXiv preprint arXiv:1803.05355 (2018). 29
- [408] I. Mollas, Z. Chrysopoulou, S. Karlos, G. Tsoumakas, Ethos: an online hate speech detection dataset, arXiv preprint arXiv:2006.08328 (2020). 29, 32
- [409] M. Nadeem, A. Bethke, S. Reddy, Stereoset: Measuring stereotypical bias in pretrained language models, arXiv preprint arXiv:2004.09456 (2020). 29, 32
- [410] A. Parrish, A. Chen, N. Nangia, V. Padmakumar, J. Phang, J. Thompson, P. M. Htut, S. R. Bowman, Bbq: A hand-built bias benchmark for question answering, arXiv preprint arXiv:2110.08193 (2021). 29
- [411] J. Zhao, T. Wang, M. Yatskar, V. Ordonez, K.-W. Chang, Gender bias in coreference resolution: Evaluation and debiasing methods, arXiv preprint arXiv:1804.06876 (2018). 29
- [412] N. Nangia, C. Vania, R. Bhalerao, S. R. Bowman, Crows-pairs: A challenge dataset for measuring social biases in masked language models, arXiv preprint arXiv:2010.00133 (2020). 29
- [413] S. Gehman, S. Gururangan, M. Sap, Y. Choi, N. A. Smith, Realtoxicityprompts: Evaluating neural toxic degeneration in language models, arXiv preprint arXiv:2009.11462 (2020). 29
- [414] D. Borkan, L. Dixon, J. Sorensen, N. Thain, L. Vasserman, Nuanced metrics for measuring unintended bias with real data for text classification, in: Companion proceedings of the 2019 world wide web conference, 2019, pp. 491–500. 29
- [415] O. Bojar, R. Chatterjee, C. Federmann, Y. Graham, B. Haddow, M. Huck, A. J. Yepes, P. Koehn, V. Logacheva, C. Monz, et al., Findings of the 2016 conference on machine translation, in: Proceedings of the First Conference on Machine Translation: Volume 2, Shared Task Papers, 2016, pp. 131–198. 29
- [416] B. Loïc, B. Magdalena, B. Ondřej, F. Christian, G. Yvette, G. Roman, H. Barry, H. Matthias, J. Eric, K. Tom, et al., Findings of the 2020 conference on machine translation (wmt20), in: Proceedings of the Fifth Conference on Machine Translation, Association for Computational Linguistics, 2020, pp. 1–55. 29
- [417] W. Li, F. Qi, M. Sun, X. Yi, J. Zhang, Ccpm: A chinese classical poetry matching dataset, arXiv preprint arXiv:2106.01979 (2021). 29
- [418] E. Dinan, S. Roller, K. Shuster, A. Fan, M. Auli, J. Weston, Wizard of wikipedia: Knowledge-powered conversational agents, arXiv preprint arXiv:1811.01241 (2018). 29
- [419] H. Rashkin, E. M. Smith, M. Li, Y.-L. Boureau, Towards empathetic open-domain conversation models: A new benchmark and dataset, arXiv preprint arXiv:1811.00207 (2018). 29
- [420] E. Dinan, V. Logacheva, V. Malykh, A. Miller, K. Shuster, J. Urbanek, D. Kiela, A. Szlam, I. Serban, R. Lowe, et al., The second conversational intelligence challenge (convai2), in: The NeurIPS'18 Competition: From Machine Learning to Intelligent Conversations, Springer, 2020, pp. 187–208. 29
- [421] H. Zhou, C. Zheng, K. Huang, M. Huang, X. Zhu, Kdconv: A chinese multi-domain dialogue dataset towards multi-turn knowledge-driven conversation, arXiv preprint arXiv:2004.04100 (2020). 29
- [422] L. CO, Iflytek: a multiple categories chinese text classifier. competition official website (2019). 29
- [423] J. Baumgartner, S. Zannettou, B. Keegan, M. Squire, J. Blackburn, The pushshift reddit dataset, in: Proceedings of the international AAAI conference on web and social media, Vol. 14, 2020, pp. 830–839. 30
- [424] A. Fan, Y. Jernite, E. Perez, D. Grangier, J. Weston, M. Auli, Eli5: Long form question answering, arXiv preprint arXiv:1907.09190 (2019). 31
- [425] Y. Wang, S. Mishra, P. Alipoormolabashi, Y. Kordi, A. Mirzaei, A. Arunkumar, A. Ashok, A. S. Dhanasekaran, A. Naik, D. Stap, et al., Benchmarking generalization via in-context instructions on 1,600+ language tasks, arXiv preprint arXiv:2204.07705 (2022). 31
- [426] T. Xie, C. H. Wu, P. Shi, R. Zhong, T. Scholak, M. Yasunaga, C.-S. Wu, M. Zhong, P. Yin, S. I. Wang, et al., Unifedskg: Unifying and multi-tasking structured knowledge grounding with text-to-text language models, arXiv preprint arXiv:2201.05966 (2022). 31
- [427] Q. Ye, B. Y. Lin, X. Ren, Crossfit: A few-shot learning challenge for cross-task generalization in nlp, arXiv preprint arXiv:2104.08835 (2021). 31
- [428] V. Aribandi, Y. Tay, T. Schuster, J. Rao, H. S. Zheng, S. V. Mehta, H. Zhuang, V. Q. Tran, D. Bahri, J. Ni, et al., Ext5: Towards extreme multi-task scaling for transfer learning, arXiv preprint arXiv:2111.10952 (2021). 31
- [429] A. Williams, N. Nangia, S. Bowman, A broad-coverage challenge corpus for sentence understanding through inference, in: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers), Association for Computational Linguistics, New Orleans, Louisiana, 2018, pp. 1112–1122. doi:10.18653/v1/N18-1101. URL <https://aclanthology.org/N18-1101> 31
- [430] Y. Zhang, J. Baldridge, L. He, PAWS: Paraphrase adversaries from word scrambling, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers), Association for Computational Linguistics, Minneapolis, Minnesota, 2019, pp. 1298–1308. doi:10.18653/v1/N19-1131. URL <https://aclanthology.org/N19-1131> 32
- [431] C. Qin, A. Zhang, Z. Zhang, J. Chen, M. Yasunaga, D. Yang, Is chatGPT a general-purpose natural language processing task solver?, in: The 2023 Conference on Empirical Methods in Natural Language Processing, 2023. URL <https://openreview.net/forum?id=u03xn1C0s0> 32
- [432] M. U. Hadi, R. Qureshi, A. Shah, M. Irfan, A. Zafar, M. B. Shaikh, N. Akhtar, J. Wu, S. Mirjalili, et al., Large language models: a comprehensive survey of its applications, challenges, limitations, and future prospects, TechRxiv (2023). 32
- [433] X. L. Dong, S. Moon, Y. E. Xu, K. Malik, Z. Yu, Towards next-generation intelligent assistants leveraging llm techniques, in: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, 2023, pp. 5792–5793. 32
- [434] K. Pandya, M. Holia, Automating customer service using langchain: Building custom open-source gpt chatbot for organizations, arXiv preprint arXiv:2310.05421 (2023). 32
- [435] J. Li, B. Hui, G. Qu, B. Li, J. Yang, B. Li, B. Wang, B. Qin, R. Cao, R. Geng, et al., Can llm already serve as a database interface? a

- big bench for large-scale database grounded text-to-sqls, arXiv preprint arXiv:2305.03111 (2023). 32
- [436] A. Rao, J. Kim, M. Kamineneni, M. Pang, W. Lie, M. D. Succi, Evaluating chatgpt as an adjunct for radiologic decision-making, medRxiv (2023) 2023-02. 32
- [437] M. Benary, X. D. Wang, M. Schmidt, D. Soll, G. Hilfenhaus, M. Nassir, C. Sigler, M. Knödler, U. Keller, D. Beule, et al., Leveraging large language models for decision support in personalized oncology, JAMA Network Open 6 (11) (2023) e2343689–e2343689. 32
- [438] C. M. Chiesa-Estomba, J. R. Lechien, L. A. Vaira, A. Brunet, G. Cammaroto, M. Mayo-Yanez, A. Sanchez-Barrueco, C. Saga-Gutierrez, Exploring the potential of chat-gpt as a supportive tool for sialendoscopy clinical decision making and patient information support, European Archives of Oto-Rhino-Laryngology (2023) 1–6. 32
- [439] S. Montagna, S. Ferretti, L. C. Klopfenstein, A. Florio, M. F. Pengo, Data decentralisation of llm-based chatbot systems in chronic disease self-management, in: Proceedings of the 2023 ACM Conference on Information Technology for Social Good, 2023, pp. 205–212. 32
- [440] D. Bill, T. Eriksson, Fine-tuning a llm using reinforcement learning from human feedback for a therapy chatbot application (2023). 32
- [441] M. Abbasian, I. Azimi, A. M. Rahmani, R. Jain, Conversational health agents: A personalized llm-powered agent framework, arXiv preprint arXiv:2310.02374 (2023). 32
- [442] K. V. Lemley, Does chatgpt help us understand the medical literature?, Journal of the American Society of Nephrology (2023) 10–1681. 32
- [443] S. Pal, M. Bhattacharya, S.-S. Lee, C. Chakraborty, A domain-specific next-generation large language model (llm) or chatgpt is required for biomedical engineering and research, Annals of Biomedical Engineering (2023) 1–4. 32
- [444] Y. Du, S. Zhao, Y. Chen, R. Bai, J. Liu, H. Wu, H. Wang, B. Qin, The calla dataset: Probing llms’ interactive knowledge acquisition from chinese medical literature, arXiv preprint arXiv:2309.04198 (2023). 32
- [445] A. Abd-Alrazaq, R. AlSaad, D. Alhuwail, A. Ahmed, P. M. Healy, S. Latifi, S. Aziz, R. Damseh, S. A. Alrazak, J. Sheikh, et al., Large language models in medical education: Opportunities, challenges, and future directions, JMIR Medical Education 9 (1) (2023) e48291. 32
- [446] A. B. Mbakwe, I. Lourentzou, L. A. Celi, O. J. Mechanic, A. Dagan, Chatgpt passing usmle shines a spotlight on the flaws of medical education (2023). 32
- [447] S. Ahn, The impending impacts of large language models on medical education, Korean Journal of Medical Education 35 (1) (2023) 103. 32
- [448] E. Waisberg, J. Ong, M. Masalkhi, A. G. Lee, Large language model (llm)-driven chatbots for neuro-ophthalmic medical education, Eye (2023) 1–3. 32
- [449] G. Deiana, M. Dettori, A. Arghittu, A. Azara, G. Gabutti, P. Castiglia, Artificial intelligence and public health: Evaluating chatgpt responses to vaccination myths and misconceptions, Vaccines 11 (7) (2023) 1217. 32
- [450] L. De Angelis, F. Baglivo, G. Arzilli, G. P. Privitera, P. Ferragina, A. E. Tozzi, C. Rizzo, Chatgpt and the rise of large language models: the new ai-driven infodemic threat in public health, Frontiers in Public Health 11 (2023) 1166120. 32
- [451] N. L. Rane, A. Tawde, S. P. Choudhary, J. Rane, Contribution and performance of chatgpt and other large language models (llm) for scientific and research advancements: a double-edged sword, International Research Journal of Modernization in Engineering Technology and Science 5 (10) (2023) 875–899. 32
- [452] W. Dai, J. Lin, H. Jin, T. Li, Y.-S. Tsai, D. Gašević, G. Chen, Can large language models provide feedback to students? a case study on chatgpt, in: 2023 IEEE International Conference on Advanced Learning Technologies (ICALT), IEEE, 2023, pp. 323–325. 32
- [453] E. Kasneci, K. Seßler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günemann, E. Hüllermeier, et al., Chatgpt for good? on opportunities and challenges of large language models for education, Learning and individual differences 103 (2023) 102274. 32
- [454] N. Rane, Enhancing the quality of teaching and learning through chatgpt and similar large language models: Challenges, future prospects, and ethical considerations in education, Future Prospects, and Ethical Considerations in Education (September 15, 2023) (2023). 32
- [455] J. C. Young, M. Shishido, Investigating openai’s chatgpt potentials in generating chatbot’s dialogue for english as a foreign language learning, International Journal of Advanced Computer Science and Applications 14 (6) (2023). 32
- [456] J. Irons, C. Mason, P. Cooper, S. Sidra, A. Reeson, C. Paris, Exploring the impacts of chatgpt on future scientific work, SocArXiv (2023). 32
- [457] P. G. Schmidt, A. J. Meir, Using generative ai for literature searches and scholarly writing: Is the integrity of the scientific discourse in jeopardy?, arXiv preprint arXiv:2311.06981 (2023). 32
- [458] Y. Zheng, H. Y. Koh, J. Ju, A. T. Nguyen, L. T. May, G. I. Webb, S. Pan, Large language models for scientific synthesis, inference and explanation, arXiv preprint arXiv:2310.07984 (2023). 33
- [459] B. Aczel, E.-J. Wagenmakers, Transparency guidance for chatgpt usage in scientific writing, PsyArXiv (2023). 33
- [460] S. Altmäe, A. Sola-Leyva, A. Salumets, Artificial intelligence in scientific writing: a friend or a foe?, Reproductive BioMedicine Online (2023). 33
- [461] S. Imani, L. Du, H. Shrivastava, Mathprompter: Mathematical reasoning using large language models, arXiv preprint arXiv:2303.05398 (2023). 33
- [462] Z. Yuan, H. Yuan, C. Li, G. Dong, C. Tan, C. Zhou, Scaling relationship on learning mathematical reasoning with large language models, arXiv preprint arXiv:2308.01825 (2023). 33
- [463] K. Yang, A. M. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. Prenger, A. Anandkumar, Leandjo: Theorem proving with retrieval-augmented language models, arXiv preprint arXiv:2306.15626 (2023). 33
- [464] K. M. Collins, A. Q. Jiang, S. Frieder, L. Wong, M. Zilka, U. Bhatt, T. Lukasiewicz, Y. Wu, J. B. Tenenbaum, W. Hart, et al., Evaluating language models for mathematics through interactions, arXiv preprint arXiv:2306.01694 (2023). 33
- [465] Y. Liu, T. Han, S. Ma, J. Zhang, Y. Yang, J. Tian, H. He, A. Li, M. He, Z. Liu, et al., Summary of chatgpt-related research and perspective towards the future of large language models, Meta-Radiology (2023) 100017. 33
- [466] J. Drápal, H. Westermann, J. Savelka, Using large language models to support thematic analysis in empirical legal studies, arXiv preprint arXiv:2310.18729 (2023). 33
- [467] J. Savelka, K. D. Ashley, M. A. Gray, H. Westermann, H. Xu, Explaining legal concepts with augmented large language models (gpt-4), arXiv preprint arXiv:2306.09525 (2023). 33
- [468] N. Guha, J. Nyarko, D. E. Ho, C. Ré, A. Chilton, A. Narayana, A. Chohlas-Wood, A. Peters, B. Waldon, D. N. Rockmore, et al., Legal-bench: A collaboratively built benchmark for measuring legal reasoning in large language models, arXiv preprint arXiv:2308.11462 (2023). 33
- [469] J. Cui, Z. Li, Y. Yan, B. Chen, L. Yuan, Chatlaw: Open-source legal large language model with integrated external knowledge bases, arXiv preprint arXiv:2306.16092 (2023). 33
- [470] H. Yang, X.-Y. Liu, C. D. Wang, Fingpt: Open-source financial large language models, arXiv preprint arXiv:2306.06031 (2023). 33
- [471] Y. Li, S. Wang, H. Ding, H. Chen, Large language models in finance: A survey, in: Proceedings of the Fourth ACM International Conference on AI in Finance, 2023, pp. 374–382. 33
- [472] A. Lykov, D. Tsetserukou, Llm-brain: Ai-driven fast generation of robot behaviour tree based on large language model, arXiv preprint arXiv:2305.19352 (2023). 33
- [473] E. Billing, J. Rosén, M. Lamb, Language models for human-robot interaction, in: ACM/IEEE International Conference on Human-Robot Interaction, March 13–16, 2023, Stockholm, Sweden, ACM Digital Library, 2023, pp. 905–906. 33
- [474] Y. Ye, H. You, J. Du, Improved trust in human-robot collaboration with chatgpt, IEEE Access (2023). 33
- [475] Y. Ding, X. Zhang, C. Paxton, S. Zhang, Leveraging commonsense knowledge from large language models for task and motion planning, in: RSS 2023 Workshop on Learning for Task and Motion Planning, 2023. 33
- [476] J. Wu, R. Antonova, A. Kan, M. Lepert, A. Zeng, S. Song, J. Bohg, S. Rusinkiewicz, T. Funkhouser, Tidybot: Personalized robot assistance with large language models, arXiv preprint arXiv:2305.05658 (2023). 33
- [477] E. Strubell, A. Ganesh, A. McCallum, Energy and policy considerations for deep learning in nlp, arXiv preprint arXiv:1906.02243 (2019). 34
- [478] E. M. Bender, T. Gebru, A. McMillan-Major, S. Shmitchell, On the dan-

- gers of stochastic parrots: Can language models be too big?, in: Proceedings of the 2021 ACM conference on fairness, accountability, and transparency, 2021, pp. 610–623. [34](#)
- [479] C. Zhang, S. Bengio, M. Hardt, B. Recht, O. Vinyals, Understanding deep learning (still) requires rethinking generalization, *Communications of the ACM* 64 (3) (2021) 107–115. [34](#)
- [480] M. Tänzler, S. Ruder, M. Rei, Memorisation versus generalisation in pre-trained language models, *arXiv preprint arXiv:2105.00828* (2021). [34](#)
- [481] S. M. West, M. Whittaker, K. Crawford, Discriminating systems, *AI Now* (2019) 1–33. [34](#)
- [482] K. Valmeekam, A. Olmo, S. Sreedharan, S. Kambhampati, Large language models still can’t plan (a benchmark for llms on planning and reasoning about change), *arXiv preprint arXiv:2206.10498* (2022). [34](#)
- [483] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, et al., Siren’s song in the ai ocean: A survey on hallucination in large language models, *arXiv preprint arXiv:2309.01219* (2023). [34](#)
- [484] A. Webson, E. Pavlick, Do prompt-based models really understand the meaning of their prompts?, *arXiv preprint arXiv:2109.01247* (2021). [34](#)
- [485] O. Shaikh, H. Zhang, W. Held, M. Bernstein, D. Yang, On second thought, let’s not think step by step! bias and toxicity in zero-shot reasoning, *arXiv preprint arXiv:2212.08061* (2022). [34](#)
- [486] B. C. Das, M. H. Amini, Y. Wu, Security and privacy challenges of large language models: A survey, *arXiv preprint arXiv:2402.00888* (2024). [34](#)
- [487] X. Liu, H. Cheng, P. He, W. Chen, Y. Wang, H. Poon, J. Gao, *Adversarial training for large neural language models*, *ArXiv* (April 2020).
URL <https://www.microsoft.com/en-us/research/publication/adversarial-training-for-large-neural-language-models/>
[34](#)
- [488] E. Shayegani, M. A. A. Mamun, Y. Fu, P. Zaree, Y. Dong, N. Abu-Ghazaleh, Survey of vulnerabilities in large language models revealed by adversarial attacks (2023). [arXiv:2310.10844](#). [34](#)
- [489] X. Xu, K. Kong, N. Liu, L. Cui, D. Wang, J. Zhang, M. Kankanhalli, An llm can fool itself: A prompt-based adversarial attack (2023). [arXiv:2310.13345](#). [34](#)
- [490] H. Zhao, H. Chen, F. Yang, N. Liu, H. Deng, H. Cai, S. Wang, D. Yin, M. Du, Explainability for large language models: A survey (2023). [arXiv:2309.01029](#). [35](#)
- [491] S. Huang, S. Mamidanna, S. Jangam, Y. Zhou, L. H. Gilpin, Can large language models explain themselves? a study of llm-generated self-explanations (2023). [arXiv:2310.11207](#). [35](#)
- [492] H. Brown, K. Lee, F. Miresghallah, R. Shokri, F. Tramèr, What does it mean for a language model to preserve privacy?, in: Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency, 2022, pp. 2280–2292. [35](#)
- [493] R. Plant, V. Giuffrida, D. Gkatzia, You are what you write: Preserving privacy in the era of large language models, *arXiv preprint arXiv:2204.09391* (2022). [35](#)
- [494] W. Niu, Z. Kong, G. Yuan, W. Jiang, J. Guan, C. Ding, P. Zhao, S. Liu, B. Ren, Y. Wang, Real-time execution of large-scale language models on mobile (2020). [arXiv:2009.06823](#). [35](#)
- [495] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y. Liu, M. Guo, Y. Zhu, Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization, in: Proceedings of the 50th Annual International Symposium on Computer Architecture, 2023, pp. 1–15. [35](#)
- [496] B. Meskó, E. J. Topol, The imperative for regulatory oversight of large language models (or generative ai) in healthcare, *npj Digital Medicine* 6 (1) (2023) 120. [35](#)
- [497] J. Zhang, X. Ji, Z. Zhao, X. Hei, K.-K. R. Choo, Ethical considerations and policy implications for large language models: Guiding responsible development and deployment, *arXiv preprint arXiv:2308.02678* (2023). [35](#)
- [498] J. Mökander, J. Schuett, H. R. Kirk, L. Floridi, Auditing large language models: a three-layered approach, *AI and Ethics* (2023) 1–31. [35](#)